Kubernetes Developer Learning Path

Version 1.0.0, 2026-08-13

Home

Introduction

Course Title: Kubernetes Developer Learning Path

Description: Course description FIXME

Duration: FIXME hours

Topics covered in this course

- Overview
- Who is this for? pre-requisites.
- Familiarizing with koo·br·neh·teez
- 1.1 — Kubernetes Resource Model
- 1.2 — Controllers, informers, client-go
- 2.1 — Controllers contd.
- 2.2 — Going distributed: locks, leases, ownership, eventual consistency
- 3.1 — Taking more control: Admissions and beyond the kube-apiserver to writing your own API Server
- 3.2 — The other stuff
- 4.1 — All about kube-scheduler
- 4.2 — Ubiquitous topics!
- Exercises
- Recommended Reading
- Suggested Reading

Prerequisites

This course assumes that you have the following prior experience:

- Course or experience…
- Course or experience…
- Course or experience…

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Overview

This course provides a comprehensive overview of the Kubernetes architecture and its powerful extensibility mechanisms. We will start by learning how the Kubernetes API functions behind the scenes, exploring how cluster requests are authenticated, authorized, and validated by the control plane. The core of the course focuses on platform automation, teaching you how to extend Kubernetes by defining custom resources and writing custom controllers or operators to automatically manage different workloads.

Weekly Roadmap

Week 0: Familiarizing with Kubernetes

At its core, Kubernetes is an API-centric, declarative configuration management and container orchestration system. This week orients you with how the API server, etcd, and controllers work together.

Go to Week 0

Week 1.1: Kubernetes Resource Model

A walkthrough on kube-apiserver essentials — the declarative intent of Kubernetes Resources and all the plugin layers (authentication, audit, impersonation, priority & fairness, authorization, admission, storage).

Go to Week 1.1

Week 1.2: Controllers, Informers, and client-go

The basic building blocks behind operators — client-go architecture, Informers, listers, workqueues, caches, and error handling.

Go to Week 1.2

Week 2.1: Controllers with controller-runtime

Writing concrete Kube APIs with controller-runtime — Reconcile loop, kubebuilder, Spec vs Status, Conditions, API validation, CRUD operations, Apply vs Patch vs Update.

Go to Week 2.1

Week 2.2: Going Distributed

Distributed systems challenges — Idempotency, locks, leases, leader election, controller sharding (with OpenShift Ingress Controller case study).

Go to Week 2.2

Week 3.1: Admissions and Beyond the kube-apiserver

From admission control webhooks into writing your own apiserver — ValidatingWebhooks, MutatingWebhooks, policy engines, API discovery, aggregated APIs, custom API servers.

Go to Week 3.1

Week 3.2: The Other Stuff

Networking (Ingress, Gateway API), Service Mesh, AuthN/AuthZ in OpenShift, Security, Cluster Hardening.

Go to Week 3.2

Week 4.1: All About kube-scheduler

kube-scheduler deep dive — scheduling, de-scheduling, resource requests and limits, Dynamic Resource Allocation.

Go to Week 4.1

Week 4.2: Ubiquitous Topics

Operator landscape and alternative control plane architectures — HyperShift, kcp, MicroShift, k3s.

Go to Week 4.2 # Overview

Overview

The Kubernetes Developer Learning Path provides an architectural deep dive into the platform's extensibility and internal mechanics. It is designed for engineers and developers with existing container experience looking to master platform automation and distributed system design.

Key focus areas include:

- **Architecture & Extensibility:** Understanding the Kubernetes control plane, the role of `kube-apiserver`, and the `etcd` persistence layer.
- **Platform Automation:** Building custom resources and operators using the reconcile loop pattern.
- **Core Mechanics:** Deep dives into `client-go` components, including informers, listers, and workqueues.
- **Distributed Systems:** Implementing idempotent state machines, leader election, and optimistic locking via Leases.
- **Advanced Control Plane:** Exploring API aggregation, admission control webhooks, and modern traffic management paradigms like the Gateway API and Service Mesh.
- **Security & Operations:** Examining AuthN/AuthZ models, cluster hardening techniques, and alternative control plane form factors.

Overview

Kubernetes architecture overview and extensibility mechanisms

Kubernetes Architecture Overview and Extensibility Mechanisms

Kubernetes is far more than a container orchestrator; it is a **distributed operating system for the cloud**. At its core, it functions as a highly available, API-centric, declarative configuration management system. Understanding its architecture is the first step toward mastering the extensibility that makes Kubernetes the industry standard for platform engineering.

The Control Plane: The Brain of the Cluster

The Kubernetes control plane is the central nervous system of the cluster. It makes global decisions about the cluster (e.g., scheduling), detects and responds to cluster events, and maintains the desired state.

- **API Server (`kube-apiserver`):** The gateway to the cluster. All internal and external components communicate through the API server. It exposes the Kubernetes API and serves as the front end for the control plane.
- **etcd:** The consistent and highly-available key-value store used as Kubernetes' backing store for all cluster data.
- **Scheduler (`kube-scheduler`):** Watches for newly created Pods with no assigned node and selects a node for them to run on.
- **Controller Manager (`kube-controller-manager`):** Runs controller processes, which regulate the state of the cluster by reconciling the current state toward the desired state.

The API-Centric Lifecycle

The `kube-apiserver` processes requests through a sophisticated plugin chain. When a user or controller submits a resource (e.g., a `Deployment` or `CustomResourceDefinition`), the request flows through:

1. **Authentication:** Who are you? (e.g., X.509 certs, OIDC).
2. **Audit:** Logging the request for security compliance.
3. **Impersonation:** Checking if the user is authorized to act as another user.
4. **Authorization:** Are you allowed to perform this action? (RBAC).
5. **Admission Control:** Mutating or Validating the resource against business logic before it is persisted.
6. **Storage:** Persistence in `etcd`.

Extensibility: Defining Your Own Reality

The true power of Kubernetes lies in its ability to be extended. You are not limited to the standard resources (Pods, Services, Deployments). You can extend the API to manage your own domain-specific abstractions.

1. Custom Resource Definitions (CRDs)

CRDs allow you to define custom objects in the Kubernetes API. Once a CRD is registered, the API server handles the lifecycle of these custom resources just as it does for native resources.

2. The Controller Pattern

The controller pattern is the mechanism that makes Kubernetes "act." Controllers watch the API server for changes to a specific resource type. When an event occurs, the controller performs a **Reconciliation Loop**:

- **Observe:** Get the current state of the resource.
- **Analyze:** Compare the current state against the desired state defined in the resource `spec`.
- **Act:** Perform operations (e.g., creating Pods, updating status) to close the gap between the two states.

3. Admission Webhooks

While controllers handle "post-persistence" logic, admission webhooks allow you to intercept requests **before** they are persisted to `etcd`. * **Validating Webhooks:** Used to reject a request if it violates your security policies (e.g., "Only allow images from the internal registry"). * **Mutating Webhooks:** Used to inject default values or modify the resource content (e.g., automatically adding sidecars).

Why Extensibility Matters

By leveraging the native Kubernetes control plane patterns, developers can build **Operators**—software that encodes human operational knowledge into code. This allows for: * **Automation:** Managing complex, stateful applications (databases, message queues) as if they were native Kubernetes objects. * **Consistency:** Ensuring every team deploys resources following the same organizational standards. * **Resiliency:** Utilizing the Kubernetes reconciliation loop to ensure systems are self-healing and continuously match the defined desired state.

In the next modules, we will dive deeper into `client-go` and `controller-runtime` to build these patterns from the ground up, moving from simple API interactions to complex, distributed reconciliation logic.

Control plane authentication, authorization, and validation

Control Plane Authentication, Authorization, and Validation

In the Kubernetes architecture, the `kube-apiserver` acts as the central gatekeeper. Before any resource is persisted to `etcd`, the request must pass through a multi-layered security pipeline. Understanding this flow is critical for developers building custom operators or extending the platform.

The API Request Lifecycle

When a user or a service account interacts with the Kubernetes API, the `kube-apiserver` processes the request through three distinct phases:

1. **Authentication (AuthN):** Determining **who** is making the request.
2. **Authorization (AuthZ):** Determining **what** the user is allowed to do.
3. **Admission Control:** Validating or mutating the request content before it is finalized.

---

1. Authentication (AuthN)

The API server relies on one or more authentication modules to verify the caller's identity. If a request cannot be authenticated, the API server rejects it with an HTTP 401 response.

- **X.509 Client Certs:** Managed by passing a CA file to the API server.
- **Static Token Files:** Simple, but generally discouraged for production.
- **OpenID Connect (OIDC):** Integration with external Identity Providers (IdPs) like Dex or Keycloak.
- **Service Account Tokens:** Standard for internal cluster communication.

2. Authorization (AuthZ)

Once the identity is established, the request must be authorized. Kubernetes supports several modes, which are checked in order:

- **RBAC (Role-Based Access Control):** The standard mechanism using `Roles`/`ClusterRoles` and `RoleBindings`/`ClusterRoleBindings` to define permissions based on actions (verbs) and resources.
- **ABAC (Attribute-Based Access Control):** Policies defined via JSON files.
- **Webhook:** The API server delegates the authorization decision to a remote HTTP service.

Note: In managed environments like OpenShift, AuthZ is tightly integrated with the cluster's identity provider, often mapping OIDC groups to internal RBAC roles.

3. Admission Control (Validation & Mutation)

This is the final phase of the lifecycle. Admission Controllers act as plugins that intercept requests after they are authenticated and authorized, but before the object is persisted to `etcd`.

Mutating Admission Controllers

These controllers can modify the object content before it is stored. * **Use case:** Injecting sidecars, setting default values for security contexts, or adding default labels.

Validating Admission Controllers

These controllers check the request against defined policies. If the validation fails, the request is rejected. * **Use case:** Ensuring that container images come from an approved registry, requiring specific labels on Namespaces, or enforcing resource limit ranges.

Practical Perspective: Admission Webhooks

While built-in controllers cover standard scenarios, developers often need to implement custom logic. This is achieved through **Dynamic Admission Webhooks**:

Type	Behavior
`MutatingAdmissionWebhook`	Modifies the object (e.g., adding `imagePullSecrets`).
`ValidatingAdmissionWebhook`	Permits or denies the object based on criteria (e.g., "Must have `owner` label").

Designing for Security

To ensure a robust control plane: * **Fail-closed:** Always configure webhooks to fail closed (deny the request) if the webhook service is unreachable. * **Idempotency:** Admission logic must be idempotent; the same request processed twice should yield the same result without side effects. * **Latency Impact:** Admission webhooks are in the critical path of the API request. Keep logic lightweight to avoid slowing down `kubectl apply` operations.

Summary Checklist for Developers

- ▢ **Identify:** Are you interacting as a user or a ServiceAccount?
- ▢ **Permission:** Does your RBAC definition follow the principle of least privilege?
- ▢ **Validation:** Is your custom resource validation logic handled via CEL (Common Expression Language) or a custom Validating Webhook?
- ▢ **Audit:** Ensure your control plane is configured to output logs so you can audit which identities performed which actions.

Platform automation through custom resources and operators

Platform Automation: Custom Resources and Operators

Overview

Kubernetes is more than a container orchestrator; it is a platform for building platforms. The primary mechanism for this extensibility is the **Kubernetes Resource Model (KRM)**, which allows developers to extend the API to manage domain-specific logic. By leveraging **Custom Resource Definitions (CRDs)** and **Controllers**, you can codify operational knowledge into software, creating what we commonly call an **Operator**.

Custom Resources: Extending the API

A **Custom Resource (CR)** is an extension of the Kubernetes API that allows you to store and retrieve structured data. When you define a CRD, the `kube-apiserver` creates a new RESTful endpoint for your resource.

- **Spec**: Defines the **desired state**—what the user wants the system to look like.
- **Status**: Reflects the **observed state**—what the system currently looks like.

Avoid putting transient data in the `Spec`. The `Spec` should contain configuration that persists even if the controller is restarted.

The Operator Pattern: Automating Operations

An Operator is a software extension that uses custom resources to manage applications and their components. It follows the **Asynchronous Controller Reconciliation Pattern**:

1. **Watch**: The controller registers informers to monitor events on specific resources.
2. **Diff**: When an event occurs, the controller compares the `Spec` (intent) against the actual state of the system.
3. **Act**: The controller executes a series of actions (CRUD operations) to move the system toward the desired state.
4. **Reconcile**: This loop runs continuously, ensuring the system remains in the desired state despite external changes.

Why Custom Controllers?

While `kubectl` and manual manifests work for simple workloads, complex systems require: * **Encapsulation**: Bundling complex installation and configuration logic. * **Self-Healing**: Automatic recovery from failures without human intervention. * **Consistency**: Ensuring every instance of an application is deployed with the same parameters.

Hands-on Lab: Scaffolding a Controller with Kubebuilder

In this lab, we will use `kubebuilder` to generate the boilerplate for a custom operator.

Prerequisites

- Go (latest version)
- `kubectl`

- `kubebuilder` CLI
- A running Kubernetes cluster (e.g., Minikube or Kind)

Step 1: Initialize the Project

Create a new directory and initialize your project:

```
mkdir my-operator
cd my-operator
kubebuilder init --domain example.com --repo github.com/example/my-operator
```

Step 2: Create the API (CRD)

Generate a new API group for your resource (e.g., `AppService`):

```
kubebuilder create api --group webapp --version v1 --kind AppService
```

Step 3: Define the Spec

Edit `api/v1/appservice_types.go` to define the schema:

```
type AppServiceSpec struct {
    // Size is the desired number of replicas
    Size int32 `json:"size,omitempty"`
}

type AppServiceStatus struct {
    // Nodes are the names of the pods in the deployment
    Nodes []string `json:"nodes,omitempty"`
}
```

Step 4: Implement the Reconcile Loop

Locate `internal/controller/appservice_controller.go` and implement the `Reconcile` function. Your logic should check if the resource exists, create child resources if missing, and update the `Status`.

```
func (r *AppServiceReconciler) Reconcile(ctx context.Context, req ctrl.Request)
(ctrl.Result, error) {
    // 1. Fetch the AppService instance
    // 2. Logic to ensure deployment size matches Spec.Size
    // 3. Update Status
    return ctrl.Result{}, nil
}
```

Step 5: Run the Controller

```
make install  # Install CRDs into the cluster
make run      # Run the controller locally
```

Key Takeaways

- **Declarative Intent**: The `Spec` represents the user's intent, not a command sequence.
- **Idempotency**: The reconcile loop must be idempotent—running it twice must produce the same result as running it once.
- **Event-Driven**: Controllers rely on `client-go` informers to efficiently listen for changes rather than polling the API server.

Distributed system principles for high availability

Distributed System Principles for High Availability in Kubernetes

In a distributed system like Kubernetes, high availability (HA) is not merely about keeping processes running; it is about ensuring the system maintains a consistent state despite network partitions, hardware failures, or concurrent updates. When extending Kubernetes through custom controllers, you are essentially building a distributed state machine.

The Challenge of Distributed State

Kubernetes operates on the principle of **eventual consistency**. The control plane does not guarantee that a cluster will be in a specific state at a specific microsecond. Instead, it guarantees that, given enough time, the cluster will converge toward the "desired state" defined in your objects.

When multiple controller instances manage the same resources, you face two primary challenges: 1. **Race Conditions:** Multiple controllers attempting to modify the same resource simultaneously. 2. **Split-Brain Scenarios:** Different controller instances making conflicting decisions, leading to state corruption or service instability.

Foundational Principles for HA Controllers

To build robust, highly available automation, your controllers must adhere to these distributed systems principles:

1. Idempotency

An operation is idempotent if performing it multiple times yields the same result as performing it once. In Kubernetes, your `Reconcile` function must be designed to be idempotent. * **Design Pattern:** Always read the current state (Status/Live state) before acting. If the resource is already in the desired state, do nothing. * **Why it matters:** Controllers can be triggered by spurious events or retries. Non-idempotent operations often lead to resource exhaustion or duplicate API calls.

2. Optimistic Concurrency Control (OCC)

Kubernetes uses `resourceVersion` in metadata to implement OCC. When you update an object, the API server checks if the `resourceVersion` you provided matches the current version in `etcd`. * **Mechanism:** If a conflict occurs (409 Conflict), your controller must handle the error by re-fetching the latest version of the object and re-running the reconciliation logic.

3. Leader Election

In high-availability deployments, you often run multiple replicas of a controller (e.g., via a `Deployment`). However, you usually only want **one** active controller to perform reconciliation at any given time to avoid conflicting actions. * **Lease Objects:** Kubernetes uses the `coordination.k8s.io/v1 Lease` API. Controllers compete for a lock on a Lease resource. The winner becomes the leader, while others wait in a standby state.

4. Controller Sharding

When a single controller cannot handle the sheer volume of resources, you must employ **sharding**. This is the process of partitioning the workload across multiple controller instances based on a specific key (e.g., namespace or a custom label). * **Case Study (OpenShift Ingress):** The ingress controller shards the routing table. Each controller instance is responsible for a subset of the total routes, ensuring that the control plane remains responsive as the number of ingress objects grows.

Distributed Sync Workflow: Hands-On

To implement high availability in your own controllers, you must move beyond a simple loop.

Example 1. Hands-on Lab: Implementing Leader Election with Leases

In this lab, you will configure your `client-go` controller to respect leader election using the `leaderelection` package.

1. **Initialize the Leader Election Config:** Define a `LeaderElectionConfig` that specifies the `Lease` name and the namespace where the lock will be held.

```
[source,go]
----
import "k8s.io/client-go/tools/leaderelection"
```

```
// Define the lock
lock := &resourcelock.LeaseLock{
    LeaseMeta: metav1.ObjectMeta{Name: "my-controller-lock", Namespace: "kube-
system"},
    Client:    client.CoordinationV1(),
    LockConfig: resourcelock.ResourceLockConfig{Identity: podName},
}
```

```
----
```

2. **Run the Election Loop:** Wrap your main reconciliation logic inside the `leaderelection.RunOrDie` function. This ensures your code only executes when the lease is acquired.

```
[source,go]
----
leaderelection.RunOrDie(context.TODO(), leaderelection.LeaderElectionConfig{
    Lock: lock,
    Callbacks: leaderelection.LeaderCallbacks{
        OnStartedLeading: func(ctx context.Context) {
            // Start your Informers and Reconcile loop here
        },
        OnStoppedLeading: func() {
            log.Fatalf("Lost leader lock")
        },
    },
})
----
```

3. **Verification:** Deploy two replicas of your controller. Observe that only one log entry indicates "Leading", and if you delete that pod, the second replica automatically picks up the lease.

Summary for Architectural Success

- **Design for failure:** Always assume your controller might restart or encounter a network partition.
- **Leverage the API Server:** Never store "state" inside your controller's memory. Always persist state in the Kubernetes API using Status fields or Custom Resources.
- **Reconcile, don't trigger:** Your logic should be driven by the difference between current state and desired state, not by a sequence of events.

Advanced networking, traffic management, and control plane architectures

Advanced Networking, Traffic Management, and Control Plane Architectures

Overview

As Kubernetes clusters grow in scale and complexity, standard Service and Ingress primitives often prove insufficient for enterprise-grade requirements. Advanced networking focuses on decoupling traffic routing from underlying infrastructure, while diverse control plane architectures address

the needs of edge computing, multi-tenancy, and high-density environments.

1. Advanced Networking and Traffic Management

Modern Kubernetes networking has evolved from simple L4 load balancing to sophisticated L7 traffic management.

Gateway API vs. Ingress

While the standard `Ingress` resource provides a basic way to expose HTTP/S routes, it suffers from fragmentation across different ingress controller implementations (e.g., NGINX, HAProxy, Contour). The **Gateway API** addresses this by: * **Role-Oriented Design:** Separating infrastructure ownership (GatewayClasses) from application routing (HTTPRoutes). * **Expressiveness:** Providing native support for advanced traffic patterns like header-based routing, traffic splitting, and weighted routing. * **Extensibility:** Allowing custom policies to be attached directly to routes.

Service Mesh Concepts

When networking requirements exceed simple routing—such as mTLS, advanced circuit breaking, or fine-grained observability—a **Service Mesh** (e.g., Istio, Linkerd) is typically employed. * **Control Plane:** Orchestrates configuration and security policies. * **Data Plane:** Sidecar proxies (like Envoy) intercept all network traffic to provide transparent encryption, retries, and telemetry collection without modifying application code.

2. Alternative Control Plane Architectures

The "standard" Kubernetes control plane (kube-apiserver, etcd, scheduler, controller-manager) is optimized for general-purpose clusters. However, varying use cases have necessitated the emergence of alternative architectures.

HyperShift (Hosted Control Planes)

HyperShift decouples the control plane from the worker nodes. By running control planes as pods inside a management cluster, you gain: * **Rapid Provisioning:** Faster deployment of clusters. * **Resource Efficiency:** Reduced overhead by sharing management cluster infrastructure. * **Isolation:** Control planes are isolated, making them easier to upgrade and manage independently.

kcp: Kubernetes-like API Server

`kcp` is designed for scenarios where a single API surface is needed to manage resources across thousands of clusters. It provides a "Kubernetes-like" experience without forcing the overhead of a full cluster control plane for every tenant. It focuses on: * **Logical Clusters:** Enabling multi-tenancy at the API layer. * **API Agnosticism:** Allowing users to define custom resources that aren’t bound to a specific runtime cluster.

Edge and Lightweight Form Factors

For constrained environments, standard Kubernetes is too heavy. Solutions like **MicroShift** (the edge-optimized distribution of OpenShift) and **k3s** minimize the footprint by: * **Consolidation:** Combining control plane components into a single binary. * **Storage Optimization:** Replacing etcd with lighter key-value stores (e.g., SQLite) in some configurations. * **Aggressive Pruning:** Removing

non-essential features and legacy drivers.

Summary of Architectural Trade-offs

| Architecture | Primary Use Case | Scaling Approach | | :--- | :--- | :--- | | **Standard K8s** | General purpose, Cloud-native | Horizontal node scaling | | **HyperShift** | Multi-tenant, Managed Services | Control Plane consolidation | | **kcp** | Global-scale API management | Logical clustering | | **MicroShift/k3s** | Edge, IoT, Industrial | Resource minimization |

Further Exploration

- **Design Considerations:** When choosing a control plane architecture, consider the "blast radius"—how failures in the control plane propagate to the managed workloads.
- **Networking Security:** Always prioritize mTLS for service-to-service communication when moving to advanced traffic management solutions to ensure zero-trust compliance.

Who Is This For? Prerequisites

Target Audience

This learning path is designed for engineers, system administrators, and developers who already possess practical experience working with containerized environments and Kubernetes. If you have deployed applications on Kubernetes and want to move beyond basic usage into platform-level development, this course is for you.

Recommended Prerequisites

A foundational understanding of distributed systems principles is strongly recommended before beginning this course. Familiarity with concepts such as concurrency, leasing, and synchronization will help you get the most out of the material on controllers, leader election, and eventual consistency.

Ideal Baseline

The ideal starting point is proficiency equivalent to one of the following:

- **Certified Kubernetes Application Developer (CKAD)** certification
- Completion of Red Hat **DO-180** (Introduction to Containers, Kubernetes, and Red Hat OpenShift) and **DO-280** (Red Hat OpenShift Administration) training tracks

Either path ensures you have the hands-on Kubernetes experience needed to engage with the advanced topics in this course.

Self-Assessment Checklist

Before starting, review the checklist below. You should be comfortable with most of these items:

- ☐ You can create and manage Deployments, Services, and ConfigMaps
- ☐ You understand container image building and registries
- ☐ You are comfortable reading and writing YAML manifests
- ☐ You have basic familiarity with Go programming
- ☐ You can use `kubectl` to inspect and debug cluster resources
- ☐ You understand the difference between a Pod, a ReplicaSet, and a Deployment
- ☐ You have worked with namespaces and resource quotas
- ☐ You are familiar with basic RBAC concepts (Roles, RoleBindings) # Who is this for? pre-requisites.

Who is this for? Pre-requisites

This learning path is designed for engineers, system administrators, and developers who possess practical experience with containerized environments. To ensure success, participants should have a foundational understanding of distributed systems.

- **Target Audience:** Professionals working with container technology and infrastructure automation.
- **Recommended Prerequisites:** Basic knowledge of distributed system principles.
- **Ideal Baseline:** Completion of CKAD certification, or Red Hat DO-180 and DO-280 tracks.

Who Is This For? Prerequisites

Target audience: Engineers, sys admins, and developers with container experience

Target Audience and Prerequisites

This learning path is designed for technical professionals who have moved beyond basic container usage and are ready to master the internal mechanics, extensibility, and architectural patterns of Kubernetes.

Who is this for?

This content is curated for:

- **Software Engineers:** Developers who need to extend Kubernetes functionality by building custom controllers, operators, or specialized API servers.
- **Systems Administrators (SREs/Platform Engineers):** Professionals responsible for managing, scaling, and hardening complex Kubernetes environments.
- **Platform Architects:** Those tasked with designing automated workflows or deploying alternative control plane architectures (e.g., HyperShift, kcp).

Recommended Prerequisites

To derive maximum value from this path, you should possess a solid foundation in container orchestration. We assume you are comfortable with:

- **Container Fundamentals:** You understand images, container lifecycles, and basic networking (e.g., `podman` or `docker` workflows).
- **Kubernetes Basics:** You are familiar with standard resources (Deployments, Services, ConfigMaps) and the `kubectl` CLI.
- **Distributed Systems Concepts:** You have a working knowledge of asynchronous communication, state consistency, and basic concurrency models.
- **Programming Literacy:** Because we will be interacting with `client-go` and implementing controllers, proficiency in **Go (Golang)** is highly recommended.

Ideal Baseline

We recommend that participants meet one of the following benchmarks to ensure a successful learning experience:

Benchmark	Description
CKAD Certification	Demonstrates hands-on proficiency in designing, building, and deploying cloud-native applications on Kubernetes.

Benchmark	Description
Red Hat DO-180	Familiarity with Introduction to Containers, Kubernetes, and Red Hat OpenShift.
Red Hat DO-280	Proficiency in OpenShift Administration: Core concepts of managing enterprise-grade Kubernetes clusters.

If you are familiar with the declarative nature of Kubernetes but are new to the internal `kube-apiserver` request/response lifecycle, we will cover these foundational concepts in the upcoming modules. If your Go programming skills are rusty, we suggest reviewing the Go Tutorial before starting the **client-go** lab exercises.

Why this path matters

Beyond managing static workloads, this curriculum moves you toward **Platform Engineering**. By the end of this path, you will transition from a consumer of Kubernetes resources to an architect capable of:

1. Automating infrastructure through **Custom Resources (CRDs)**.
2. Building resilient **Controllers** that handle complex state reconciliation.
3. Securing the cluster via **Admission Webhooks**.
4. Understanding the distributed systems "magic" that keeps the Control Plane stable under high load.

Recommended prerequisites: Basic distributed systems knowledge

Prerequisites: Preparing for the Kubernetes Developer Learning Path

To succeed in this advanced Kubernetes development track, it is essential to have a solid foundation. While this course focuses on the internal mechanics of Kubernetes extensibility—such as custom controllers, admission webhooks, and distributed state management—these concepts rely heavily on the principles of distributed systems.

Who is this for?

This curriculum is designed for **Software Engineers, Site Reliability Engineers (SREs), and Platform Architects** who have moved beyond simply deploying applications and are now tasked with extending the Kubernetes control plane itself.

Recommended Prerequisites

Before beginning this module, you should be comfortable with the following competencies:

Category	Requirement
Containerization	Proficiency in container lifecycles, OCI image standards, and resource management (Requests/Limits). You should be familiar with the DO-180 (Introduction to Containers, Kubernetes, and Red Hat OpenShift) level of knowledge.
Kubernetes Administration	Familiarity with standard cluster operations. You should be comfortable with `kubectl`, the structure of common resources (Pods, Deployments, Services), and standard RBAC (Role-Based Access Control) configuration (equivalent to DO-280 or CKAD certification).
Programming	Strong proficiency in **Go (Golang)** is highly recommended. As the Kubernetes codebase is primarily written in Go, and `client-go` is the primary library for interaction, comfort with Go interfaces, concurrency primitives (goroutines/channels), and error handling is vital.

Distributed Systems Fundamentals

Because we will be building controllers that interact with a highly available, eventually consistent system, you should have a baseline understanding of:

- **Eventual Consistency:** Understanding that the state of a cluster is a distributed consensus problem. You should understand how the control plane moves from a "desired state" to an "actual state."
- **Concurrency and Race Conditions:** Knowledge of how to handle concurrent modifications to resources, which is critical when writing controllers that may be deployed in high-availability (HA) modes with multiple replicas.
- **API Semantics:** Understanding RESTful interactions, HTTP status codes, and the difference between synchronous (blocking) and asynchronous (event-driven) processing.

Why these prerequisites matter

In this course, we move away from "using" the cluster to "orchestrating" it. When you build a custom operator, you are essentially writing code that operates as part of the Kubernetes control plane. If your controller logic is not designed with distributed systems principles (such as idempotency and proper error handling), you risk causing "flapping" states or system-wide instability.

If you feel your knowledge in these areas is limited, we suggest reviewing: * **Designing Distributed Systems** by Brendan Burns (O'Reilly). * The official Kubernetes documentation on the [Reference Architecture](https://kubernetes.io/docs/concepts/architecture/).

Ideal baseline: CKAD certification, or Red Hat DO-180 and DO-280 tracks

Preparing for the Kubernetes Developer Learning Path

To succeed in this curriculum, you must possess a solid foundation in container orchestration. This learning path is designed for engineers, system administrators, and developers who are ready to transition from **using** Kubernetes to **extending** it.

The Ideal Baseline

We expect participants to have a working understanding of the Kubernetes ecosystem. Our curriculum assumes you are comfortable with declarative configuration and the standard resource lifecycle. You should be prepared to meet one of the following benchmarks:

- **CKAD (Certified Kubernetes Application Developer):** This certification confirms your ability to design, build, and deploy cloud-native applications. If you hold this certification, you already have the necessary skills in handling Pods, Deployments, Services, and ConfigMaps—the fundamental building blocks we will manipulate via custom controllers.
- **Red Hat DO-180 (Introduction to Containers, Kubernetes, and Red Hat OpenShift):** This track provides the essential vocabulary for containerized environments, image management, and basic orchestration.
- **Red Hat DO-280 (Red Hat OpenShift Administration II: Operating a Production Kubernetes Cluster):** This track dives deeper into the operational aspects of the platform, providing the necessary context regarding how clusters are secured, networked, and maintained—all of which are critical when writing controllers that interact with the cluster's control plane.

If you do not hold these specific credentials, ensure you have equivalent practical experience. You should be familiar with: * The `kubectl` command-line interface and basic API interaction. * YAML-based manifest management. * The fundamental concepts of distributed systems (e.g., eventual consistency, API-centric communication).

Why these prerequisites matter

The content we will cover—ranging from `client-go` informers to Custom Resource Definitions (CRDs) and admission webhooks—builds directly upon the core Kubernetes resource model.

If you are unfamiliar with how the `kube-apiserver` processes a request or how a deployment maintains its state through reconciliation, the jump into `client-go` architecture and controller reconciliation patterns will be significantly more difficult. By starting with the proficiency level defined by the **CKAD** or **DO-180/DO-280** tracks, you ensure that you can focus on the **extension** mechanisms of Kubernetes rather than struggling with basic application deployment patterns.

If you feel your knowledge in these areas is rusty, revisit the official Kubernetes documentation on **Declarative Management** and the **Kubernetes API** before beginning the hands-on labs in this learning path.

Familiarizing with koo·br·neh·teez

Familiarizing with Kubernetes

Kubernetes is an API-centric, declarative system designed for container orchestration and configuration management. Rather than executing imperative commands, users define the desired state of their infrastructure as data, which the system then works to maintain.

- **API-Centric Architecture:** The `kube-apiserver` acts as the central coordination point for all cluster operations, processing requests and managing lifecycle states.
- **Declarative Model:** Users submit resource configurations representing their desired intent, moving away from manual, step-by-step administrative tasks.
- **Persistence Layer:** The cluster state is stored within **etcd**, a distributed key-value store that serves as the single source of truth for the control plane.
- **Reconciliation Patterns:** Kubernetes utilizes asynchronous controllers to continuously observe current cluster status and take automated actions to align it with the declared desired state.

API-centric declarative configuration management

API-Centric Declarative Configuration Management

At its core, Kubernetes is not merely a container orchestrator; it is a **declarative, API-centric configuration management system**. Understanding this architecture is fundamental to transitioning from a "user of Kubernetes" to a "Kubernetes developer."

The Declarative Philosophy

In an **imperative** system, you provide a sequence of commands to achieve a goal (e.g., "start container A, then connect to network B"). If a step fails or the environment changes, the system remains in a broken state unless you manually intervene.

In a **declarative** system, you define the **desired state** of your infrastructure. You describe what the world should look like, and the system works continuously to make the **actual state** match that desired state.

Key Components of the Declarative Model

1. **Desired State:** Expressed as pure data (typically YAML or JSON). You define the **intent** (e.g., "I want 3 replicas of my web server") rather than the **process** of creating them.
2. **The API Server (`kube-apiserver`):** The front door of the control plane. It exposes the Kubernetes API, which acts as the single source of truth for the cluster's state.
3. **etcd:** A distributed, consistent key-value store. This is the persistence layer where the `kube-apiserver` stores the state of all cluster resources.
4. **Reconciliation Loop:** The engine of Kubernetes. Controllers constantly watch the state of the system via the API server, compare it to the desired state, and execute operations to bridge the

gap.

The API Server Role

The `kube-apiserver` is the central coordination point of the entire architecture. Every operation in Kubernetes flows through it.

- **Request Lifecycle:** When you submit a resource (e.g., a `Deployment`), the `kube-apiserver` performs authentication, authorization, and validation. Once the resource is validated, it is persisted to `etcd`.
- **Asynchronous Processing:** The API server does not "do" the work. It records the request. It is the responsibility of asynchronous controllers (watching the API) to notice the new object and take action (e.g., scheduling pods).
- **Extensibility:** Because the system is API-centric, you can extend the model by introducing Custom Resource Definitions (CRDs) or even by building your own API servers that plug into the Kubernetes API Aggregation Layer.

Why Declarative Matters for Developers

For developers, the declarative nature of Kubernetes offers several advantages:

- **Self-Healing:** If a process crashes, the controller detects that the actual state (0 replicas) does not match the desired state (3 replicas) and automatically recreates the pods.
- **Idempotency:** You can apply the same configuration file multiple times without side effects. If the object already exists and is in the correct state, the system does nothing.
- **Observability:** Because the entire state is stored in the API, you can query the system at any time to see the gap between what you asked for and what currently exists.

Comparison Summary

| Feature | Imperative | Declarative (Kubernetes) | | :--- | :--- | :--- | | **User Input** | "Do this now" | "Keep this state" | | **Reliability** | Fragile (state drift) | Robust (reconciliation) | | **System State** | Implicit/Hidden | Explicit (etcd) | | **Tooling** | Scripts/CLI commands | API-centric resource manifests |

Conclusion

As you progress through this learning path, remember that every extension you build—whether it is a custom controller, an operator, or a new API server—must honor this declarative intent. Your goal is to move the system toward the desired state defined by the user in the most reliable and efficient way possible.

Kubernetes control plane and API server role

Kubernetes Control Plane and API Server Role

The Kubernetes control plane is the brain of the cluster. It orchestrates the state of the system, ensuring that the actual state of your infrastructure matches the desired state defined by users. At

the center of this operation lies the `kube-apiserver`, the primary gateway for all administrative tasks and internal component communication.

The Role of the API Server

The `kube-apiserver` acts as the frontend for the Kubernetes control plane. Every interaction with the cluster—whether from `kubectl`, internal controllers, or external automation—must pass through the API server.

Its primary responsibilities include: * **Exposure of the API:** It exposes the Kubernetes API using JSON over HTTP. * **Request Lifecycle Management:** It manages the end-to-end flow of a request, from authentication to final persistence. * **State Management:** It is the only component that talks directly to the `etcd` key-value store, ensuring a single source of truth for the cluster configuration.

The API Request Lifecycle

Understanding how the API server processes a request is critical for any Kubernetes developer. When a request hits the API server, it traverses several plugin layers before interacting with storage:

1. **Authentication:** Identifies who is making the request (e.g., X509 certificates, service account tokens, or OIDC).
2. **Audit:** Records the request for security and compliance monitoring.
3. **Impersonation:** Allows one user to act as another (useful for troubleshooting and administrative delegation).
4. **Priority & Fairness:** Manages request execution based on defined priorities to prevent API starvation.
5. **Authorization:** Determines if the authenticated user has permission to perform the specific action (RBAC).
6. **Admission Control:** Executes plugins (Validating and Mutating webhooks) to modify or reject requests based on cluster policies.
7. **Storage:** Persistence layer interaction with `etcd`.

Admission webhooks (both Mutating and Validating) are triggered during `CREATE` and `UPDATE` operations. They are not invoked during `GET` or `LIST` operations, as those are read-only and do not alter the cluster state.

Persistence Layer: etcd

While the `kube-apiserver` manages the logic, `etcd` provides the persistence. It is a distributed, consistent key-value store.

- **Single Source of Truth:** All Kubernetes objects (Pods, Deployments, CustomResources) are stored as serialized JSON/Protobuf in `etcd`.
- **Consistency:** Because `etcd` uses the Raft consensus algorithm, it guarantees that once a write is acknowledged, the data is consistent across the control plane nodes.

- **Scaling:** Note that the control plane, specifically `etcd` and the `kube-apiserver`, scales vertically. Adding more CPU and memory to existing nodes is generally preferred over horizontal scaling for these components to maintain the integrity of the consensus store.

Declarative Intent

Kubernetes operates on a declarative model. Instead of issuing commands (e.g., "start a container"), users submit a desired state (e.g., "I want 3 replicas of this image").

The API server accepts this intent and stores it. It is then the role of the asynchronous controllers (like the `DeploymentController`) to watch these resources, calculate the delta between the current state and the desired state, and execute the necessary actions to reconcile the difference.

For a deep dive into the internal mechanics of how these components interact, refer to the session materials: **The Life (or Death) of a Kubernetes API Request (2025 Edition)** by Abu Kashem and Stefan Schimanski.

Next Steps

Now that you understand the role of the control plane and the API server, the next module will cover how we interact with this API programmatically using `client-go` and how controllers monitor these resources to maintain cluster health.

Persistence layer: etcd key-value store

Persistence Layer: etcd

The robustness of the Kubernetes control plane relies heavily on its ability to maintain a consistent "source of truth." In Kubernetes, that source of truth is `etcd`, a distributed, consistent, key-value store.

What is etcd?

`etcd` is a strongly consistent, distributed key-value store used as Kubernetes' backing store for all cluster data. It is the only stateful component in the control plane; the `kube-apiserver` itself is stateless, relying on `etcd` to persist the desired state of the system.

Core Characteristics

- **Consistency:** It uses the Raft consensus algorithm to ensure that all nodes in an `etcd` cluster agree on the sequence of state changes.
- **Key-Value Hierarchy:** While it is a flat key-value store, Kubernetes organizes data using a hierarchical path structure (typically under the `/registry/` prefix).
- **Watch Mechanism:** `etcd` provides a watch API that allows the `kube-apiserver` (and other components) to receive notifications when specific keys change, which is the foundation of the asynchronous reconciliation pattern used by controllers.
- **Atomic Transactions:** It supports transactional updates, ensuring that either all changes in a

batch are applied or none are.

The Role of etcd in the API Request Lifecycle

When you submit a resource (e.g., a `Pod` definition) to the `kube-apiserver`, the following flow occurs regarding persistence:

1. **Authentication/Authorization:** The `kube-apiserver` validates the request.
2. **Admission Control:** Validating/Mutating webhooks are executed.
3. **Persistence:** If validation passes, the `kube-apiserver` transforms the JSON object into a serialized format and commits it to `etcd`.
4. **Event Propagation:** Once persisted, `etcd` notifies the `kube-apiserver` via a watch, which in turn triggers informers/controllers to observe the change.

Architectural Considerations

Reliability and Consensus

Because `etcd` uses the Raft consensus algorithm, it requires a quorum to function. In a production cluster, `etcd` is typically deployed with an odd number of members (3, 5, or 7). If a majority of members are lost, the cluster loses the ability to write new state, effectively rendering the Kubernetes cluster read-only.

Performance and Scaling

- **Vertical Scaling:** Unlike worker nodes that scale horizontally, the control plane persistence layer (`etcd`) requires vertical scaling. Increasing the IOPS (disk performance) and CPU/Memory allocation of the `etcd` nodes is the primary method for handling increased cluster load.
- **Compaction and Defragmentation:** Since `etcd` tracks history for the watch mechanism, it can grow indefinitely. The `kube-apiserver` triggers periodic compaction to remove old revisions, and manual defragmentation is often necessary to reclaim disk space.

Key Takeaways for Developers

- **Stateless Controllers:** Your custom controllers should treat `etcd` as the primary source of truth. Never store local state in a controller that isn't mirrored or derived from `etcd`.
- **Resource Versions:** Every object in `etcd` has a `metadata.resourceVersion`. Kubernetes uses this for optimistic concurrency control—if you attempt to update an object that has changed since you last read it, the API server will reject the request with a `409 Conflict` error.

Direct interaction with `etcd` is strictly reserved for the `kube-apiserver`. Developers should never interact with the `etcd` database directly using `etcdctl` to modify cluster state, as this bypasses the API server's validation, admission webhooks, and the cache consistency logic essential for cluster health.

Asynchronous controller reconciliation patterns

Asynchronous Controller Reconciliation Patterns

In Kubernetes, the control plane does not use a push-based model to command components. Instead, it relies on an **asynchronous reconciliation loop**. This design is fundamental to the platform's robustness, allowing it to recover from partial failures, network partitions, and unexpected state changes without manual intervention.

The Reconciliation Concept

At its core, a Kubernetes controller acts as a control loop that constantly compares the **observed state** of the system with the **desired state** defined in the API object (the `Spec`).

The reconciliation process follows these principles: * **Declarative Intent:** Users define "what" they want (the Spec), not "how" to achieve it. * **Idempotency:** The controller must be able to run the same logic repeatedly without causing side effects beyond the intended state change. * **Level-Triggered, not Edge-Triggered:** Unlike an edge-triggered system (which triggers only on the exact moment a change happens), Kubernetes is level-triggered. It cares about the **current state** of the world, making it resilient to missed events.

Anatomy of the Reconciliation Loop

The reconciliation loop is the "heartbeat" of any controller. Using `client-go`, this is typically structured as follows:

1. **Watch:** The controller watches the API server for changes to resources (e.g., Pods, Deployments, or Custom Resources).
2. **Enqueue:** When an event (Add, Update, Delete) occurs, the resource key (namespace/name) is added to a `workqueue`.
3. **Process:** The controller pulls the key from the `workqueue`.
4. **Sync:** The controller fetches the latest version of the resource from a local cache (via `Informer` and `Lister`).
5. **Reconcile:** The controller logic runs. It compares the current state (`Status`) against the desired state (`Spec`).
6. **Act:** If the states don't match, the controller performs actions (e.g., calling the API server to create a service) to move the system toward the desired state.

Asynchronous Nature and Eventual Consistency

Because the system is asynchronous, several factors influence its behavior:

Why Queues are FIFO

The `workqueue` (and `DeltaFIFO` in informers) operates on a First-In-First-Out basis. This ensures that operations are processed in the order they were observed. Since the controller is asynchronous, multiple events for the same object might be coalesced in the queue—if the controller hasn't

processed the first update yet, the second update will simply replace the entry in the queue, ensuring the controller always reconciles against the **latest** version.

Caches and Informers

Directly polling the `kube-apiserver` for every reconciliation cycle would overwhelm the API server and introduce unacceptable latency. * **Informers:** These maintain a local, high-performance cache of resources in memory. * **Event Propagation:** When an event occurs, the `Reflector` (part of the Informer) watches the API server and populates the cache. The controller logic reads from this local cache, ensuring the system remains responsive even under high load.

Error Handling and Requeuing

If a reconciliation attempt fails (e.g., due to a temporary network error or a validation failure), the controller does not crash. Instead: 1. The error is captured. 2. The key is **requeued** with an exponential backoff. 3. The controller will retry the operation until it succeeds or reaches a threshold where it logs the error for human intervention.

Designing for Idempotency

To build a reliable asynchronous controller, every operation must be idempotent. If a controller creates a resource and the network drops before the confirmation is received, the subsequent reconciliation loop will attempt to create it again. Your code must be able to handle "Already Exists" errors gracefully by checking the existing state before acting.

In a distributed environment, the reconciliation loop is where you must consider **Optimistic Locking**. When updating a resource, always provide the `resourceVersion` found in the object. If another process modified the object while you were working, the API server will reject your update, prompting your controller to requeue and reconcile against the new, correct state.

1.1 — Kubernetes Resource Model

1.1 — Kubernetes Resource Model

The Kubernetes Resource Model centers on declarative configuration, where users define the desired state of the system rather than executing imperative commands. The `kube-apiserver` acts as the primary gateway for these definitions, processing requests through a rigorous pipeline.

Key aspects include:

- **Declarative Intent:** Resources represent a persistent record of intent, allowing the system to reconcile the current state with the declared configuration.
- **Request Lifecycle:** Every write operation (CREATE/UPDATE) flows through a sequence of plugin layers, including authentication, authorization, audit logging, and admission control.
- **Admission Webhooks:** Both Mutating and Validating webhooks intercept write requests to enforce policies or modify objects before they are persisted to `etcd`.
- **Scalability:** Control plane components like the `kube-apiserver` and `etcd` primarily support vertical scaling to maintain performance under load.

The declarative intent of Kubernetes Resources

The Declarative Intent of Kubernetes Resources

In traditional systems administration, management often relies on **imperative** actions: you send commands like "start this container," "update this configuration," or "restart this service." If a command fails or the system state drifts, manual intervention is required to bring the system back into alignment.

Kubernetes shifts this paradigm entirely by utilizing a **declarative** model. Instead of instructing the system on **how** to perform a task, you provide the system with a definition of **what** the final state should look like.

Declarative vs. Imperative

To understand the declarative nature of Kubernetes, consider the difference between a set of instructions and a target goal:

- **Imperative (The "How"):** "Log into the server, download the binary, create a service file, start the process, and verify it is running."
- **Declarative (The "What"):** "Ensure that 3 replicas of the `my-app` container, based on image `v1.2.0`, are running at all times."

In the declarative model, the user submits a Resource definition (typically YAML or JSON) to the Kubernetes API. The system then assumes the responsibility of reconciling the current state of the cluster to match your declared intent.

The Role of the API Server

The `kube-apiserver` acts as the gateway for this declarative intent. When you submit a resource (e.g., `kubectl apply -f deployment.yaml`), you are performing an operation on the Kubernetes API.

The API server processes these requests through a series of stages designed to validate and persist your desired state:

1. **Authentication:** Identifying who is making the request.
2. **Authorization:** Determining if the user is allowed to perform the requested action.
3. **Admission Control:** Validating or mutating the object before it is persisted (e.g., checking if the image registry is approved).
4. **Storage:** Once validated, the desired state is persisted in `etcd`, the cluster's source of truth.

Idempotency and Convergence

A core strength of the declarative model is **idempotency**. An operation is idempotent if performing it multiple times results in the same outcome as performing it once.

If you apply a manifest five times in a row, the Kubernetes control plane performs the following: 1. Compares the requested state (in the YAML) with the current state (in `etcd`). 2. Determines that they already match. 3. Takes no action (or reports "unchanged").

This provides **self-healing** capabilities. If a node fails and a Pod is terminated, the Controller Manager detects the mismatch between the declared state (3 replicas) and the current state (2 replicas). It then automatically schedules a new Pod to restore the desired state without requiring an imperative trigger from the administrator.

Key Takeaways

- **Source of Truth:** The API server and `etcd` hold the "Desired State."
- **Reconciliation Loop:** Kubernetes controllers continuously observe the actual state and perform actions to align it with the declared intent.
- **Version Control:** Because state is defined as data (YAML), you can store your infrastructure configuration in Git, enabling GitOps workflows.

For further study on the architectural philosophy behind this, refer to the [Kubernetes Resource Management design proposal](https://github.com/kubernetes/design-proposals-archive/blob/main/architecture/resource-management.md), which serves as the foundational text for how Kubernetes handles resource intent.

kube-apiserver plugin request/response lifecycle

The kube-apiserver Plugin Request/Response Lifecycle

The `kube-apiserver` acts as the brain of the Kubernetes cluster. It is not a monolithic service but a

sophisticated aggregation of plugin layers that process every incoming request. Understanding this lifecycle is critical for developers who need to implement custom controllers, admission webhooks, or extended API servers.

The Request Lifecycle

When a client (like `kubectl` or a custom controller) sends a request to the `kube-apiserver`, the request traverses a strict chain of plugin layers before it is persisted to `etcd` or returned to the user.

```
@startuml
skinparam componentStyle uml2
[Client] --> [Authentication]
[Authentication] --> [Audit]
[Audit] --> [Impersonation]
[Impersonation] --> [Priority & Fairness]
[Priority & Fairness] --> [Authorization]
[Authorization] --> [Mutating Admission]
[Mutating Admission] --> [Object Validation]
[Object Validation] --> [Validating Admission]
[Validating Admission] --> [Storage (etcd)]
@enduml
```

1. Authentication

The server verifies who the caller is. Plugins include X509 client certs, static token files, OIDC, or Webhook token authentication. If authentication fails, the request is rejected with a `401 Unauthorized` response.

2. Audit

Once authenticated, the request is recorded by the Audit Logging backend. This layer ensures that every action is traceable for compliance and debugging, capturing the `who, what, when, and where` of the request.

3. Impersonation

This allows an authenticated user to perform actions on behalf of another user. This is frequently used by `kubectl auth can-i` or administrative tooling to verify permissions without switching user contexts.

4. Priority & Fairness (APF)

This layer prevents a single client or controller from overwhelming the `kube-apiserver`. It classifies incoming requests into flows and applies rate limiting based on the configured `PriorityLevelConfiguration` and `FlowSchema` objects.

5. Authorization

The server determines if the authenticated user has permission to perform the requested action (e.g., `GET`, `POST`, `PATCH`) on the specific resource. Common plugins include RBAC (Role-Based Access

Control) and Node authorization.

6. Admission Control

Admission controllers are the final gatekeepers. They are divided into two phases: * **Mutating:** These webhooks can modify the object definition (e.g., injecting sidecars or default values). * **Validating:** These webhooks ensure the object meets custom policies (e.g., "Only allow images from the private registry") before persisting to storage.

7. Storage

If the request passes all validation phases, the `kube-apiserver` serializes the object and commits it to the `etcd` backend.

Hands-on Lab: Observing API Discovery

To understand how the API server exposes its capabilities, we can inspect the API discovery endpoints.

Prerequisites

- A running Kubernetes cluster (e.g., `minikube` or `oc cluster up`).
- Access to `curl` and a proxy to the cluster.

Step-by-Step Activity

1. **Start the local proxy:** Open a terminal and establish a connection to your cluster. [source,bash] ---- kubectl proxy --port=8001 ----
2. **Query the API Discovery endpoint:** Use the specific header requested to retrieve the discovery information. This demonstrates how the `kube-apiserver` handles content negotiation. [source,bash] ---- curl -v http://127.0.0.1:8001/api \ -H "Accept: application/json;v=v2;g=apidiscovery.k8s.io;as=APIGroupDiscoveryList" ----
3. **Analyze the Response:** Notice the `Content-Type` header and the structured JSON returned. This is the mechanism by which the `kube-apiserver` dynamically informs clients of the resources available within the cluster, enabling the `kubectl` client to function without needing hard-coded resource definitions for every CRD installed.

Troubleshooting Tips

- **401 Unauthorized:** Check your `kubeconfig` and ensure the identity provider (OIDC/Certs) is reachable.
- **403 Forbidden:** The request reached the API server, but the `RBAC` layer blocked it. Review your `ClusterRole` and `RoleBinding` definitions.
- **Latency Issues:** If your API server is sluggish, check the `Priority and Fairness` metrics to see if specific controllers are saturating your available request flows.

1.2 — Controllers, informers, client-go

1.2 — Controllers, informers, client-go

This module covers the foundational building blocks for Kubernetes operators and resource synchronization. It focuses on the `client-go` library, which provides the necessary tooling for interaction with the Kubernetes API.

Key components include:

- **Informers:** Act as the "eyes" of the controller, watching for cluster events and reflecting changes into a local state.
- **Listers:** Provide read-only access to the local cache, allowing controllers to retrieve objects without constant API server round-trips.
- **Workqueues:** Serve as buffers that decouple event detection from processing, ensuring tasks are handled efficiently via FIFO (First-In-First-Out) logic.
- **Reconciliation Patterns:** Strategies for error handling and requeueing, enabling controllers to react to state changes and drive resources toward their desired declarative intent.

client-go architecture fundamentals

client-go Architecture Fundamentals

`client-go` is the official Go-based client library for communicating with the Kubernetes API server. It is the foundation upon which almost all Kubernetes controllers and operators are built. Understanding its architecture is essential for moving beyond simple `kubectl` commands and into the realm of custom automation.

The Core Purpose

In Kubernetes, the control plane relies on a declarative model. Your controller's job is to observe the current state of the cluster and drive it toward a desired state. `client-go` provides the mechanisms to perform this observation efficiently without overwhelming the API server with constant polling.

Architecture Components

The strength of `client-go` lies in its ability to provide a local, synchronized view of the cluster state. This is achieved through three primary components:

1. Informers (The "Eyes")

An **Informer** is a client that watches the Kubernetes API for changes to a specific resource (e.g., Pods or Deployments). * **Event Handling:** Instead of manual polling, the Informer uses a `Watch` request to receive a stream of events (Add, Update, Delete) from the API server. * **Efficiency:** By maintaining a persistent connection, it reduces the load on the API server significantly compared to periodic list requests.

2. Listers (The "Local Cache")

Every Informer maintains a local, read-only cache of the objects it is watching. * **The Role:** The **Lister** acts as an interface to this cache. When your code needs to know the current state of a resource, it queries the Lister rather than querying the API server directly. * **Performance:** Reading from local memory is orders of magnitude faster than a network round-trip to the API server.

3. WorkQueues (The "Buffers")

Because events can happen rapidly (e.g., a burst of Pod deletions), you do not want your controller to execute a complex logic loop for every single event immediately. * **Decoupling:** Informers push the name/namespace of changed objects into a **WorkQueue**. * **FIFO Ordering:** The workqueue is strictly First-In-First-Out. * **Rate Limiting & Retries:** The `workqueue` package provides built-in mechanisms to handle errors (e.g., if your reconciliation fails, you can put the item back into the queue with an exponential backoff).

Data Flow: How it works together

1. **Event:** A change occurs in the cluster (e.g., a user creates a new resource).
2. **Informer:** The Informer detects the event via the watch stream and updates the **local store (cache)**.
3. **Handler:** The Informer triggers an event handler (a function you define), which adds the key (namespace/name) of the affected resource into the **WorkQueue**.
4. **Worker:** Your controller's worker loop pulls the key from the queue and calls the **Lister** to get the actual object data from the local cache.
5. **Reconciliation:** Your code processes the object and interacts with the API server only when a change is strictly necessary.

Key Gotcha: Always remember that your cache is **eventually consistent**. There is a slight propagation delay between an event happening in the cluster and that event reflecting in your local cache. Your controller logic must be designed to be idempotent to account for this.

Why this architecture matters

If you were to write a controller without this pattern, you would be forced to call `List` on the API server in a loop. This would: * Increase API server CPU and memory pressure. * Lead to latency issues as the cluster grows. * Result in an unscalable application.

By using the `client-go` architecture, you move the heavy lifting of state synchronization to the client side, ensuring your controller is performant, responsive, and resilient.

Informers, listers, and workqueues

Informers, Listers, and Workqueues: The Controller Engine

In Kubernetes, controllers act as the "brain" of the system, observing the cluster state and

performing operations to move the current state toward the desired state. To do this efficiently without overwhelming the `kube-apiserver`, controllers rely on three core components: **Informers**, **Listers**, and **Workqueues**.

The Architecture of Synchronization

To build a high-performance controller, you must minimize round-trips to the `kube-apiserver`. Instead of polling the API for every single event, controllers use a local cache populated by an Informer.

1. Informers: The "Eyes"

An Informer is the primary mechanism for watching resources. It handles the communication between your controller and the `kube-apiserver`.

- **List and Watch**: Upon startup, the Informer performs a full `list` operation to get the current state and then establishes a `watch` stream to receive subsequent updates.
- **Event Propagation**: When an event (add, update, delete) occurs, the Informer receives the object and triggers predefined event handlers (e.g., `AddFunc`, `UpdateFunc`).
- **Reflector**: Inside the Informer, a Reflector component watches the API server and pushes events into a `DeltaFIFO` queue.

2. Listers: The Local Cache

Directly querying the `kube-apiserver` is expensive and latency-prone. Listers provide an interface to interact with a local, in-memory cache managed by the Informer.

- **Efficiency**: Because the data resides in your process memory, `Get` and `List` operations are nearly instantaneous.
- **Consistency**: Listers ensure your controller logic always operates on a consistent snapshot of the cluster state as seen by the Informer.

3. Workqueues: The Buffer

The controller should not perform heavy lifting inside the event handler. If the API server sends a flurry of events, you need a way to buffer and process them reliably.

- **Decoupling**: The event handler simply adds the "key" (usually `namespace/name`) of the object to a Workqueue.
- **FIFO Behavior**: Workqueues in `client-go` are **First-In-First-Out (FIFO)**. This ensures events are processed in the order they were received.
- **Requeue Strategies**: If the reconciliation logic fails, you can put the key back into the queue for a retry. Advanced queues support "exponential backoff" to prevent hot-looping on broken resources.

Summary Workflow

1. **Reflector** watches API server and adds events to `DeltaFIFO`.

2. **Informer** processes `DeltaFIFO`, updates the **Local Store (Cache)**, and triggers your event handlers.
3. **Event Handler** adds a key to the **Workqueue**.
4. **Reconcile Loop** pulls keys from the Workqueue and uses the **Lister** to retrieve the full object state for processing.

Hands-on Lab: Barebones Controller Skeleton

This activity demonstrates how to wire an Informer to a Workqueue using `client-go`.

Prerequisites

- A Go environment installed.
- `k8s.io/client-go` and `k8s.io/apimachinery` modules initialized in your project.

Step 1: Initialize the Workqueue

Create a workqueue that supports rate limiting.

```
import (
    "k8s.io/client-go/util/workqueue"
)

// Create a new rate-limiting workqueue
queue := workqueue.NewNamedRateLimitingQueue(workqueue.DefaultControllerRateLimiter(),
"MyController")
```

Step 2: Define the Informer and Event Handler

Set up the Informer so that whenever a Pod is updated, its key is pushed to the queue.

```
informer.AddEventHandler(cache.ResourceEventHandlerFuncs{
    AddFunc: func(obj interface{}) {
        key, _ := cache.MetaNamespaceKeyFunc(obj)
        queue.Add(key)
    },
    UpdateFunc: func(oldObj, newObj interface{}) {
        key, _ := cache.MetaNamespaceKeyFunc(newObj)
        queue.Add(key)
    },
})
```

Step 3: Implement the Worker Process

This loop runs continuously, pulling items off the queue and processing them.

```
func processNextItem(queue workqueue.RateLimitingInterface) bool {
    key, quit := queue.Get()
    if quit { return false }
    defer queue.Done(key)

    // Logic: Use a Lister here to fetch the object from cache
    // err := reconcile(key)

    // If err != nil, handle requeue:
    // queue.AddRateLimited(key)

    return true
}
```

Pro-Tip: Error Handling

Always ensure your `reconcile` logic is idempotent. If a process crashes while handling a key, the `workqueue.Done(key)` call ensures you don't lose track of the resource, but the controller's restart will pick up the state again from the Informer's cache.

Cache mechanisms and event propagation

Cache Mechanisms and Event Propagation

To build robust Kubernetes controllers, one must understand how they maintain a high-performance, consistent view of the cluster state. Querying the `kube-apiserver` directly for every reconciliation loop is inefficient, introduces latency, and puts undue stress on the control plane. Instead, `client-go` utilizes a sophisticated local caching mechanism.

The Role of Caching in Kubernetes Controllers

In a distributed system like Kubernetes, the "source of truth" resides in `etcd`, accessed via the `kube-apiserver`. A controller's primary goal is to reach a desired state. To do this efficiently, the controller maintains a **Local Cache**.

- **Efficiency:** By storing objects locally, the controller can perform lookups (Get/List) in memory, avoiding network round-trips.
- **Decoupling:** The cache allows the controller to continue operating even if there are momentary network blips between the controller and the API server.
- **Performance:** Controllers often react to thousands of events. Direct API calls would lead to rate-limiting and increased latency.

Event Propagation: From API Server to Local Cache

The mechanism that keeps the local cache synchronized with the cluster state is known as an **Informer**. The Informer architecture ensures that the controller is notified only of the changes it

cares about.

1. The Watch Mechanism

Everything begins with the `WATCH` verb. When an Informer starts, it initiates a long-running HTTP request to the `kube-apiserver` asking to be notified of changes to specific resources (e.g., Pods or Deployments).

2. DeltaFIFO: The Buffer

The API server sends a stream of events (Added, Updated, Deleted). These events are received by the Informer and placed into a **DeltaFIFO** queue. * **FIFO (First-In, First-Out):** As noted in our core principles, queues in `client-go` are strictly FIFO. This ensures that operations are processed in the order they occurred in the cluster. * **Delta:** The queue stores the object along with the "delta" type (e.g., `Added`, `Updated`, `Deleted`), allowing the controller to understand the transition that occurred.

3. Reflectors

The **Reflector** is the component responsible for watching the API server. It polls the API server for changes and populates the `DeltaFIFO`. It acts as the bridge between the remote `etcd` state and the local `DeltaFIFO` buffer.

4. Indexers and Local Store

The Informer pops items from the `DeltaFIFO` and updates the **Indexer** (the local cache). * **Listers:** These provide a read-only interface to the Indexer. When your controller logic calls `lister.Get(name)`, it is querying this local memory-resident cache, not the API server. * **Event Handling:** Once the cache is updated, the Informer triggers the `ResourceEventHandler` functions (`OnAdd`, `OnUpdate`, `OnDelete`) registered by the controller, which typically place the object's key into the **WorkQueue** for processing.

Summary of the Flow

1. **Reflector** watches the `kube-apiserver`.
2. Events are pushed to the **DeltaFIFO** buffer.
3. **Informer** pops the event, updates the **Indexer (Local Cache)**, and notifies the **EventHandlers**.
4. **EventHandlers** push the object key into the **WorkQueue**.
5. The **Controller's Reconciliation Loop** pulls the key from the **WorkQueue** and uses the **Lister** to fetch the current state from the **Local Cache**.

Why caches exist? Caches act as a "read-replica" of the cluster state. Because the cache is eventually consistent with `etcd`, the controller must be designed to handle the possibility that its local view is slightly stale. This is why reconciliation loops must be **idempotent**—they should be able to run multiple times against the same object without causing unintended side effects.

Error handling and requeue strategies

Error Handling and Requeue Strategies in Controllers

In a Kubernetes controller, the `Reconcile` function is rarely a "one-and-done" operation. Due to the distributed nature of the system, network partitions, API conflicts, and dependency delays, your reconciliation logic must be resilient. Effective error handling ensures that your controller eventually brings the cluster state to the desired configuration without crashing or entering an infinite loop.

The Role of the Workqueue

The `client-go` workqueue is the heartbeat of your controller. When an `Informer` detects a change (Add, Update, or Delete), it pushes the object key (namespace/name) onto the queue. Your worker processes these keys by calling `Reconcile(key)`.

The Reconciliation Pattern

A robust `Reconcile` function should always follow this pattern:

1. **Retrieve:** Fetch the object from the `Lister` (local cache).
2. **Validate/Execute:** Perform the necessary business logic.
3. **Handle Result:** Determine the outcome:
    - **Success:** Return `nil` (no error).
    - **Transient Error:** Return an error to trigger a retry.
    - **Terminal Error:** Log the error and do not requeue (e.g., invalid configuration).

Requeue Strategies

When your reconciliation logic fails, you must decide how to handle the next attempt. In `client-go`, you return an error, and the controller infrastructure manages the backoff.

1. Rate-Limited Requeuing

To prevent "hot-looping" (constantly trying to reconcile a broken object and consuming all CPU), you should use a rate-limiter. The `workqueue` package provides implementations like `DefaultControllerRateLimiter`, which uses an exponential backoff strategy.

- **First failure:** Wait a short time.
- **Subsequent failures:** Exponentially increase the wait time (e.g., 5s, 10s, 20s, up to a cap).

2. Requeue After Duration

Sometimes, your controller isn't failing, but it needs to wait for a state that isn't triggered by an event. For example, if you are waiting for an external API to become available, you might return a result with a specific delay.

```
// Example of requeuing after a delay
return ctrl.Result{RequeueAfter: time.Second * 30}, nil
```

Implementation: Best Practices

- **Avoid infinite loops:** Never requeue immediately (without delay) if the error is persistent. Always allow the rate-limiter to intervene.
- **Use `Forget`:** Once a key is successfully processed, ensure you call `queue.Forget(key)` so the rate-limiter resets the failure count for that object.
- **Handle Conflict Errors:** If you encounter `k8s.io/apimachinery/pkg/api/errors.IsConflict`, it means another controller or a user modified the object while you were working. A simple retry is usually the correct response.

Hands-on Activity: Implementing Error Handling

In this exercise, you will modify your basic controller to handle a "simulated" API error using exponential backoff.

1. **Identify the Reconcile loop:** Find your worker loop where you invoke `Reconcile()`.
2. **Add Error Logic:** Introduce a conditional check that returns an error if a specific field in your custom resource is missing.
3. **Configure the Queue:** When calling `queue.AddRateLimited(item)` in your event handler, ensure you are using the `RateLimitingInterface`.
4. **Verify:** Run your controller and observe the log output. Notice how the time between reconciliation attempts increases after each failure.

Remember: The workqueue is **FIFO (First-In, First-Out)**. If you have multiple items in the queue, the controller will process them in the order they arrived, but it will apply the retry delay independently to each item based on its failure history.

Troubleshooting Gotchas

- **Queue Saturation:** If you requeue too many items, your controller's memory usage may spike. Monitor the queue length.
- **Informer Desync:** If you catch an error, don't assume the local cache is stale. If the error is an `IsNotFound` error, treat it as a deletion event and clean up your local state.
- **Blocking:** Never perform long-running blocking operations inside `Reconcile` without considering the impact on the controller's ability to process other items in the queue. Use `go routines` only if absolutely necessary, but prefer the `RequeueAfter` pattern for time-based operations.

Hands-on Lab: Writing a barebones controller using client-go from scratch without controller-runtime.

Hands-on Lab: Building a Barebones Controller with client-go

In this lab, you will bypass the high-level abstractions of `controller-runtime` to build a functional Kubernetes controller from scratch. This exercise is designed to give you a deep understanding of the `client-go` library, specifically how Informers, Listers, and WorkQueues interact to maintain the desired state of a cluster.

Prerequisites

- A running Kubernetes cluster (e.g., `kind` or `minikube`).
- Go 1.21 or later installed.
- `client-go` and `apimachinery` modules initialized in your `go.mod`.

The Architecture of a Manual Controller

To build a controller, we must implement three primary components: 1. **Informer**: Watches the API server for resource events (Add, Update, Delete) and updates a local cache. 2. **Lister**: A read-only interface that provides access to the local cache, preventing excessive API server hits. 3. **WorkQueue**: A buffer that stores keys (namespace/name) of resources that need processing, allowing for decoupled and retriable reconciliation.

Lab Steps

Step 1: Initialize the Informer Factory

The `SharedInformerFactory` is the heart of your controller. It ensures that multiple controllers share the same connection to the API server and the same cache.

```
import (
    "k8s.io/client-go/informers"
    "k8s.io/client-go/kubernetes"
    "k8s.io/client-go/tools/cache"
)

// Setup the factory
factory := informers.NewSharedInformerFactory(clientset, time.Minute*10)
podInformer := factory.Core().V1().Pods().Informer()
```

Step 2: Implement Event Handlers

We must register functions that trigger whenever a resource changes. Crucially, we do not perform logic **inside** these handlers; we simply extract the resource key and push it to the `WorkQueue`.

```
podInformer.AddEventHandler(cache.ResourceEventHandlerFuncs{
```

```
    AddFunc: func(obj interface{}) {
        key, _ := cache.MetaNamespaceKeyFunc(obj)
        queue.Add(key)
    },
    // Implement UpdateFunc and DeleteFunc similarly...
})
```

Step 3: The Reconciliation Loop

The "Brain" of the controller runs in a loop. It pulls keys from the queue and calls the `Sync` function.

```
func (c *Controller) runWorker() {
    for {
        key, quit := c.queue.Get()
        if quit { return }

        err := c.syncHandler(key.(string))
        if err != nil {
            // Handle error and requeue
            c.queue.AddRateLimited(key)
        } else {
            c.queue.Forget(key)
        }
        c.queue.Done(key)
    }
}
```

Step 4: The `syncHandler` Logic

This is where your business logic lives. Use the Lister to retrieve the object from the local cache.

```
func (c *Controller) syncHandler(key string) error {
    namespace, name, _ := cache.SplitMetaNamespaceKey(key)
    pod, err := c.lister.Pods(namespace).Get(name)
    if err != nil {
        return err // Resource likely deleted
    }

    // PERFORM BUSINESS LOGIC HERE
    // Example: Check if Pod has a required label
    return nil
}
```

Key Takeaways

- **Caches are critical:** Using the `Lister` ensures your controller remains performant by querying local memory rather than hitting the `kube-apiserver` on every loop.
- **WorkQueues provide safety:** By adding items to a queue, you gain the ability to use

`AddRateLimited`, which provides built-in exponential backoff for failing reconciliations.

- **Decoupling:** By separating event detection (Handlers) from processing (Sync), your controller remains responsive even under high load.

Troubleshooting Tips

- **Stale Caches:** Ensure your `Informer` has synced before processing the queue using `cache.WaitForCacheSync`.
- **Resource Leaks:** Always ensure you call `c.queue.Done(key)` to prevent the work queue from becoming blocked.
- **Logging:** When debugging `client-go`, use the `klog` library to inspect the internal event flow.

2.1 — Controllers contd.

2.1 — Controllers contd.

This module builds upon foundational controller knowledge by focusing on the `controller-runtime` abstraction to simplify the development of custom Kubernetes APIs.

- **Abstraction and Scaffolding:** Utilizing `controller-runtime` to streamline reconciliation logic and `kubebuilder` for project scaffolding.
- **State Management:** Implementing the distinction between `Spec` (desired state) and `Status` (actual state), including robust condition management.
- **API Integrity:** Enforcing API schemas through validation and linting.
- **Data Operations:** Mastering CRUD operations and the specific semantics of `apply`, `patch`, and `update` to ensure efficient state synchronization.

Reconcile loop and controller-runtime abstraction

Reconcile Loop and Controller-Runtime Abstraction

In Kubernetes, the intelligence of the system lies in its ability to constantly observe the current state of the cluster and drive it toward a desired state. This is achieved through the **Reconcile Loop**. While `client-go` provides the low-level primitives to interact with the API server, the `controller-runtime` library provides the higher-level abstractions necessary to build robust, production-grade operators.

The Reconcile Loop Explained

The reconcile loop is the heart of every Kubernetes controller. It is an infinite loop that ensures that for any given resource, the "Actual State" (what is running) matches the "Desired State" (what is defined in the manifest).

Unlike imperative programming, where you explicitly tell the system **how** to do something, the reconcile loop is **declarative**. It follows a simple logic:

1. **Observe**: Fetch the current state of the object.
2. **Analyze**: Compare the current state against the desired state defined in the `Spec`.
3. **Act**: Perform the necessary operations (CRUD) to align the actual state with the desired state.
4. **Report**: Update the `Status` field of the resource to reflect the current progress or outcome.

The Controller-Runtime Abstraction

Manually managing `Informers`, `Listers`, and `Workqueues` using raw `client-go` is error-prone and repetitive. `controller-runtime` simplifies this by providing:

- **Manager**: The entry point that orchestrates the lifecycle of controllers, webhooks, and cache

synchronization.

- **Reconciler Interface**: A high-level abstraction (`Reconcile(ctx, request)`) that abstracts away the complexities of the workqueue.
- **Cache/Client**: A buffered client that uses local caches to reduce load on the `kube-apiserver` while maintaining strict consistency.
- **Event Filtering**: Predicates that allow you to ignore updates that don't change relevant fields, reducing unnecessary reconciliations.

Key Components of a Reconciler

When writing a controller with `controller-runtime`, you typically implement the `reconcile.Reconciler` interface.

```
type ReconcileMyResource struct {
    client.Client
    Scheme *runtime.Scheme
}

func (r *ReconcileMyResource) Reconcile(ctx context.Context, req ctrl.Request)
(ctrl.Result, error) {
    // 1. Fetch the instance
    myResource := &v1alpha1.MyResource{}
    if err := r.Get(ctx, req.NamespacedName, myResource); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }

    // 2. Logic to reach desired state
    // ... logic goes here ...

    // 3. Update Status
    if err := r.Status().Update(ctx, myResource); err != nil {
        return ctrl.Result{}, err
    }

    return ctrl.Result{}, nil
}
```

Spec vs. Status: The Core Contract

The separation of concerns between `Spec` and `Status` is critical to the Kubernetes Resource Model:

- **Spec (Desired State)**: Provided by the user. It is immutable from the controller's perspective. Never change the `Spec` from within the reconcile loop; this causes infinite loops and system instability.
- **Status (Actual State)**: Owned by the controller. It informs the user about the health, progress, and errors encountered while trying to achieve the `Spec`.

Hands-on: Scaffolding with Kubebuilder

To start working with `controller-runtime`, we use `kubebuilder`, the industry-standard SDK for scaffolding Kubernetes projects.

1. **Initialize the project**: [source,bash] ---- mkdir my-operator && cd my-operator kubebuilder init --domain example.com --repo example.com/my-operator ----
2. **Create an API**: [source,bash] ---- kubebuilder create api --group webapp --version v1 --kind Guestbook ----
3. **Explore the Controller**: Open `internal/controller/guestbook_controller.go`. You will see the scaffolded `Reconcile` function. Note how `controller-runtime` has already set up the `Manager` to watch for changes to the `Guestbook` resource.

Best Practices for Reconcile Loops

- **Idempotency**: Your reconcile loop must be idempotent. If it fails halfway and restarts, it should be able to continue without creating duplicate resources.
- **Error Handling**: If a step fails, return the error. `controller-runtime` will automatically requeue the request based on an exponential backoff.
- **Avoid API Flooding**: Use the local cache provided by the manager. Do not perform expensive list operations on the API server inside the loop.

Scaffolding with kubebuilder

Scaffolding with Kubebuilder

Kubebuilder is the industry-standard framework for building Kubernetes APIs using Custom Resource Definitions (CRDs) and controllers. While writing a controller from scratch using `client-go` (as explored in previous modules) is essential for understanding the underlying machinery, **Kubebuilder** provides a high-level abstraction that significantly reduces boilerplate and enforces Kubernetes API best practices.

Why Kubebuilder?

Building a production-grade controller involves complex tasks: setting up informers, managing caches, handling event queues, and ensuring thread safety. Kubebuilder scaffolds this infrastructure for you, allowing you to focus on the business logic inside the **Reconciliation Loop**.

It leverages `controller-runtime`—a set of libraries used by Kubernetes-native projects—to simplify: * **API Scaffolding:** Generating Go structs for your Custom Resources (CRs). * **Controller Boilerplate:** Setting up the manager, leader election, and health probes. * **Webhook Integration:** Simplified generation of conversion, mutating, and validating webhooks. * **Standardization:** Ensuring your API adheres to Kubernetes conventions (GVK, scope, and status subresources).

The Development Workflow

Kubebuilder follows an iterative command-line workflow to transition from an idea to a deployed controller.

Command	Purpose
`kubebuilder init`	Initializes a new project, setting up the Go module and `controller-runtime` scaffolding.
`kubebuilder create api`	Scaffolds a new CRD (Spec/Status) and its associated Controller logic.
`make manifests`	Generates the YAML manifests (CRDs, RBAC roles) based on Go struct tags.
`make install`	Installs the CRDs into the target Kubernetes cluster.
`make run`	Runs the controller locally against your current `~/.kube/config`.

Best Practices for API Design

When scaffolding with Kubebuilder, your design must adhere to the Kubernetes API Conventions.

1. Spec vs. Status

- **Spec (Desired State):** Fields that represent the state the user **wants**. This should be immutable where possible and represents the "contract" between the user and your controller.
- **Status (Actual State):** Fields that represent the **current** state of the object as observed by the controller. Users should never manually modify the Status field.

2. API Validation and Linting

To catch architectural issues early, integrate the **Kube API Linter (KAL)**. It checks your Go structs against standard Kubernetes patterns, such as verifying that your CRD fields follow naming conventions and utilize appropriate validation markers (e.g., `+kubebuilder:validation:Minimum=1`).

```
// Example of validation markers in a CRD struct
type CronJobSpec struct {
    // +kubebuilder:validation:Minimum=0
    ConcurrencyPolicy string `json:"concurrencyPolicy,omitempty"`
}
```

Hands-on: Scaffolding a Basic Operator

Follow these steps to bootstrap your project.

1. **Initialize the Project:**

   ```
   mkdir my-controller && cd my-controller
   kubebuilder init --domain example.com --repo github.com/example/my-controller
   ```

2. **Create the API:**

   ```
   kubebuilder create api --group batch --version v1 --kind CronJob
   ```

Note: This command generates both the API definition (`api/v1/cronjob_types.go`) and the controller logic (`internal/controller/cronjob_controller.go`).

3. **Explore the Controller:** Open `internal/controller/cronjob_controller.go`. You will see a `Reconcile` function. This is where you inject your reconciliation logic to ensure the "Actual State" (Status) matches the "Desired State" (Spec).
4. **Verify with Manifests:** Run `make manifests` to update your `config/crd/bases` directory. Inspect the generated YAML to see how Kubebuilder translated your Go structs into a valid Kubernetes CRD schema.

Summary: Kubebuilder vs. Operator SDK

While `kubebuilder` is the underlying framework, the **Operator SDK** is a wrapper that adds additional features like Helm/Ansible operator support and advanced operator lifecycle management. For high-performance Go-based controllers, standard `kubebuilder` remains the preferred, lightweight choice for developers who want full control over their code.

Spec vs Status and condition management

Spec vs. Status and Condition Management

In the Kubernetes declarative resource model, separating the "what" from the "how" is fundamental. This separation is achieved through the architectural division of a resource into two primary sections: the `spec` and the `status`.

The Spec: The Desired State

The `spec` (short for **specification**) defines the **intended** state of an object. It represents the user's "truth"—the configuration that the user wants to see realized in the cluster.

- **Immutable intent:** Once defined, the `spec` should remain static until the user or an automated process explicitly changes it.
- **Source of Truth:** It acts as the input for controllers. When a controller observes a change in the `spec`, it triggers a reconciliation process to align the current cluster state with this requested state.

The Status: The Observed State

The `status` represents the **actual** state of the resource at any given moment. It is the output of the controller's work.

- **Controller-managed:** The `status` is intended to be updated **only** by the controller. Users should not manually modify the `status` field of a resource.
- **Feedback loop:** It provides visibility into whether the `spec` has been successfully realized, if there are errors, or if the system is currently "in progress."

Condition Management: Expressing Progress

While `status` provides the broad state, **Conditions** provide granular, machine-readable details about that state. Conditions follow a standardized pattern that allows external tools and humans to interpret the lifecycle of an object without needing to parse complex logic.

A standard Condition structure includes: * **Type:** The specific aspect of the resource being reported (e.g., `Ready`, `Available`, `Progressing`). * **Status:** The state of the condition, which must be one of `True`, `False`, or `Unknown`. * **Reason:** A CamelCase, programmatic identifier explaining why the condition is in its current state. * **Message:** A human-readable explanation of the status. * **LastTransitionTime:** The timestamp indicating when the status of this condition last changed.

Why Conditions Matter

Conditions are essential for "Eventual Consistency." Since Kubernetes controllers run asynchronously, an operation (like provisioning a cloud resource) might take time. Conditions allow the controller to communicate: 1. **Pending:** `Ready` = `False`, `Reason` = `Provisioning` 2. **Success:** `Ready` = `True`, `Reason` = `Ready` 3. **Failure:** `Ready` = `False`, `Reason` = `ConfigurationError`

Best Practices for Controllers

- **Don't rely on `resourceVersion` for business logic:** While `metadata.resourceVersion` is used for optimistic concurrency, always use `.status.conditions[].observedGeneration` to determine if your controller has processed the current `spec` version.
- **The Reconciliation Loop:** Always check `status` before acting. If the `spec` matches the `status` (and all conditions indicate success), the controller should exit the loop without performing unnecessary actions. This ensures **idempotency**.
- **Avoid Deep Nesting:** Keep your `status` fields clean and well-defined. Over-populating the status can lead to large object sizes, which impacts performance during API server transmission.

Erratum/Clarification: Ensure you distinguish between `resourceVersion` and `generation`. While `resourceVersion` is an internal string used by etcd for concurrency control, `.metadata.generation` is an integer that increments whenever the `spec` changes. Comparing `generation` against `observedGeneration` is the standard way to check if the controller needs to reconcile a new request.

API validation and linting

API Validation and Linting

In the Kubernetes Resource Model, ensuring that your Custom Resource Definitions (CRDs) are robust, consistent, and safe is critical. Because Kubernetes is an API-centric system, "API Validation" is the first line of defense against malformed intent being persisted into `etcd`.

Understanding API Validation

API validation ensures that the `Spec` provided by a user conforms to the expected structure and business logic before the controller even touches the data. There are two primary levels of

validation:

- **Structural Validation:** Validating the data types, required fields, and constraints (e.g., regex patterns for names or range limits for integers) using OpenAPI v3 schemas within your CRD.
- **Semantic Validation:** Complex business logic validation (e.g., checking if a registry is approved or if a requested port is within a range) that cannot be expressed via static OpenAPI schemas. This typically requires a **Validating Webhook**.

The Role of OpenAPI v3

When you define your Go structs in `controller-runtime` or `kubebuilder`, these are translated into an OpenAPI v3 schema. Kubernetes uses this schema to reject requests that do not match the expected structure (e.g., passing a string where an integer is expected).

You can enforce these rules using marker comments in your Go code, which `controller-gen` processes:

```
// +kubebuilder:validation:Minimum=1
// +kubebuilder:validation:Maximum=10
Replicas int32 `json:"replicas"`
```

Linting with Kube-API-Linter (KAL)

Beyond simple schema validation, **API linting** ensures that your API design follows established community best practices. The `kube-api-linter` (often referred to as `api-linter`) is an essential tool for developers to catch design flaws before they are set in stone.

Common issues caught by linters include: * **Naming Conventions:** Ensuring fields are camelCase and follow standard Kubernetes naming patterns. * **Pluralization:** Ensuring `Kind` names are singular and `List` types are plural. * **Documentation:** Verifying that every field has a clear description for end-users. * **Standard Fields:** Ensuring that `status` and `spec` are correctly defined as optional or required based on typical patterns.

Why Linting Matters

Without linting, APIs often suffer from "drift"—where different CRDs in the same ecosystem use inconsistent naming styles or lack the necessary metadata for `kubectl` output. Using `kube-api-linter` during your CI/CD pipeline ensures that your CRDs remain idiomatic and predictable for cluster administrators.

Hands-On Activity: Validating Your API

In this short exercise, you will add a validation constraint to a field in your existing controller project.

Goal: Prevent users from setting a negative number of replicas in your custom resource.

Step 1: Update your Go API definition

Navigate to your `api/v1alpha1/` directory and locate your `Spec` struct. Add the `kubebuilder` validation markers:

```
type MyResourceSpec struct {
    // +kubebuilder:validation:Minimum=1
    // +kubebuilder:validation:Maximum=10
    // +kubebuilder:validation:Required
    Replicas int32 `json:"replicas"`
}
```

Step 2: Regenerate the CRD Manifests

Run the following command to update your YAML manifests based on the Go markers:

```
make manifests
```

Step 3: Verify the change

Check the `config/crd/bases/` directory. You will see that the OpenAPI schema for your CRD now contains `minimum: 1` and `maximum: 10` under the `replicas` field.

Step 4: Test the validation

Attempt to apply a YAML manifest with a replica count of `0` or `11`:

```
kubectl apply -f config/samples/myresource_v1alpha1_sample.yaml
```

Expected Result: The `kube-apiserver` will reject the request immediately with a validation error, preventing the invalid object from reaching your controller.

CRUD operations and apply/patch/update semantics

CRUD Operations and Apply/Patch/Update Semantics

In the Kubernetes control plane, understanding how data is modified is critical for writing robust controllers. Kubernetes does not simply "overwrite" files; it manages state through specific API interaction patterns. Choosing the wrong method can lead to race conditions, lost updates, or unexpected resource behavior.

Understanding API Semantics

When interacting with the `kube-apiserver`, you generally perform four types of operations. While they map to traditional CRUD (Create, Read, Update, Delete), Kubernetes has specific implementations for the "Update" phase.

1. Create and Read

- **Create:** Posts a new manifest to the API.
- **Read (Get/List):** Retrieves the current state of a resource. This is often served from the controller's local cache (Informer) rather than hitting `etcd` directly.

2. Update (The "Full Replace")

The `Update` operation (HTTP `PUT`) requires the client to send the **entire object** back to the server. * **How it works:** The server compares the received object with the existing one. * **Risk:** If another process modified the resource between your `Get` and your `Update`, your `Update` will fail with a `409 Conflict` (Optimistic Concurrency). * **Use case:** When you need to change multiple fields simultaneously and ensure the entire object state is synchronized.

3. Patch (The Partial Update)

The `Patch` operation (HTTP `PATCH`) allows you to modify specific fields without sending the whole object. This is the preferred method for controllers to avoid massive data transfer and reduce conflict frequency.

There are three primary strategies: * **JSON Patch:** A list of operations (add, remove, replace) defined in RFC 6902. * **Merge Patch:** A partial JSON document representing the intended state of the object. * **Strategic Merge Patch:** Kubernetes-specific. It understands the schema of resources. For example, it knows how to merge lists (like container definitions in a Pod) based on unique keys rather than simply overwriting the entire list.

4. Server-Side Apply (SSA)

Server-Side Apply is the modern standard for managing resources. It shifts the logic of "who owns what" to the API server.

- **Field Ownership:** SSA tracks which field manager (e.g., your controller) owns which fields.
- **Resolution:** If two controllers try to set the same field, the API server handles the conflict.
- **Benefit:** It eliminates the need for the `Get → Modify → Update` cycle, which is prone to race conditions.

Technical Comparison Summary

Method	Data Sent	Concurrency Strategy
Update (PUT)	Entire Object	Optimistic Locking (ResourceVersion)
Patch	Partial JSON	Server-side atomic merge
Server-Side Apply	Partial / Intentional State	Field Management (Ownership)

Hands-On: Experimenting with Patching

In this activity, you will observe the difference between a standard `Update` and a `Patch`.

Prerequisites

- A running Kubernetes cluster (e.g., `kind` or `minikube`).
- `kubectl` installed and configured.

Step 1: Create a test resource

Create a simple ConfigMap to experiment with.

```
kubectl create configmap demo-map --from-literal=key1=value1
```

Step 2: Simulate a Conflict (Update)

1. Open two terminals.
2. In Terminal A, get the object: `kubectl get cm demo-map -o yaml > map.yaml`
3. In Terminal B, change the value in `map.yaml` and run `kubectl apply -f map.yaml`.
4. In Terminal A, change the value to something else and run `kubectl apply -f map.yaml`.
5. **Observation:** You will see an error indicating the `resourceVersion` has changed.

Step 3: Using Patch (Strategic Merge)

Use the `patch` command to update only the specific data field without touching the rest of the object.

```
kubectl patch configmap demo-map -p '{"data": {"key1": "new-value"}}'
```

Note: Observe how the `metadata.resourceVersion` increments, but you never had to fetch the original object to perform the change.

Pro-Tips for Controllers

- **Avoid PUTs:** Inside your `Reconcile` loop, prefer `client.Patch` or `client.Status().Update`.
- **Use `$retainKeys`:** If you are performing a strategic merge and need to explicitly drop a field that was previously managed, use the `$retainKeys` directive in your patch payload.
- **Status vs. Spec:** Always use `status().update()` when changing the status subresource to avoid triggering extra reconciliation loops unnecessarily.

2.2 — Going distributed: locks, leases, ownership, eventual consistency

2.2 — Going distributed: locks, leases, ownership, eventual consistency

As controllers scale, managing state across distributed instances requires robust synchronization to maintain eventual consistency and system stability. This module focuses on the principles of designing idempotent state machines and handling concurrency in Kubernetes.

- **Idempotency:** Designing state machines that ensure atomicity and consistent outcomes, regardless of how many times a reconcile loop executes.
- **Locks and Leases:** Utilizing Kubernetes `Lease` objects to manage synchronization and ensure that specific tasks—such as leader election—are handled by a single, authoritative controller instance.
- **Leader Election:** Implementing mechanisms to prevent conflicts when multiple controller replicas attempt to perform the same operations.
- **Controller Sharding:** Distributing workload across multiple controller instances to improve scalability and availability, as demonstrated by the OpenShift ingress controller patterns.

Designing idempotent state machines

Designing Idempotent State Machines in Kubernetes

In a distributed system like Kubernetes, the controller model operates on the principle of **eventual consistency**. Because network partitions, pod restarts, and API server latencies are inevitable, your controllers must be resilient to repeated, redundant, or out-of-order events.

The mechanism that enables this resilience is the **Idempotent State Machine**.

The Core Principle: Idempotency

An operation is idempotent if performing it multiple times yields the same result as performing it once. In the context of a Kubernetes controller, your `Reconcile` function must be designed so that:

- **Current State == Desired State:** If the system is already in the correct state, the controller performs no action.
- **Partial Failure Recovery:** If a controller crashes halfway through a task, restarting the reconcile loop should complete the remaining steps without duplicating side effects.

Designing for Atomicity and Consistency

To build an idempotent state machine, you must transition from thinking in "commands" to thinking in "states."

1. Spec vs. Status Pattern

The `Spec` represents the desired state (what the user wants), while the `Status` represents the observed state (what the controller has achieved). Your logic should flow as follows:

1. **Observe:** Read the `Spec` and the current state of the infrastructure (e.g., existing ConfigMaps, Deployments).
2. **Diff:** Calculate the difference between the `Spec` and the observed state.
3. **Act:** Only apply the minimal change required to move the observed state closer to the `Spec`.
4. **Report:** Update the `Status` to reflect the new state.

2. Guarding against side effects

When your controller interacts with external systems (like cloud APIs or databases), you cannot simply "update" as you would with a K8s resource. You must implement protective guards:

- **Pre-flight checks:** Before creating a resource, check if it already exists by a unique identifier (e.g., a standard Kubernetes label or a generated name).
- **Resource Versioning:** Utilize the `metadata.resourceVersion` to perform optimistic locking. If the resource changed since you last read it, the API server will reject your update, preventing "lost updates."
- **Finalizers:** Use finalizers to ensure that cleanup logic (like deleting an external cloud load balancer) is handled idempotently when a resource is deleted.

Practical Implementation Strategy

When writing your logic, follow this pseudo-code pattern to maintain idempotency:

```
func (r *Reconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result,
error) {
    // 1. Fetch the custom resource
    instance := &v1alpha1.MyCustomResource{}
    if err := r.Get(ctx, req.NamespacedName, instance); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }

    // 2. Determine what needs to be done
    // Logic: Do we already have the sub-resource?
    if !r.exists(instance) {
        // 3. Act: Only perform the creation if it doesn't exist
        if err := r.create(instance); err != nil {
            return ctrl.Result{}, err
        }
    }

    // 4. Update status to match current reality
    instance.Status.Phase = "Ready"
```

```
    return ctrl.Result{}, r.Status().Update(ctx, instance)
}
```

Key Takeaways for Distributed Controllers

- **Avoid Statefulness:** Store as much information as possible in the Kubernetes API itself, rather than in local memory or local files.
- **Handle Requeues:** Always expect the `Reconcile` function to be called again for the same resource. Do not assume the world has stayed the same since the last execution.
- **Don't ignore errors:** If your controller fails to reach an external API, return the error so the `workqueue` can implement an exponential backoff.

When designing your state machine, ask yourself: **"If I ran this logic 100 times in a row without changing the input, would the environment change after the first time?"** If the answer is yes, your logic is not yet idempotent.

Locks and Leases for synchronization

Locks and Leases for Synchronization

In a distributed system like Kubernetes, running multiple instances of a controller is often necessary for high availability or to handle high workloads. However, running multiple replicas of the same controller can lead to "split-brain" scenarios or conflicting state updates if they all attempt to reconcile the same resources simultaneously.

To ensure consistency, we rely on **distributed synchronization primitives**: Locks and Leases.

Distributed Synchronization Fundamentals

When scaling controllers, we move from a single-process model to a distributed model. This introduces two primary challenges: 1. **Race Conditions:** Multiple controllers processing the same event. 2. **Resource Contention:** Overwriting status fields or triggering redundant external API calls.

To solve these, Kubernetes provides the `Lease` API (found in the `coordination.k8s.io` API group). A `Lease` represents a temporary, time-bound lock held by a specific process.

The Anatomy of a Lease

A `Lease` object is essentially a distributed lock with a heartbeat. It contains: * **HolderIdentity:** The identifier of the process currently holding the lock. * **RenewTime:** The timestamp of the last heartbeat. * **LeaseDurationSeconds:** How long the lock is valid before another process can claim it.

By using leases, controllers can participate in **Leader Election**. Only the controller that successfully acquires and maintains the `Lease` is permitted to perform the `Reconcile` loop.

Implementing Leader Election

In the Kubernetes ecosystem, leader election is typically handled by the `client-go` library's `leaderelection` package. This abstracts the complexity of interacting with the `Lease` API directly.

The Leader Election Lifecycle

1. **Acquisition:** The controller attempts to create or update a `Lease` resource. If successful, it becomes the Leader.
2. **Heartbeating:** The Leader periodically updates the `RenewTime` in the `Lease` object.
3. **Failsafe:** If the Leader crashes, it stops sending heartbeats. Once the `LeaseDurationSeconds` expires, the `Lease` is considered "expired."
4. **Takeover:** A standby controller observes the expired lease and attempts to acquire it, becoming the new Leader.

Hands-on Lab: Adding Leader Election to a Controller

This lab demonstrates how to wrap your controller's `Run` logic with `client-go's leader election mechanism.

Prerequisites

- A working `client-go` environment.
- Permission to create `Lease` objects in the target namespace.

Step 1: Define the Lock Identity

You must uniquely identify your controller instance. Often, this is the Pod name.

```
import (
    "os"
    "k8s.io/client-go/tools/leaderelection/resourcelock"
)

id := os.Getenv("HOSTNAME") // Unique ID for this replica
```

Step 2: Configure the Leader Election

Use the `resourcelock` to define where the lock lives.

```
lock := &resourcelock.LeaseLock{
    LeaseMeta: metav1.ObjectMeta{
        Name:      "my-controller-lock",
        Namespace: "kube-system",
    },
    Client: client.CoordinationV1(),
    LockConfig: resourcelock.ResourceLockConfig{
        Identity: id,
```

```
    },
}
```

Step 3: Start the Election Loop

Wrap your main controller logic inside the `OnStartedLeading` callback.

```
leaderelection.RunOrDie(ctx, leaderelection.LeaderElectionConfig{
    Lock:            lock,
    ReleaseOnCancel: true,
    LeaseDuration:   15 * time.Second,
    RenewDeadline:   10 * time.Second,
    RetryPeriod:     2 * time.Second,
    Callbacks: leaderelection.LeaderCallbacks{
        OnStartedLeading: func(ctx context.Context) {
            // Your primary controller logic starts here
            runMyController(ctx)
        },
        OnStoppedLeading: func() {
            log.Printf("Leader lost, shutting down.")
        },
    },
})
```

Key Takeaways

- **Idempotency is king:** Even with leader election, your reconciliation logic must remain idempotent, as network partitions can theoretically allow brief periods where two controllers believe they are leaders.
- **Leases > Annotations:** Always use the dedicated `Lease` API rather than custom annotations on ConfigMaps, as `Lease` provides built-in support for time-to-live (TTL) and atomic updates.
- **Observe then Act:** Use informers to track the status of the lease and ensure your controller reacts appropriately when it loses its leadership status.

Leader election mechanisms

Leader Election Mechanisms in Kubernetes Controllers

In a distributed system, running multiple instances of a controller is often necessary for high availability (HA). However, if multiple controllers attempt to modify the same set of resources simultaneously, it leads to race conditions, conflicting state transitions, and "split-brain" scenarios.

Leader election is the architectural pattern that ensures only one instance of a controller (the "leader") is actively processing the workqueue, while other instances (the "followers") remain in a standby state, ready to take over if the leader fails.

The Role of Leases

Kubernetes utilizes the `coordination.k8s.io/v1` API `Lease` resource to manage distributed locking. A `Lease` acts as a heartbeat-based lock:

- **Atomic Updates:** The `kube-apiserver` ensures that `Lease` updates are atomic, allowing only one client to successfully claim or renew the lock.
- **Time-to-Live (TTL):** Each lease has a defined duration. If the leader fails to renew its lease before the expiration, other instances perceive this as a failure and initiate a new election.
- **Liveness:** The `client-go` library provides the tooling to abstract this complexity, allowing developers to focus on the `OnStartedLeading` and `OnStoppedLeading` lifecycle hooks.

Leader Election Lifecycle

The `client-go/tools/leaderelection` package manages the state machine for the leader:

1. **Candidate Phase:** All controller instances attempt to acquire the lease.
2. **Acquisition:** The first instance to successfully create or update the `Lease` object becomes the leader.
3. **Renewal:** The leader periodically updates the `Lease` resource (typically via a heartbeat interval) to signal it is still operational.
4. **Failover:** If the leader stops renewing the lease, the TTL expires. Followers detect the stale lease and attempt to acquire it, promoting one of themselves to the new leader.

Hands-on Lab: Implementing Leader Election

In this lab, you will modify your existing `client-go` controller to support leader election using the `leaderelection` package.

Prerequisites

- A functional `client-go` controller from the previous module.
- Access to a Kubernetes cluster (e.g., `kind` or `minikube`).

Step 1: Initialize the Leader Election Configuration

Import the necessary packages and define the `LeaderElectionConfig`.

```
import (
    "k8s.io/client-go/tools/leaderelection"
    "k8s.io/client-go/tools/leaderelection/resourcelock"
)

// Configure the lock using a Lease object
lock := &resourcelock.LeaseLock{
    LeaseMeta: metav1.ObjectMeta{
        Name:      "my-controller-lock",
        Namespace: "kube-system",
```

```
    },
    Client: clientset.CoordinationV1(),
    LockConfig: resourcelock.ResourceLockConfig{
        Identity: podName, // Unique identifier for this instance
    },
}
```

Step 2: Running the Controller with Election

Wrap your controller's start logic within the `RunOrDie` function. This ensures your controller logic only executes when the instance holds the lock.

```
leaderelection.RunOrDie(context.TODO(), leaderelection.LeaderElectionConfig{
    Lock:            lock,
    ReleaseOnCancel: true,
    LeaseDuration:   15 * time.Second,
    RenewDeadline:   10 * time.Second,
    RetryPeriod:     2 * time.Second,
    Callbacks: leaderelection.LeaderCallbacks{
        OnStartedLeading: func(ctx context.Context) {
            // Start your controller's Run() loop here
            controller.Run(ctx)
        },
        OnStoppedLeading: func() {
            log.Info("Leader lost: shutting down")
            os.Exit(0)
        },
    },
})
```

Step 3: Verification

1. Deploy two pods running your controller image to the cluster.
2. Check the logs of both pods: `kubectl logs -f <pod-name-1>` `kubectl logs -f <pod-name-2>`
3. You will observe that only one pod reports "Started leading," while the other remains in a waiting state.
4. Delete the leader pod: `kubectl delete pod <leader-pod-name>`.
5. Observe the second pod automatically assuming the leadership role.

Troubleshooting Tips

- **Permissions:** Ensure the ServiceAccount used by your controller has `get`, `create`, and `update` permissions on the `leases` resource in the specified namespace.
- **Clock Skew:** Leader election relies on the `kube-apiserver` time; ensure your node clocks are synchronized via NTP.
- **Lease Contention:** If your TTL (`LeaseDuration`) is too short, transient network issues may cause

frequent, unnecessary leader re-elections (flapping).

Controller sharding strategies

Controller Sharding Strategies

In distributed systems, a single controller instance can quickly become a bottleneck, especially when managing thousands of resources or handling high-frequency event streams. When the cluster size grows or the reconciliation logic becomes computationally expensive, we must move beyond a single, monolithic controller to a **sharded architecture**.

What is Controller Sharding?

Controller sharding is the practice of splitting the workload of a controller across multiple instances, where each instance is responsible for only a subset of the total resources. Unlike **Leader Election** (where only one instance performs work and others stay on standby), sharding allows all instances to be "active," horizontally scaling the reconciliation throughput.

Why Shard?

- **Throughput**: Distribute the processing load of reconciliation loops.
- **Memory Efficiency**: Reduce the memory footprint by having each instance cache only its shard of the total state.
- **Blast Radius Reduction**: A failure in one shard does not impact resources managed by other shards.

Sharding Mechanics

To implement sharding, you must define a deterministic way to assign a resource to a specific controller instance.

Sharding requires **consistent hashing** or **modulo arithmetic** based on a resource identifier (like the Name or UID of the object). If you use `hash(resource.Name) % total_instances`, you ensure that a specific resource is always handled by the same shard, preventing "flapping" where different instances fight over the same resource.

Strategies for Sharding

1. **Static Sharding**: You manually assign IDs to controller instances (e.g., `shard-0`, `shard-1`). This is simple but requires manual intervention when scaling the number of instances.
2. **Dynamic Sharding via Ownership**: Use a `Lease` or a custom `Shard` resource. Instances monitor the `Shard` objects. If an instance sees a `Shard` object assigned to its ID, it begins reconciling those resources.
3. **Label-Based Filtering**: Controllers use an `Informer` that includes a label selector filter. Each controller is started with an argument (e.g., `--shard-id=1`) and only watches objects that match that shard's label.

Case Study: OpenShift Ingress Controller

In large OpenShift clusters, the `openshift-ingress-controller` manages thousands of Routes. To avoid overwhelming the control plane, it utilizes ingress sharding: * **Mechanism**: Routes are annotated with `router.openshift.io/name`. * **Implementation**: Multiple ingress controller deployments are created, each configured to watch only specific route shards based on this annotation. * **Result**: This prevents a single controller from becoming the bottleneck for cluster-wide ingress configuration.

Hands-on Conceptual Exercise: Designing a Sharded Controller

In this exercise, you will define how to modify a standard `client-go` controller to support sharding.

```
// Example: Basic Sharding Logic
func shouldReconcile(objectName string, shardID int, totalShards int) bool {
    // Convert string name to hash, then use modulo to determine ownership
    hash := fnv.New32a()
    hash.Write([]byte(objectName))
    return int(hash.Sum32()) % totalShards == shardID
}
```

Lab Steps

1. **Modify your Informer**: Update your `Informer` to include a `LabelSelector`.
2. **Inject Shard Config**: Pass `--shard-id` and `--total-shards` as flags to your container.
3. **Filter Events**: In your `FilterFunc` (within the Informer's `AddEventHandler`), verify the resource owner via the `shouldReconcile` function above.
4. **Observe**: Deploy two instances of your controller with different `--shard-id` values. Create resources with naming patterns and observe how only the specific shard logs the reconciliation event.

Troubleshooting Tips

- **Consistent Hashing**: If you change the `totalShards` value, existing resources will be re-assigned. Ensure your controller handles "cleanup" of resources it no longer owns.
- **Overlapping Shards**: In some complex setups (like ingress), you might allow overlapping shards to provide high availability (if one shard dies, the other covers the gap). This requires careful handling of **optimistic locking** (resource versions) to prevent concurrent modification conflicts.

Hands-on Lab: Adding optimistic locking through Lease for leader election on the client-go controller.

Hands-on Lab: Implementing Leader Election with `client-go`

In a distributed environment, running multiple instances of the same controller can lead to race

conditions, double-processing of events, and conflicting state updates. To prevent this, we use **Leader Election**, which ensures that only one instance of your controller acts as the "Leader" and executes the reconciliation loop, while others stand by as "Followers."

In this lab, we will use the `k8s.io/client-go/tools/leaderelection` package to implement a leader election mechanism using Kubernetes `Lease` objects.

Prerequisites

- A working `client-go` controller (from Exercise 1).
- `k8s.io/client-go` and `k8s.io/apimachinery` modules initialized in your `go.mod`.
- A running Kubernetes cluster (or `kind`).

Step 1: Define the Lock Configuration

The `leaderelection` package requires a `LeaderElectionConfig`. This struct defines the identity of your controller, the resource lock (Lease), and the callback functions for when the controller gains or loses leadership.

Add the following configuration to your `main.go` or controller initialization logic:

```
import (
    "k8s.io/client-go/tools/leaderelection"
    "k8s.io/client-go/tools/leaderelection/resourcelock"
    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
)

// Define the lease lock
lock := &resourcelock.LeaseLock{
    LeaseMeta: metav1.ObjectMeta{
        Name:      "my-controller-lock",
        Namespace: "default",
    },
    Client: clientset.CoordinationV1(),
    LockConfig: resourcelock.ResourceLockConfig{
        Identity: podName, // Unique identifier for this instance (e.g., hostname)
    },
}
```

Step 2: Implement the Leader Election Loop

Instead of starting your controller's `Run()` method directly, wrap it within the `leaderelection.RunOrDie` function. This function handles the logic of acquiring the lease and calling your reconciliation logic only when leadership is held.

```
leaderelection.RunOrDie(ctx, leaderelection.LeaderElectionConfig{
    Lock:            lock,
    ReleaseOnCancel: true,
```

```
    LeaseDuration:   15 * time.Second,
    RenewDeadline:   10 * time.Second,
    RetryPeriod:     2 * time.Second,
    Callbacks: leaderelection.LeaderCallbacks{
        OnStartedLeading: func(ctx context.Context) {
            // This is where your controller's main loop starts
            runMyController(ctx)
        },
        OnStoppedLeading: func() {
            log.Printf("Leader lost; exiting.")
            os.Exit(0)
        },
        OnNewLeader: func(identity string) {
            log.Printf("New leader elected: %s", identity)
        },
    },
})
```

Step 3: Understanding the Leases

When you run this controller: 1. The `client-go` library creates a `Lease` object in the `coordination.k8s.io` API group. 2. The controller attempts to acquire the lock by updating the `HolderIdentity` field in the Lease. 3. If successful, it proceeds to start the `Informer` and `Workqueue`. 4. If it fails to renew the lease within the `RenewDeadline`, it loses leadership and stops the controller logic.

Step 4: Verification

1. **Deploy the controller:** Deploy two replicas of your controller pod in the same namespace.
2. **Check the logs:** Observe that only one pod logs "New leader elected" and begins processing items.
3. **Inspect the Lease:**

   ```
   kubectl get lease my-controller-lock -o yaml
   ```

   Observe the `holderIdentity` field matching the pod name of the current leader.
4. **Simulate Failure:** Delete the leader pod. Notice how the second pod automatically detects the expiry of the lease, assumes leadership, and begins the reconciliation loop.

Troubleshooting

- **RBAC:** Ensure the ServiceAccount used by your pod has `get`, `create`, `update`, and `patch` permissions on `leases` resources in the `coordination.k8s.io` group.
- **Clock Skew:** Ensure your nodes are NTP-synchronized, as leader election relies heavily on timeouts and lease durations.

3.1 — Taking more control: Admissions and beyond the kube-apiserver to writing your own API Server

3.1 — Taking more control: Admissions and beyond the kube-apiserver

This module covers advanced extension mechanisms for the Kubernetes control plane, focusing on intercepting and customizing API requests, as well as extending the API surface itself.

Key concepts include:

- **Admission Control:** Implementing `Validating` and `Mutating` webhooks to enforce business logic and policy enforcement, and understanding how these differ from specialized Policy Engines.
- **API Aggregation:** Utilizing the API Aggregation Layer to register and serve custom APIs alongside standard Kubernetes resources.
- **Custom API Servers:** Designing and deploying standalone API servers that integrate seamlessly with the cluster, including considerations for API discovery.
- **Extensibility Patterns:** Analyzing real-world use cases, such as the Operator Lifecycle Manager (OLM) package manager, for implementing custom API servers.

Admission control webhooks (Validating and Mutating)

Admission Control Webhooks: Validating and Mutating

In the Kubernetes request lifecycle, the `kube-apiserver` acts as the gatekeeper. While Authentication and Authorization verify **who** is making a request and **what** they can do, **Admission Control** determines **whether** the object itself is valid, or if it should be modified before being persisted to `etcd`.

Admission control is implemented as a set of plugins that run after a request is authenticated and authorized, but before the object is stored. When built-in admission controllers are insufficient, Kubernetes provides **Dynamic Admission Control** via Webhooks.

The Admission Webhook Lifecycle

An admission webhook is an HTTP callback. When the `kube-apiserver` receives a request (e.g., `POST /apis/apps/v1/namespaces/default/deployments`), it sends an `AdmissionReview` object to your webhook server.

1. Mutating Admission Webhooks

Mutating webhooks are invoked first. They allow you to modify the object before it is persisted. * **Use Case:** Injecting sidecars, setting default resource limits, or adding default labels/annotations. * **Mechanism:** The webhook returns a JSON Patch that describes the modifications the `kube-apiserver` should apply to the resource.

2. Validating Admission Webhooks

Validating webhooks are invoked after mutations are complete. * **Use Case:** Enforcing compliance, security policies, or organizational constraints (e.g., "Only allow images from the internal registry"). * **Mechanism:** The webhook returns a simple "allow" or "deny" response. If denied, the request is rejected with a message returned to the user.

If a webhook fails (e.g., network timeout), the `failurePolicy` configuration determines the outcome: * `Ignore`: The request continues as if the webhook didn't exist. * `Fail`: The request is rejected, preventing the potentially un-validated resource from entering the cluster.

Webhooks vs. Policy Engines

While you can build custom webhooks for any validation logic, the ecosystem has shifted toward **Policy Engines** (e.g., OPA Gatekeeper, Kyverno).

- **Custom Webhooks:** Best for complex, high-performance logic that requires deep integration with external systems or highly specific business logic that is difficult to express declaratively.
- **Policy Engines:** These are specialized admission controllers that provide a declarative language (like Rego or YAML) to define policies. They are easier to audit, manage, and share across teams compared to writing and maintaining custom Go binaries.

Hands-on Lab: Custom Validating Webhook

In this exercise, you will create a webhook that restricts image registries.

Prerequisites

- A running Kubernetes cluster (e.g., Kind or OpenShift).
- `cert-manager` installed (to handle TLS for your webhook server).

Step 1: Define the Webhook Configuration

The `ValidatingWebhookConfiguration` tells the `kube-apiserver` which resources to watch and where to send the requests.

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: image-registry-validator
```

```
webhooks:
  - name: image-validator.example.com
    rules:
      - apiGroups: ["apps"]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources: ["deployments"]
    clientConfig:
      service:
        name: webhook-server
        namespace: default
        path: "/validate"
    admissionReviewVersions: ["v1"]
    sideEffects: None
```

Step 2: Implement the Validation Logic (Pseudo-code)

Your Go binary must accept the `AdmissionReview` object, inspect the `Deployment` spec, and respond.

```
func validateImage(ar admissionv1.AdmissionReview) admissionv1.AdmissionResponse {
    // 1. Unmarshal the object from ar.Request.Object.Raw
    // 2. Iterate over containers in the deployment
    // 3. If image does not start with "trusted-registry.io/":
    return admissionv1.AdmissionResponse{
        Allowed: false,
        Result: &metav1.Status{Message: "Unauthorized registry!"},
    }
}
```

Step 3: Deployment & Testing

1. **Build and deploy** your binary as a `Deployment` inside the cluster.
2. **Expose** the service with a `ClusterIP` on port 443.
3. **Create a TLS Secret** and mount it into your pod so the `kube-apiserver` can verify your webhook's identity.
4. **Test:** Try to create a Deployment with `image: nginx`. The `apiserver` should reject it if your logic requires `trusted-registry.io/`.

When testing, check the `apiserver` logs if your requests hang; they often contain detailed information about why a webhook handshake failed (e.g., TLS certificate mismatch).

Differences between webhooks and Policy Engines

Understanding Admission Control: Webhooks vs. Policy Engines

In the Kubernetes ecosystem, ensuring that resources conform to security, compliance, and organizational standards is critical. While both Admission Webhooks and Policy Engines serve as "gatekeepers" to the `kube-apiserver`, they differ significantly in their operational philosophy, lifecycle management, and intended use cases.

Admission Webhooks: The Native Extension Mechanism

Admission Webhooks are a core feature of the Kubernetes API server. They allow you to define HTTP callbacks that are triggered during the request lifecycle—specifically, after the request is authenticated and authorized, but before the object is persisted in `etcd`.

There are two primary types of admission webhooks: * **Mutating Webhooks:** These allow you to modify the object before it is stored. Common uses include injecting sidecars, setting default values (like default resource limits), or adding specific labels. * **Validating Webhooks:** These allow you to verify the object against a set of rules and reject the request if the object does not meet specific criteria.

Key Characteristics

- **Development:** They are custom-built applications. You must write code (usually in Go using `client-go` or `controller-runtime`), handle TLS certificates, manage high availability, and maintain the deployment of the webhook server itself.
- **Coupling:** They are highly coupled to the specific logic you write. If you need to change a rule, you often need to update the application code, re-build, and re-deploy.
- **Performance:** Because they are synchronous calls in the API request pipeline, poorly performing webhooks can induce latency in the `kube-apiserver` and even cause cluster-wide outages if they fail to respond.

Policy Engines: The Declarative Abstraction

Policy Engines (such as OPA/Gatekeeper or Kyverno) are essentially sophisticated implementations of admission webhooks. Instead of hard-coding logic into a web server, they provide a declarative framework where you define policies as data (using languages like Rego for OPA or YAML-based DSLs for Kyverno).

Why use a Policy Engine instead of a custom Webhook?

| Feature | Custom Webhook | Policy Engine | | :--- | :--- | :--- | | **Logic** | Procedural (Go code) | Declarative (Rego/YAML) | | **Flexibility** | High (can perform any logic) | High (optimized for policy) | | **Lifecycle** | Manual dev/deploy cycle | Centralized management | | **Auditability** | Difficult (requires reading code) | Built-in (policy-as-data) |

Strategic Advantages

1. **Separation of Concerns:** Policy engines separate the "engine" (the software that enforces rules) from the "policy" (the rules themselves). This allows security teams to manage policies independently of developers.

2. **Audit and Governance:** Policy engines often provide built-in visibility, allowing you to see which resources are out of compliance across the entire cluster without needing to manually query individual webhook logs.
3. **Dry-run/Audit Mode:** Many policy engines allow you to run policies in "audit" mode. This logs non-compliant resources without actually blocking them, providing a safer way to roll out new security requirements without breaking existing workloads.
4. **Extensibility:** Policy engines handle the complexities of admission (such as TLS handling, retries, and failure policies) so you don't have to build those features from scratch.

When to choose which?

- **Choose a custom Admission Webhook when:** You need to perform complex business logic that cannot be expressed via policy (e.g., calling an external legacy database to validate a resource, or performing complex data transformations that require specific libraries).
- **Choose a Policy Engine when:** You are focused on security, compliance, or best practices (e.g., "all images must come from our internal registry," "all pods must have resource limits," or "prevent privileged containers").

Summary for Operators

While custom webhooks offer the raw power of the Kubernetes API, Policy Engines are the preferred architectural choice for scaling governance. They transform security requirements from "black-box" code deployments into transparent, auditable, and version-controlled configuration, aligning perfectly with the Kubernetes philosophy of declarative state management.

API discovery and Aggregation Layer

API Discovery and the Aggregation Layer

In a standard Kubernetes deployment, the `kube-apiserver` handles all core API requests. However, as your cluster grows or requires specialized resources that don't fit the standard CRUD models, you need a way to extend the API surface area. The **API Aggregation Layer** and **Discovery API** provide the mechanisms to make these extensions feel like native parts of the cluster.

The API Aggregation Layer

The API Aggregation Layer allows you to add custom APIs to your Kubernetes cluster by running an additional API server alongside the primary `kube-apiserver`.

Instead of routing requests for these custom resources to the core `kube-apiserver` and relying on CRDs, the aggregation layer forwards these requests to your custom API server.

How it Works

1. **Registration**: Your custom API server registers itself with the primary `kube-apiserver` by creating an `APIService` object.
2. **Proxying**: When a user (or `kubectl`) makes a request to the `/apis/mycompany.com/v1/…` endpoint, the `kube-apiserver` recognizes that this path is managed by an aggregated server.

3. **Forwarding**: The `kube-apiserver` acts as a proxy, forwarding the request to your custom server. Your server handles authentication and authorization, processes the request, and returns the result to the client.

Why use the Aggregation Layer?

- **API Control**: You have full control over the storage layer and the internal logic of your API, which is not possible with standard CRDs.
- **Custom Logic**: It is ideal for implementing complex APIs (like the OLM Package Manager) that require non-standard interaction patterns.
- **Seamless Integration**: To the end-user, the resource looks exactly like a native Kubernetes object.

API Discovery

Kubernetes clients (such as `kubectl` or custom controllers using `client-go`) need to know what APIs are available. The **Discovery API** allows clients to programmatically query the API server to understand the available groups, versions, and resources.

The Discovery Flow

When you run `kubectl api-resources`, it performs a discovery request to identify the supported resources in the cluster.

- **Standard Discovery**: The client hits the `/api` or `/apis` endpoints.
- **Aggregated Discovery**: Since aggregated servers may add new resources, they must expose a discovery endpoint that the `kube-apiserver` includes in the cluster's overall discovery information.

Troubleshooting Discovery

When working with custom API servers, ensure that the `APIService` object is correctly configured so that the `kube-apiserver` can successfully reach your service's discovery endpoint.

A common debugging tip is to check the discovery response directly to ensure your API group is correctly exposed.

Testing API Discovery

```
# Use oc proxy or kubectl proxy to access the API locally
# Requesting the discovery list for a specific group version
curl -v http://127.0.0.1:8001/api \
  -H "Accept: application/json;v=v2;g=apidiscovery.k8s.io;as=APIGroupDiscoveryList"
```

If you receive an empty response or unexpected data, verify that your `APIService` definition points to the correct `Service` and `port` where your custom API server is listening. The `kube-apiserver` must be able to reach your pod over the network to fetch these discovery documents.

Key Differences: Aggregated Discovery vs. Traditional

- **Traditional**: Relies on discovery documents generated statically by the core API server.
- **Aggregated**: Enables a dynamic, "pluggable" architecture where the cluster's API surface area can evolve at runtime as new API services are registered.

Further Reading

- Kubernetes Discovery API Documentation
- KEP: Aggregated Discovery
- Sample API Server implementation

Implementing custom API servers

Implementing Custom API Servers

When the standard Kubernetes Custom Resource Definition (CRD) mechanism is insufficient for your architectural requirements, the Kubernetes API Aggregation Layer provides the extensibility needed to run your own API server.

When to move beyond CRDs

While CRDs are the standard for extending Kubernetes, they are managed by the built-in `kube-apiserver`. A custom API server is necessary when: * **Complex Data Storage:** Your data does not map well to the etcd-based model or requires a completely different storage backend (e.g., legacy databases, external SaaS APIs). * **Granular API Semantics:** You require advanced HTTP path handling, custom serialization logic, or proxying behavior that the standard CRUD-focused `kube-apiserver` cannot provide. * **Aggregated Discovery:** You want your custom resources to appear as native first-class citizens in the discovery document (`/apis`) without the overhead of the standard `kube-apiserver` controllers.

The API Aggregation Layer

The API Aggregation Layer allows you to add additional APIs to your cluster by running a separate server process. The `kube-apiserver` acts as a proxy, forwarding requests to your service.

```
graph LR
    User((Client)) --> APIServer[kube-apiserver]
    APIServer -->|/apis/myapi.example.com| AggregatedServer[Custom API Server]
    APIServer -->|Standard Resources| etcd[(etcd)]
```

Key Components of a Custom API Server

To implement an API server, you typically utilize the `k8s.io/apiserver` library, which provides the foundational plumbing:

1. **GenericAPIServer:** The core component that handles the HTTP/S stack, authentication, and

authorization.

2. **Storage Backends:** You define how objects are stored and retrieved. You are not forced to use etcd; you can implement custom storage interfaces.
3. **Discovery API:** To expose your API to `kubectl`, your server must implement the `APIGroupDiscovery` interface.

When debugging discovery for your custom server, use the aggregated discovery header to ensure you are seeing the latest API metadata: `curl -v http://127.0.0.1:8001/apis -H "Accept: application/json;v=v2;g=apidiscovery.k8s.io;as=APIGroupDiscoveryList"`

Architectural Case Study: OLM Package Server

The Operator Lifecycle Manager (OLM) `package-server` is a prime example of an aggregated API server. It provides a read-only interface for package manifests. Because the data volume can be large and the query patterns specific, the maintainers chose to use an aggregated server rather than CRDs to offload the `kube-apiserver` and provide a tailored response format.

Implementation Steps

1. **Define your API Schema:** Use `apimachinery` and `api` libraries to define the resources your server will serve.
2. **Initialize the Server:** Use the `genericapiserver` library to create a server instance.
3. **Register the API:** Create an `APIService` object in the cluster that points to your service running inside the cluster.
4. **Handling Auth:** Because you are running a custom server, you must ensure it trusts the cluster's CA or handles `TokenReview` requests to verify the identity of the incoming callers forwarded by the `kube-apiserver`.

Recommended Resources

- **Reference Implementation:** Kubernetes Sample API Server
- **Case Study:** [OLM Package Server Source](https://github.com/openshift/operator-framework-olm/tree/main/cmd/package-server-manager)
- **K8s Docs:** [Extending the Kubernetes API](https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/customresources/#choosing-a-method-for-adding-custom-resources)

3.2 — The other stuff

3.2 — The other stuff

This module covers essential Kubernetes ecosystem components that extend beyond core controllers and the API server. It focuses on the architectural features required for production-grade environments:

- **Advanced Networking:** Evolution of traffic management, covering Ingress and the Gateway API.
- **Service Mesh:** Concepts and implementation for managing microservice communication.
- **Authentication and Authorization:** Specialized AuthN/AuthZ mechanisms specific to the OpenShift platform.
- **Cluster Security and Hardening:** Implementing fine-grained security controls, including RBAC, Security Context Constraints (SCC), and Seccomp profiles.

Advanced networking: Ingress and Gateway API

Advanced Networking: Ingress and Gateway API

In the Kubernetes ecosystem, managing external access to services is a critical architectural requirement. While initial implementations relied on basic primitives, the community has evolved toward more expressive, role-oriented, and extensible standards.

The Evolution of Traffic Management

Historically, the `Ingress` resource served as the standard for exposing HTTP and HTTPS routes. However, as Kubernetes deployments grew in complexity, the `Ingress` API faced limitations: it lacked standardization across providers (relying on implementation-specific annotations), had a rigid structure, and provided limited support for advanced traffic shifting or header-based routing.

To address these shortcomings, the **Gateway API** was introduced as the modern, successor standard.

Ingress vs. Gateway API

Ingress: The Classic Approach

The `Ingress` resource describes a collection of rules that allow inbound connections to reach the cluster services. * **Strengths:** Simple to deploy; broad support across almost all cloud providers and ingress controllers (e.g., Nginx, HAProxy). * **Weaknesses:** Vendor lock-in via custom annotations; lack of separation between infrastructure management and application routing.

Gateway API: The Modern Standard

The Gateway API is a collection of resources (GatewayClass, Gateway, HTTPRoute) that model service networking as an infrastructure-managed asset. * **Role-Oriented:** Separates concerns by

allowing cluster operators to manage infrastructure (Gateway) while developers define traffic routing (HTTPRoute). * **Extensible:** Built to support complex traffic patterns, including weighted traffic splitting, header-based routing, and cross-namespace sharing by default. * **Expressive:** Standardizes features that previously required diverse, non-portable annotations.

If you are designing a new system or migrating a complex architecture, prioritize the **Gateway API**. It is the community-accepted path forward for Kubernetes networking, offering better consistency across different implementations.

Hands-on Lab: Routing Traffic with Gateway API

This activity demonstrates how to define a `Gateway` and an `HTTPRoute` to expose a service.

Prerequisites

Ensure your cluster has a Gateway API-compatible implementation installed (e.g., Istio, Cilium, or Contour). Verify by checking for the presence of the Gateway API Custom Resource Definitions (CRDs).

Step 1: Define a Gateway

The `Gateway` resource defines the entry point into your cluster.

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: example-gateway
  namespace: default
spec:
  gatewayClassName: istio # Replace with your provider's class name
  listeners:
  - name: http
    protocol: HTTP
    port: 80
```

Step 2: Define an HTTPRoute

The `HTTPRoute` attaches to the `Gateway` and specifies how to route traffic to your application service.

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app-route
spec:
  parentRefs:
  - name: example-gateway
  rules:
  - matches:
    - path:
```

```
        type: PathPrefix
        value: /api
    backendRefs:
    - name: my-api-service
      port: 8080
```

Step 3: Apply and Verify

1. Apply the manifests using `oc apply -f <file.yaml>`.
2. Check the status of your gateway to ensure it is programmed:

   ```
   oc get gateway example-gateway -o yaml
   ```

3. Ensure the `status.conditions` indicate that the Gateway is "Accepted" and "Programmed".

Advanced Concepts

- **GAMMA (Gateway API for Mesh Management):** This initiative extends Gateway API semantics to Service Mesh, allowing you to use the same `HTTPRoute` resources to manage internal Pod-to-Pod traffic alongside external ingress traffic.
- **Service Mesh:** Unlike the Gateway API (which focuses on edge routing), a Service Mesh (e.g., Istio, Linkerd) provides observability, security (mTLS), and reliability (retries/circuit breaking) for service-to-service communication.

Service Mesh concepts

Service Mesh Concepts

In a microservices architecture, managing inter-service communication becomes increasingly complex as the number of services grows. While Kubernetes provides native `Service` resources for basic discovery and load balancing, it does not inherently solve challenges like observability, fine-grained traffic control, and secure communication (mTLS) between services. This is where a **Service Mesh** becomes essential.

What is a Service Mesh?

A Service Mesh is an infrastructure layer dedicated to handling service-to-service communication. It abstracts the networking logic out of the application code and pushes it into the infrastructure, typically implemented as a set of lightweight proxies deployed alongside each service instance.

These proxies are known as **Sidecars**. When a service sends a network request, it passes through its local sidecar, which manages the communication, security, and telemetry before forwarding the request to the destination service's sidecar.

Core Capabilities

- **Traffic Management:** Advanced routing (canary deployments, blue-green, circuit breaking, and

traffic splitting).

- **Observability:** Automated golden metrics (latency, traffic, error rates) and distributed tracing without application changes.
- **Security:** Mutual TLS (mTLS) by default, service-to-service authentication, and fine-grained authorization policies.
- **Resiliency:** Retries, timeouts, and circuit breaking to prevent cascading failures.

The Sidecar Architecture vs. Proxyless

In a traditional service mesh (like Istio), the sidecar pattern (typically using Envoy Proxy) is used. The control plane pushes configuration to these proxies to dictate how traffic should flow.

Recently, the industry has begun exploring "proxyless" or "ambient" models, where the infrastructure handles traffic via node-level agents rather than injecting a sidecar container into every single Pod, reducing resource overhead.

Integrating with Gateway API (GAMMA)

The **Gateway API** has expanded its scope beyond North-South traffic (ingress) to include East-West traffic (service-to-service). The **GAMMA (Gateway API for Mesh Management and Administration)** initiative aims to standardize how service meshes are configured using the same Kubernetes-native API resources used for ingress.

This allows developers to manage internal traffic routing using familiar `HTTPRoute` or `TCPRoute` objects, providing a unified interface for both edge and internal traffic management.

Hands-on: Observing Service Mesh Traffic Flow

While a full Service Mesh installation (e.g., Istio) requires a complex control plane, you can visualize the concept of traffic interception using a simple sidecar pattern.

```
# 1. Define a deployment with a sidecar proxy container
cat <<EOF > deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 1
  template:
    spec:
      containers:
      - name: app
        image: nginx
      - name: sidecar-proxy
        image: envoyproxy/envoy
        args: ["-c", "/etc/envoy/envoy.yaml"]
EOF
```

```
# 2. Apply the manifest
kubectl apply -f deployment.yaml

# 3. Verify that the pod contains both the application and the proxy
kubectl get pods -o jsonpath='{.items[0].spec.containers[*].name}'
# Output: app sidecar-proxy
```

Key Takeaways

- **Decoupling:** Service Mesh decouples the "how" of networking from the "what" of business logic.
- **Standardization:** Use the Gateway API (GAMMA) to future-proof your mesh configurations as the ecosystem moves toward unified traffic standards.
- **Observability:** Service meshes are often adopted primarily for the "instant observability" they provide to legacy applications.

When deciding whether to adopt a Service Mesh, weigh the operational complexity of managing the Control Plane against the necessity of features like mTLS and distributed tracing. For smaller clusters, Kubernetes native `NetworkPolicies` and basic `Services` are often sufficient.

AuthN/AuthZ in OpenShift

AuthN/AuthZ in OpenShift

In the Kubernetes ecosystem, Authentication (AuthN) and Authorization (AuthZ) are the gatekeepers of cluster security. While upstream Kubernetes provides the foundational mechanisms, Red Hat OpenShift extends these with integrated identity providers and granular security policies designed for enterprise environments.

Authentication (AuthN) in OpenShift

Unlike standard Kubernetes, which often relies on static tokens or client certificates for administrative access, OpenShift provides a centralized `OAuth` server.

- **OAuth Server:** OpenShift includes a built-in OAuth server that allows users to obtain access tokens for authenticating to the API.
- **Identity Providers:** You can integrate OpenShift with external identity sources such as LDAP, GitHub, Google, GitLab, or OIDC providers.
- **The User/Identity Resource:** In OpenShift, an `Identity` object represents a login from an external provider, which is mapped to a `User` object within the cluster.

Authorization (AuthZ) in OpenShift

Authorization determines what an authenticated user is permitted to do. OpenShift builds upon Kubernetes Role-Based Access Control (RBAC) but adds significant enhancements to support multi-tenancy and high-security requirements.

1. RBAC and ClusterRoles

OpenShift uses standard Kubernetes RBAC to control access to API resources. However, it ships with a comprehensive set of default `ClusterRoles` that are pre-configured to ensure the "principle of least privilege" is easier to manage.

2. Security Context Constraints (SCC)

This is a critical differentiator. While upstream Kubernetes uses `SecurityContext` at the Pod or Container level, OpenShift uses **Security Context Constraints (SCC)** to control the actions a pod can perform and what it has the ability to access.

- **Policy Enforcement:** SCCs control features like:
- Running privileged containers.
- Accessing host namespaces (PID, IPC, Network).
- Capabilities and seccomp profiles.
- Volume mounting requirements.
- **How it works:** When a pod is created, the controller attempts to assign the least restrictive SCC for which the user has permission. If no SCC matches, the pod is rejected.

3. The 'system:masters' Group

A key security concept in both K8s and OpenShift is the `system:masters` group. This group bypasses all RBAC authorization checks. In an enterprise environment, membership in this group should be strictly limited to the initial bootstrap process or emergency recovery scenarios, as it provides god-mode access to the cluster.

Comparison Summary: K8s vs. OpenShift Security

Feature	Kubernetes Native	OpenShift
Identity	User/ServiceAccount	OAuth + Identity Providers
AuthZ	RBAC / Webhooks	RBAC + SCC
Security Policy	PodSecurityAdmission (PSA)	Security Context Constraints (SCC)
Admin Access	Kubeconfig certs	OAuth tokens + `oc login`

Hands-on Lab: Inspecting SCCs

In this exercise, you will explore how OpenShift enforces security policies via SCCs.

Prerequisites: * An active OpenShift cluster. * The `oc` CLI tool logged in as a cluster administrator.

Step 1: List existing SCCs OpenShift comes with pre-defined SCCs ranging from `privileged` to `restricted`. Run the following command:

```
oc get scc
```

Step 2: Inspect a specific SCC View the details of the `restricted` SCC to understand the default constraints applied to most user pods:

```
oc describe scc restricted
```

Observe the `SELinuxContext`, `RunAsUser`, and `FsGroup` settings.

Step 3: Attempt to run a privileged pod Create a pod that attempts to run as a privileged user to see how the SCC admission controller rejects it if you lack the proper permissions:

```
# save as privileged-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: privileged-pod
spec:
  containers:
  - name: alpine
    image: alpine
    securityContext:
      privileged: true
---
oc apply -f privileged-pod.yaml
```

Troubleshooting: If the pod fails to start with an `error: pods "privileged-pod" is forbidden: violates existing SCC`, it is because your service account lacks the permission to use the `privileged` SCC. You can resolve this by adding the role to your service account, though this should be done with caution in production environments:

```
oc adm policy add-scc-to-user privileged -z default
```

Cluster security and hardening techniques

Cluster Security and Hardening Techniques

Securing a Kubernetes cluster requires a defense-in-depth approach. Because Kubernetes is a distributed system with a complex API surface, security must be applied at multiple layers: the API server, the container runtime, the network, and the workload identity.

Security Fundamentals in OpenShift and Kubernetes

While upstream Kubernetes provides the primitives for security, platforms like OpenShift extend these with opinionated defaults and hardened policy engines.

1. Fine-Grained RBAC

Role-Based Access Control (RBAC) is the primary mechanism for controlling who can access the Kubernetes API and what operations they can perform. * **Principle of Least Privilege:** Avoid using `cluster-admin` for developers. Scope access to specific namespaces using `Role` and `RoleBinding` rather than `ClusterRole` and `ClusterRoleBinding`. * **ServiceAccount Hardening:** Every Pod runs with a `ServiceAccount`. Avoid using the `default` service account; create specific identities for your workloads and ensure they are not mounted with unnecessary API tokens (`automountServiceAccountToken: false`).

2. Security Context Constraints (SCC) and Pod Security

In OpenShift, **Security Context Constraints (SCC)** govern the security capabilities of a pod. This includes: * **RunAsUser:** Defining the UID the container process runs as. * **Capabilities:** Restricting Linux kernel capabilities (e.g., preventing `CAP_SYS_ADMIN` unless explicitly required). * **SELinux/AppArmor/Seccomp:** Using these kernel-level security modules to restrict system calls and file access.

3. Admission Control for Enforcement

Static configuration is rarely enough. You must enforce security policies programmatically: * **Validating Admission Webhooks:** Intercepts requests to the API server and rejects those that violate security policies (e.g., requiring mandatory labels, forbidding images from unauthorized registries). * **Policy Engines:** Tools like OPA Gatekeeper or Kyverno provide a declarative way to manage these policies without writing custom Go code for every rule.

Hands-on Lab: Implementing a Validating Webhook for Registry Enforcement

In this lab, we will conceptualize a mechanism to prevent developers from deploying containers from unauthorized registries (e.g., non-corporate registries).

Prerequisites

- A running Kubernetes cluster (v1.20+)
- `kubectl` configured with cluster-admin access
- `cert-manager` installed for webhook certificate management

Step 1: Define the Validation Logic

The goal is to inspect the `Pod` object during the `CREATE` operation. The logic should iterate through all containers in the `spec` and verify the image prefix.

```
// Simplified validation logic
func validateImageRegistry(pod *corev1.Pod) error {
    allowedRegistry := "registry.example.com/"
    for _, container := range pod.Spec.Containers {
        if !strings.HasPrefix(container.Image, allowedRegistry) {
            return fmt.Errorf("image %s is not from allowed registry",
container.Image)
        }
```

```
    }
    return nil
}
```

Step 2: Configure the ValidatingWebhookConfiguration

Register the webhook with the `kube-apiserver` so it sends `AdmissionReview` requests to your service.

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: image-registry-validator
webhooks:
  - name: image.example.com
    rules:
      - apiGroups: [""]
        apiVersions: ["v1"]
        operations: ["CREATE"]
        resources: ["pods"]
    clientConfig:
      service:
        name: webhook-service
        namespace: default
        path: "/validate"
    admissionReviewVersions: ["v1"]
    sideEffects: None
```

Step 3: Deployment and Testing

1. Deploy your webhook controller as a standard `Deployment` in the cluster.
2. Deploy a service to expose the webhook endpoint.
3. Attempt to deploy a Pod using an image from `docker.io` (e.g., `nginx:latest`).
4. **Expected Result:** The `kubectl create` command should return an error: `Error from server (Forbidden): admission webhook "image.example.com" denied the request…`

Best Practices for Cluster Hardening

- **API Server Security:** Ensure the API server is not exposed to the public internet. Use OIDC providers for user authentication.
- **Network Policies:** By default, all pods can talk to all pods. Implement `NetworkPolicy` to restrict traffic to an allow-list model.
- **Secrets Management:** Never store sensitive data in plaintext. Integrate with external secret stores (Vault, AWS Secrets Manager) using the CSI Driver.
- **Audit Logging:** Enable audit logs to track who did what and when, ensuring you have a forensic trail for compliance.

Hands-on Lab: Writing a custom Validating Webhook that allows using container images from only approved container registries.

Hands-on Lab: Implementing a Custom Validating Webhook

In this lab, you will implement a **Validating Admission Webhook**. Admission webhooks are HTTP callbacks that receive admission requests from the `kube-apiserver` and process them to decide whether to accept or reject an operation.

Our goal is to enforce a security policy: **only container images from approved registries are permitted** when creating or updating specific resources.

Prerequisites

- A running Kubernetes cluster (e.g., `kind` or `minikube`).
- `go` version 1.20+ installed.
- `cfssl` or `openssl` for generating self-signed certificates.
- `kubectl` configured with cluster-admin access.

Step 1: Define the Webhook Logic

We will write a small Go server using the `net/http` package. The server must handle the `admission.k8s.io/v1` `AdmissionReview` object.

1. Create a new directory and initialize a Go module:

```
mkdir image-validator && cd image-validator
go mod init image-validator
```

2. Create `main.go`. The logic needs to parse the `AdmissionReview` request, inspect the `Deployment` or `Pod` spec, and check the image strings against a whitelist provided via command-line flags.

```
// Simplified logic snippet for the validator
func validateImage(image string, approvedRegistries []string) bool {
    for _, reg := range approvedRegistries {
        if strings.HasPrefix(image, reg) {
            return true
        }
    }
    return false
}
```

Step 2: Handling the AdmissionReview

The `kube-apiserver` sends an `AdmissionReview` object. Your code must: 1. Decode the JSON into an `AdmissionReview` struct. 2. Inspect `request.Object` (the resource being created/updated). 3. Validate the `image` fields against your approved list. 4. Return an `AdmissionResponse` with `Allowed: true` or `Allowed: false`.

Step 3: Securing the Webhook with TLS

The `kube-apiserver` requires HTTPS to communicate with webhooks. Since internal cluster traffic needs a trusted CA, you must:

1. Generate a CA key and certificate.
2. Generate a server key/cert for the webhook service.
3. Create a `Secret` in your namespace containing these certificates.
4. Update the `ValidatingWebhookConfiguration` to include the `caBundle` (the base64 encoded CA certificate).

Step 4: Configuration

Create the `ValidatingWebhookConfiguration` YAML. This tells the API server when to call your service.

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: image-validator-webhook
webhooks:
  - name: image-validator.example.com
    rules:
      - apiGroups: ["apps"]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources: ["deployments"]
    clientConfig:
      service:
        name: image-validator-svc
        namespace: default
        path: "/validate"
      caBundle: <BASE64_CA_CERT>
    admissionReviewVersions: ["v1"]
    sideEffects: None
```

Step 5: Deployment and Testing

1. **Deploy the Webhook**: Build your Go binary, containerize it, and deploy it to your cluster along with the associated `Service`.
2. **Apply Configuration**: Apply the `ValidatingWebhookConfiguration` YAML.

3. **Verify**: Attempt to create a Deployment using an image from `docker.io` (if not in your whitelist):

```
kubectl create deployment nginx --image=nginx:latest
```

Expected Output: The `apiserver` should reject the request with: `Error from server: admission webhook "image-validator.example.com" denied the request: image from unapproved registry.`

Troubleshooting

- **Logs**: Check the logs of your webhook pod: `kubectl logs -l app=image-validator`.
- **Connectivity**: Ensure the `kube-apiserver` can reach your pod IP or the Service endpoint.
- **Certificates**: Ensure the `caBundle` in the configuration matches the certificate used by the server exactly.

4.1 — All about kube-scheduler

4.1 — All about kube-scheduler

The `kube-scheduler` is the central component responsible for workload distribution across the cluster. It ensures that pods are placed on appropriate nodes based on resource availability and constraints.

Key responsibilities include:

- **Workload Scheduling:** Orchestrating pod placement, managing de-scheduling, and facilitating auto-scaling operations.
- **Resource Management:** Enforcing container hardware allocation through defined Requests and Limits to ensure cluster stability.
- **Dynamic Resource Allocation:** Providing advanced control for specialized hardware, such as GPUs, through dynamic resource slicing and allocation mechanisms.

Pod scheduling, de-scheduling, and auto-scaling

Pod Scheduling, De-scheduling, and Auto-scaling

In the Kubernetes ecosystem, the `kube-scheduler` is the control plane component responsible for ensuring that every Pod finds a home on a suitable Node. While it is the orchestrator of placement, scaling and de-scheduling mechanisms provide the dynamic adjustments required for high-availability and cost-optimized clusters.

The Scheduling Lifecycle

The `kube-scheduler` operates by watching the `kube-apiserver` for new, unscheduled Pods. Its decision-making process follows a two-step sequence:

1. **Filtering (Predicates):** The scheduler identifies all Nodes that satisfy the Pod's resource requirements (CPU/Memory requests), node selectors, affinity rules, and taints/tolerations.
2. **Scoring (Priorities):** The remaining Nodes are ranked based on a set of scoring plugins. These plugins consider factors like spreading workloads across availability zones, resource usage balance, and hardware-specific affinities.

Resource Requests and Limits

Proper scheduling relies on the **Resource Model**. Defining `requests` (the amount guaranteed to a container) and `limits` (the maximum allowed) is critical: * **Requests:** Used by the scheduler to determine if a Node has enough capacity. * **Limits:** Used by the kubelet to enforce runtime constraints (e.g., OOM killing).

De-scheduling: Managing Flux

While the scheduler places Pods, it does not constantly re-balance them once they are running. **De-**

scheduling addresses the issue of "scheduler drift," where a cluster becomes sub-optimal due to node additions, hardware upgrades, or changing workloads.

The **Kubernetes Descheduler** is an optional component that performs "eviction-based" balancing. It identifies Pods that violate current constraints—such as: * **Low Node Utilization:** Moving workloads to consolidate them onto fewer nodes to save energy/costs. * **Affinity Violations:** Moving Pods that no longer satisfy updated anti-affinity rules. * **Duplicate Pods:** Ensuring that multiple replicas of the same service are not running on the same node.

Auto-scaling: Elasticity in Practice

Auto-scaling is the mechanism that ensures the cluster matches the demand of the incoming traffic. It occurs at two distinct layers:

1. Pod-level Scaling

- **Horizontal Pod Autoscaler (HPA):** Monitors metrics (CPU, Memory, or custom metrics) and increases/decreases the number of Pod replicas in a Deployment or StatefulSet.
- **Vertical Pod Autoscaler (VPA):** Automatically adjusts the resource `requests` and `limits` of containers based on historical usage. **Note: VPA and HPA should generally not be used together on the same metric (e.g., both on CPU).**

2. Cluster-level Scaling

- **Cluster Autoscaler (CA):** If the scheduler cannot find a node for a pending Pod due to resource exhaustion, the Cluster Autoscaler interacts with the Cloud Provider API to provision new Nodes.
- **Karpenter:** A more advanced, modern approach to auto-scaling that bypasses the traditional static node group model, provisioning just-in-time infrastructure tailored specifically to the pending Pod's requirements.

Summary Table: Comparison of Mechanisms

| Mechanism | Scope | Primary Objective | | :--- | :--- | :--- | | **Scheduler** | Pod | Initial placement and constraint satisfaction. | | **Descheduler** | Node/Pod | Post-placement optimization and drift correction. | | **HPA** | Pod | Scaling out (horizontal) to handle traffic bursts. | | **Cluster Autoscaler** | Infrastructure | Adding/Removing Nodes to meet Pod demands. |

Practical Considerations

- **Eventual Consistency:** Remember that scheduling is asynchronous. A Pod status may remain `Pending` for several seconds while the control plane orchestrates the binding.
- **Idempotency:** When writing custom controllers that interact with scheduling (e.g., custom schedulers or mutating webhooks), ensure your logic is idempotent; the scheduler may attempt to bind the same Pod multiple times under race conditions.
- **Resource Slicing:** For specialized workloads, look into **Dynamic Resource Allocation (DRA)** to manage complex hardware like GPUs or FPGAs more effectively than traditional static requests.

Managing resource Requests and Limits

Managing Resource Requests and Limits

In the Kubernetes ecosystem, the `kube-scheduler` makes decisions based on the available capacity of nodes and the requirements defined within a Pod specification. Managing resource requests and limits is the foundational mechanism for ensuring cluster stability, predictable performance, and efficient bin-packing of workloads.

Understanding Resource Requests and Limits

Every container within a Pod can define two primary attributes for CPU and memory: **Requests** and **Limits**.

Requests: The "Guaranteed" Floor

A **request** is the amount of resource that the `kube-scheduler` guarantees to a container. * **Scheduling:** The scheduler uses the sum of all requests in a Pod to determine if a node has enough remaining capacity to host that Pod. * **Allocation:** If a node does not have enough capacity to satisfy the sum of requests, the Pod will remain in a `Pending` state.

Limits: The "Hard" Ceiling

A **limit** is the maximum amount of a resource a container is allowed to consume. * **CPU:** CPU is a **compressible** resource. If a container hits its CPU limit, the kernel throttles the container's execution, but it will not be terminated. * **Memory:** Memory is an **incompressible** resource. If a container attempts to exceed its memory limit, the OOM (Out-of-Memory) Killer will terminate the container process.

Quality of Service (QoS) Classes

Kubernetes classifies Pods into three QoS classes based on their resource configuration. The QoS class determines the order in which Pods are evicted when a node runs out of resources (e.g., during memory pressure).

1. **Guaranteed:** All containers in the Pod have requests and limits set, and they are equal for each resource. These are the last to be evicted.
2. **Burstable:** The Pod has requests, but they do not match the limits, or some containers lack specific resource definitions.
3. **BestEffort:** No requests or limits are defined for any containers in the Pod. These are the first to be evicted.

Hands-on Lab: Configuring Resource Requirements

In this lab, you will define a deployment with explicit resource constraints.

Step 1: Create a Resource-Constrained Pod

Create a file named `resource-demo.yaml` with the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-demo
spec:
  containers:
  - name: nginx-container
    image: nginx
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```

Step 2: Deploy and Inspect

Apply the configuration to your cluster:

```
kubectl apply -f resource-demo.yaml
```

Verify the Pod's status and inspect the assigned QoS class:

```
kubectl get pod resource-demo -o jsonpath='{.status.qosClass}'
# Output should be: Burstable
```

Step 3: Observe Scheduling Behavior

If you attempt to request more resources than your cluster nodes have available, the scheduler will fail to place the Pod. You can verify this by checking the events:

```
kubectl describe pod resource-demo | grep -A 5 Events
```

Best Practices for Production

- **Always set requests:** Failing to set requests leads to poor scheduling decisions and potential over-subscription of nodes.
- **Size for reality:** Use monitoring tools (like Prometheus or Vertical Pod Autoscaler) to observe actual usage and tune your requests/limits accordingly.
- **Avoid over-committing:** While Burstable Pods are useful, ensure your nodes have enough "headroom" to handle spikes to prevent node-level instability.

When defining CPU resources, `1000m` is equivalent to `1` full vCPU. A request of `250m` ensures your application gets at least 1/4 of a core, while a limit of `500m` ensures it

> never consumes more than half a core.

Dynamic Resource Allocation

Error: Unable to get response from Gemini model after multiple attempts.

4.2 — Ubiquitous topics!

4.2 — Ubiquitous topics!

This module explores the diversity of Kubernetes control plane architectures and their deployment form factors, moving beyond standard cluster configurations. Key focus areas include:

- **Alternate Control Planes:** Understanding specialized control plane architectures designed for non-standard use cases, such as `kcp` for Kubernetes-like APIs on unconventional infrastructure.
- **HyperShift:** Exploring nested OpenShift deployments where control planes are decoupled from worker nodes to enable massive scalability and resource isolation.
- **Edge and Lightweight Form Factors:** Examining resource-constrained distributions like `MicroShift`, `k3s`, and `k0s` optimized for edge computing and small-footprint environments.

Alternative control plane architectures

Alternative Kubernetes Control Plane Architectures

While standard Kubernetes clusters follow a centralized architecture where the control plane components (`kube-apiserver`, `etcd`, `kube-controller-manager`, and `kube-scheduler`) reside together, modern operational requirements have pushed the boundaries of these patterns. Different "form factors" have emerged to solve specific challenges such as edge computing density, multi-tenancy, and rapid cluster provisioning.

The Evolution of Control Planes

In a standard deployment, the control plane is tightly coupled to the underlying infrastructure. However, for specialized use cases, we must decouple the management plane from the data plane. The following architectures represent the most prominent shifts in how Kubernetes control planes are deployed and consumed today.

1. HyperShift (Hosted Control Planes)

HyperShift revolutionizes cluster management by separating the control plane from the worker nodes. In this model, the control plane itself runs as a set of pods within a management cluster, rather than being statically provisioned on dedicated control plane nodes.

- **Key Advantage:** Rapid provisioning and reduced resource footprint. Since control planes are just containers, you can host hundreds of control planes on a single management cluster.
- **Use Case:** Ideal for multi-tenant environments and service providers offering Kubernetes-as-a-Service.

2. kcp: The Meta-Control Plane

`kcp` takes a different approach by focusing on a "Kubernetes-like" API surface that acts as a control plane for multiple physical clusters. It is essentially a framework to build high-performance, multi-cluster control planes.

- **Key Advantage:** It does not necessarily need to be a full Kubernetes cluster; it provides the API machinery (workspaces, logical clusters) to manage resources across disparate environments.
- **Use Case:** Building platforms that need to manage thousands of workspaces or clusters without the overhead of standard Kubernetes control plane components.

3. MicroShift and Small-Footprint Distributions

At the opposite end of the spectrum are distributions designed for the Edge, where resources are severely constrained.

- **MicroShift:** A stripped-down version of OpenShift designed to run on small form-factor devices. It integrates the control plane and data plane into a single binary, optimized for reliability in disconnected or low-bandwidth environments.
- **k3s / k0s / microk8s:** These distributions optimize the control plane by consolidating components (e.g., using SQLite instead of etcd) and reducing the memory overhead of the API server and controllers.

Comparison Matrix

Architecture	Primary Focus	Best For
HyperShift	Multi-tenancy & Speed	Managed Service Providers
kcp	Large-scale abstraction	Multi-cluster platforms
MicroShift	Edge/Resource constraint	Industrial IoT & Edge
Standard K8s	General purpose	Enterprise Data Centers

Hands-on Concept: Control Plane Isolation

When working with these architectures, the most important transition is moving from **static, node-bound components** to **pod-based control planes**.

Thinking Exercise: The Reconciliation Pattern in Nested Architectures

In a standard controller, the `reconcile` loop interacts with a single `kube-apiserver`. In an alternative architecture like HyperShift:

1. The **Management Cluster** contains the `HyperShift Operator`.
2. The Operator watches a `HostedCluster` CRD.
3. The Operator reconciles by deploying pods that contain the `kube-apiserver` and `etcd` for the **Guest Cluster**.

When designing controllers for these alternative architectures, ensure your `client-go` configurations are dynamic. Never hardcode the `RESTConfig` to `in-cluster` without accounting for the fact that your controller might be managing a remote "guest" control plane rather than the one it is physically running inside.

Summary

Choosing an alternative control plane architecture is a trade-off between **operational complexity** and **resource efficiency**. If you are building for massive scale, investigate `kcp` or `HyperShift`. If you are building for the edge, look into `MicroShift` or `k3s`. As a Kubernetes developer, understanding that the `kube-apiserver` is just a component that can be orchestrated—rather than a static service—is key to mastering advanced cluster management.

HyperShift and kcp

Kubernetes Control Plane Architectures: HyperShift and kcp

As Kubernetes matures, the requirement for different operational form factors has led to the emergence of alternative control plane architectures. Beyond the standard "all-in-one" cluster model, advanced use cases—such as massive-scale managed services or lightweight edge deployments—have necessitated the decoupling of control planes from the underlying infrastructure.

Two prominent examples of this evolution are **HyperShift** and **kcp**.

HyperShift: Hosted Control Planes

HyperShift (often referred to as Hosted Control Planes) changes how OpenShift clusters are deployed and managed. In a traditional OpenShift deployment, the control plane runs on the same infrastructure (nodes) as the worker nodes. HyperShift decouples these layers.

Architectural Approach

- **Control Plane Isolation:** In HyperShift, the control plane components (API server, etcd, controller manager) are run as a set of pods inside a "management" cluster.
- **Worker Isolation:** The worker nodes (the data plane) reside in a separate "hosted" cluster, which connects back to the managed control plane.
- **Benefits:**
- **Efficiency:** Multiple control planes can be scheduled onto a single management cluster, drastically reducing the resource overhead for running thousands of small clusters.
- **Security:** Control plane components are isolated from the worker nodes, minimizing the blast radius if a worker node is compromised.
- **Speed:** Because control planes are just collections of pods, they can be provisioned and updated in seconds rather than minutes.

kcp: The Kubernetes-like Control Plane

While HyperShift keeps the Kubernetes API as the primary interface for managing clusters, **kcp** takes a more radical approach. kcp is a "Kubernetes-like" control plane that provides a set of APIs for managing state at extreme scale, without necessarily implying a traditional "Kubernetes cluster" at the end of the line.

Why kcp?

kcp is designed for scenarios where you need the power of the Kubernetes API (workqueues, controllers, CRDs) but want to manage resources across thousands of locations or different types of infrastructure—not just standard Kubernetes clusters.

Core Concepts

- **Logical Clusters:** kcp introduces the concept of a `Logical Cluster`, which acts as a namespace-like boundary but at the control plane level. This allows for multi-tenancy that is isolated at the API server layer.
- **API Aggregation:** Unlike a standard kube-apiserver which is tightly coupled to etcd, kcp is designed to be highly modular. You can define APIs and have them synced to various locations (called "Sync Targets").
- **Separation of Concerns:** kcp focuses on the "what" (intent) and delegates the "where" (execution) to specialized controllers that move resources to the appropriate infrastructure providers.

Comparing the Architectures

| Feature | HyperShift | kcp | | :--- | :--- | :--- | | **Primary Goal** | High-density OpenShift management | Extreme scale API management | | **Interface** | Full Kubernetes/OpenShift API | Kubernetes-like, modular API | | **Use Case** | Managed Cloud/Edge Clusters | Global fleet/infrastructure management | | **Relationship** | Strictly Kubernetes-based | Kubernetes-inspired, extensible |

Key Takeaways for Developers

When designing for these architectures, remember the principle of **eventual consistency**. Because these control planes often operate across network boundaries (e.g., between the management cluster and the edge worker nodes), your controllers must be:

1. **Idempotent:** Ensure that repeating a reconciliation loop does not cause duplicate side effects.
2. **State-Aware:** Use `metadata.resourceVersion` to ensure you are operating on the latest version of the object, especially when dealing with distributed control planes where latency might lead to stale caches.
3. **Observant:** Utilize `status.conditions` to provide clear, actionable feedback to users, as the underlying infrastructure may not be immediately reachable.

For developers familiar with the standard `client-go` reconciliation pattern, moving to these architectures requires a shift in mindset: assume the network between your controller and the object's destination is unreliable. Focus heavily on robust error handling and backoff strategies in your `WorkQueue`.

Microshift and k3s form factors

Microshift and k3s Form Factors

In the ecosystem of Kubernetes, the standard control plane is designed for data centers and cloud-scale environments. However, the rise of Edge Computing, IoT, and restricted-resource environments has necessitated the development of specialized "form factors."

Microshift and k3s represent two prominent approaches to reducing the footprint of the Kubernetes control plane while maintaining compatibility with the core Kubernetes API.

MicroShift: Enterprise-Ready for the Edge

MicroShift is designed as an optimized, lightweight implementation of OpenShift. It focuses on providing a consistent developer and operational experience for resource-constrained edge devices.

Key Architectural Characteristics

- **Single-Node Focus:** Unlike traditional OpenShift, which assumes a multi-node cluster architecture, MicroShift is optimized for single-node deployments.
- **Containerized Control Plane:** It strips away the complexity of traditional OpenShift management components (like the Machine API or complex auto-scaling controllers) while retaining the core features needed for edge workloads.
- **OpenShift Compatibility:** It maintains compatibility with OpenShift APIs and security policies, allowing teams to use the same CI/CD pipelines and security tools across their data centers and edge locations.
- **Small Footprint:** By removing unnecessary abstractions and optimizing the binary size, MicroShift consumes significantly less CPU and memory, making it suitable for industrial PCs and small servers.

k3s: The Lightweight Kubernetes Distribution

Created by Rancher (now SUSE), k3s is a highly popular CNCF-certified distribution built for environments where resources are at a premium.

Key Architectural Characteristics

- **Binary Consolidation:** k3s packages the Kubernetes control plane components (API server, scheduler, controller manager) into a single, small binary (typically under 100MB).
- **Embedded Databases:** While standard Kubernetes relies on `etcd`, k3s allows you to use SQLite as a lightweight storage backend for single-node setups, though it supports external databases like etcd, MySQL, or PostgreSQL for high-availability clusters.
- **Removed Legacy Code:** k3s excludes legacy drivers, cloud-specific provider code, and alpha features that are rarely used in edge scenarios, significantly reducing the "noise" and attack surface of the control plane.
- **Simplicity:** With a focus on ease of installation, k3s is often deployed with a single command, making it ideal for developers and automated provisioning at scale.

Comparative Summary

| Feature | MicroShift | k3s | | :--- | :--- | :--- | | **Primary Goal** | OpenShift experience at the Edge | Maximum minimalism and portability | | **Control Plane** | OpenShift-based | Standard Kubernetes-based | | **Database** | Integrated/Simplified | SQLite (default), etcd (optional) | | **Target Use Case** | Enterprise Edge / Industrial | General-purpose IoT / CI-CD / Edge |

Why Form Factors Matter

When designing distributed systems for the edge, your choice of control plane architecture directly impacts your operations:

1. **Resource Efficiency:** In restricted environments, every megabyte of RAM allocated to the `kube-apiserver` or `etcd` is memory taken away from your application workload.
2. **Consistency:** Both MicroShift and k3s aim to provide a "Kubernetes-conformant" API. This ensures that the declarative intent of your resources (Deployment, ConfigMap, etc.) remains identical regardless of where the cluster is running.
3. **Lifecycle Management:** These form factors simplify the upgrade path. Instead of managing complex control plane upgrades, these distributions often provide a streamlined, binary-based upgrade process suitable for remote sites with limited connectivity.

When choosing between these form factors, consider your existing toolchain. If your organization is already invested in the OpenShift ecosystem, **MicroShift** provides the lowest friction path to parity. If you require a generic Kubernetes distribution for diverse, multi-cloud or hardware-agnostic edge workloads, **k3s** offers the most flexibility.

Exercises

These hands-on labs reinforce the concepts covered in the weekly modules. Each exercise builds a small but complete Go program that you can run against a local Kubernetes cluster.

- Exercise 1: Build a Controller — Wire up the client-go pipeline from scratch
- Exercise 2: Leader Election — Add leader election to your controller
- Exercise 3: Validating Webhook — Build and deploy a validating admission webhook # Exercises

Exercises

The practical curriculum focuses on reinforcing Kubernetes controller development and extensibility through the following hands-on labs:

- **Controller Fundamentals:** Build a functional controller using `client-go` from scratch to understand event propagation and synchronization primitives.
- **Distributed Coordination:** Implement optimistic locking and leader election mechanisms using the `Lease` API to ensure state consistency.
- **Admission Control:** Develop a custom Validating Webhook to enforce organizational security policies, specifically restricting container image sources to approved registries.

Exercise 1: Build a Controller

In this exercise you will build a Kubernetes controller from scratch using client-go. By the end you will have a working program that watches for Pod events, processes them through a workqueue, and logs their state — implementing the full data flow pipeline described in client-go Architecture.

Prerequisites

- Go 1.22+ installed
- Access to a Kubernetes cluster (Kind, Minikube, or OpenShift Local)
- `kubeconfig` configured to point to your cluster

Part 1: Project Setup

Create your project directory and pull in the required dependencies:

```
mkdir k8s-controller-lab && cd k8s-controller-lab
go mod init k8s-controller-lab
go get k8s.io/client-go@latest
go get k8s.io/apimachinery@latest
```

Part 2: Connect to the Cluster

Use the standard `kubeconfig` lookup to build a REST config and create a typed clientset:

```
package main

import (
    "flag"
    "fmt"
    "path/filepath"
    "time"

    "k8s.io/client-go/kubernetes"
    "k8s.io/client-go/tools/clientcmd"
    "k8s.io/client-go/util/homedir"
)

func main() {
    var kubeconfig *string
    if home := homedir.HomeDir(); home != "" {
        kubeconfig = flag.String("kubeconfig",
            filepath.Join(home, ".kube", "config"), "path to kubeconfig")
    }
    flag.Parse()

    config, err := clientcmd.BuildConfigFromFlags("", *kubeconfig)
    if err != nil {
        panic(err)
    }
    clientset, err := kubernetes.NewForConfig(config)
    if err != nil {
        panic(err)
    }
    // ... controller setup continues below
}
```

Part 3: Set Up the Informer

Create a `SharedInformerFactory` and register event handlers that enqueue object keys. Review Informers, Listers, and Workqueues for why we enqueue keys rather than processing directly in the handler.

```
factory := informers.NewSharedInformerFactory(clientset, 10*time.Minute)
podInformer := factory.Core().V1().Pods().Informer()

queue := workqueue.NewNamedRateLimitingQueue(
    workqueue.DefaultControllerRateLimiter(), "pods")

podInformer.AddEventHandler(cache.ResourceEventHandlerFuncs{
```

```
    AddFunc: func(obj interface{}) {
        key, err := cache.MetaNamespaceKeyFunc(obj)
        if err == nil {
            queue.Add(key)
        }
    },
    UpdateFunc: func(old, new interface{}) {
        key, err := cache.MetaNamespaceKeyFunc(new)
        if err == nil {
            queue.Add(key)
        }
    },
    DeleteFunc: func(obj interface{}) {
        key, err := cache.DeletionHandlingMetaNamespaceKeyFunc(obj)
        if err == nil {
            queue.Add(key)
        }
    },
})

stopCh := make(chan struct{})
defer close(stopCh)
factory.Start(stopCh)
cache.WaitForCacheSync(stopCh, podInformer.HasSynced)
```

Always call `cache.WaitForCacheSync` before processing items. Starting your reconcile loop before the cache is warm leads to missed resources.

Part 4: The Worker Loop

Implement a worker that dequeues keys and processes them. On failure, use `queue.AddRateLimited(key)` to requeue with exponential backoff. On success, call `queue.Forget(key)` to reset the backoff counter.

```
func processNextItem(queue workqueue.RateLimitingInterface,
    lister corelisters.PodLister) bool {
    key, shutdown := queue.Get()
    if shutdown {
        return false
    }
    defer queue.Done(key)

    namespace, name, err := cache.SplitMetaNamespaceKey(key.(string))
    if err != nil {
        queue.Forget(key)
        return true
    }

    pod, err := lister.Pods(namespace).Get(name)
```

```
    if err != nil {
        queue.AddRateLimited(key)
        return true
    }

    fmt.Printf("Reconciling Pod %s/%s (phase: %s)\n",
        pod.Namespace, pod.Name, pod.Status.Phase)
    queue.Forget(key)
    return true
}
```

Run the worker in a goroutine and block until `stopCh` closes.

Challenge

Extend your reconcile logic: if the Pod has no labels, log a warning `"Pod {namespace}/{name} is missing required labels!"`. Then try running two instances of your controller simultaneously and observe what happens — this motivates the leader election exercise in Exercise 2.

References

- Operator SDK Go Tutorial
- Controller Rough Structure

Exercise 2: Leader Election

In this exercise you will add leader election to your controller so that only one replica is active at a time. This builds on the concepts from Leader Election in Kubernetes and Distributed Locking Theory.

Prerequisites

- The working controller from Exercise 1
- `k8s.io/client-go/tools/leaderelection` and `k8s.io/client-go/tools/leaderelection/resourcelock` imported

Step 1: Define a Unique Identity

Each controller replica needs a unique identity. In a real Deployment, use the Pod name from the downward API. For local testing, the process ID works:

```
import (
    "fmt"
    "os"

    rl "k8s.io/client-go/tools/leaderelection/resourcelock"
)
```

```
identity := fmt.Sprintf("%d", os.Getpid())
```

Step 2: Create the Lease Lock

Define a `LeaseLock` that specifies which Lease object to use for coordination:

```
lock := &rl.LeaseLock{
    LeaseMeta: metav1.ObjectMeta{
        Name:      "my-controller-lock",
        Namespace: "default",
    },
    Client: clientset.CoordinationV1(),
    LockConfig: rl.ResourceLockConfig{
        Identity: identity,
    },
}
```

Step 3: Wire Up the Election Loop

The `leaderelection` package provides a callback-based framework. Your controller's `Run` method goes inside `OnStartedLeading`:

```
leaderelection.RunOrDie(ctx, leaderelection.LeaderElectionConfig{
    Lock:            lock,
    ReleaseOnCancel: true,
    LeaseDuration:   15 * time.Second,
    RenewDeadline:   10 * time.Second,
    RetryPeriod:     2 * time.Second,
    Callbacks: leaderelection.LeaderCallbacks{
        OnStartedLeading: func(ctx context.Context) {
            runController(ctx) // your controller logic
        },
        OnStoppedLeading: func() {
            log.Println("Lost leadership, shutting down")
            os.Exit(0)
        },
        OnNewLeader: func(id string) {
            log.Printf("Current leader: %s", id)
        },
    },
})
```

The timing parameters matter. `LeaseDuration` must be greater than `RenewDeadline`, which must be greater than `RetryPeriod`. Setting `LeaseDuration` too short causes flapping during network jitter; too long delays failover.

Step 4: Test Failover

1. Run two instances of your controller in separate terminals.
2. Observe that only one prints `"I am the leader!"` and begins reconciling.
3. Kill the leader process and wait approximately 15 seconds.
4. The standby instance should acquire the Lease and start reconciling.

Verify the Lease state at any time:

```
kubectl get lease my-controller-lock -o yaml
```

The `holderIdentity` field shows which instance currently holds the lock. The `leaseTransitions` counter increments with each leadership change.

Step 5: Required RBAC

Your controller's ServiceAccount needs permission to operate on the Lease:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: leader-election
  namespace: default
rules:
- apiGroups: ["coordination.k8s.io"]
  resources: ["leases"]
  verbs: ["get", "create", "update"]
```

A Note on Fencing

The `leaderelection` package does not guarantee single-leader semantics under all conditions. If a leader pauses long enough for its Lease to expire, a new leader is elected — but the old leader may resume and briefly believe it is still leading. Kubernetes mitigates this through optimistic concurrency: updates with a stale `resourceVersion` are rejected. For a deeper discussion, see Distributed Locking Theory.

Exercise 3: Validating Webhook

In this exercise you will build and deploy a validating admission webhook that rejects Pods referencing unauthorized container registries. This applies the concepts from Admission Control Webhooks.

Prerequisites

- A running Kubernetes cluster (Kind or Minikube)

- cert-manager installed (for webhook TLS certificates)
- Go 1.22+ installed

Step 1: Implement the Webhook Handler

Create an HTTP server that receives an `AdmissionReview` request, inspects the Pod's container images, and returns an allow or deny response:

```
func handleValidation(w http.ResponseWriter, r *http.Request) {
    var review admissionv1.AdmissionReview
    if err := json.NewDecoder(r.Body).Decode(&review); err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }

    var pod corev1.Pod
    json.Unmarshal(review.Request.Object.Raw, &pod)

    allowed := true
    var message string
    for _, c := range pod.Spec.Containers {
        if !isImageAllowed(c.Image) {
            allowed = false
            message = fmt.Sprintf("image %q is not from an approved registry",
c.Image)
            break
        }
    }

    review.Response = &admissionv1.AdmissionResponse{
        UID:     review.Request.UID,
        Allowed: allowed,
        Result:  &metav1.Status{Message: message},
    }
    json.NewEncoder(w).Encode(review)
}
```

Step 2: Registry Allow-List

Accept a list of approved registry prefixes via a command-line flag:

```
var allowedRegistries []string

func init() {
    flag.Func("allowed-registries",
        "Comma-separated list of approved registries", func(s string) error {
        allowedRegistries = strings.Split(s, ",")
        return nil
```

```
    })
}

func isImageAllowed(image string) bool {
    for _, prefix := range allowedRegistries {
        if strings.HasPrefix(image, prefix) {
            return true
        }
    }
    return false
}
```

Step 3: Register the Webhook

Create a `ValidatingWebhookConfiguration` that tells the API server to send Pod CREATE and UPDATE requests to your webhook:

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: registry-validator
webhooks:
- name: validate.registry.example.com
  rules:
  - operations: ["CREATE", "UPDATE"]
    apiGroups: [""]
    apiVersions: ["v1"]
    resources: ["pods"]
  clientConfig:
    service:
      name: webhook-service
      namespace: default
      path: "/validate"
  admissionReviewVersions: ["v1"]
  sideEffects: None
  failurePolicy: Fail
```

Set `failurePolicy: Fail` in production so that Pods are blocked when the webhook is unreachable. Using `Ignore` silently skips validation, which defeats the purpose of the policy.

Step 4: Deploy and Test

1. Build a container image for your webhook and push it to your cluster's registry.
2. Create a `Deployment` and `Service` for the webhook in the `default` namespace.
3. Ensure cert-manager injects TLS certificates (the API server requires HTTPS to reach webhooks).

4. Apply the `ValidatingWebhookConfiguration`.

Test with an unauthorized image:

```
kubectl run test --image=docker.io/library/nginx:latest
# Expected: Error from server: admission webhook
#   "validate.registry.example.com" denied the request:
#   image "docker.io/library/nginx:latest" is not from an approved registry
```

Test with an authorized image (assuming `quay.io` is in your allow-list):

```
kubectl run test --image=quay.io/myorg/myapp:latest
# Expected: pod/test created
```

Tips

- Use `kubectl run --dry-run=server` during development to test webhook responses without creating actual Pods.
- Check webhook logs if requests are being denied unexpectedly — the `AdmissionReview` request object contains the full Pod spec.
- For a declarative alternative to hand-coded webhooks, see Policy Engines and Gatekeeper.

Exercises

1: Build a controller using client-go from scratch

Exercise: Building a Kubernetes Controller from Scratch

Building a controller from scratch using `client-go` is the best way to understand the "magic" that hides behind higher-level abstractions like `controller-runtime` or `kubebuilder`.

In this exercise, you will create a controller that watches for `ConfigMaps` and logs a message whenever one is added, updated, or deleted.

Prerequisites

- A running Kubernetes cluster (e.g., `kind` or `minikube`).
- `go` (version 1.21+).
- `kubectl` configured to point to your cluster.
- `client-go` dependency:

```
go mod init my-controller
go get k8s.io/client-go@latest
go get k8s.io/api@latest
go get k8s.io/apimachinery@latest
```

Conceptual Architecture

To build a controller, you must implement the "Control Loop" pattern using these primitives: * **Informers:** The "Eyes." They list and watch resources, pushing events to a local cache. * **Listers:** The "Cache." They provide read-only access to the local cache, preventing excessive API server load. * **WorkQueue:** The "Buffer." It decouples the informer's event notification from the processing logic, allowing for retries and rate-limiting.

Hands-on: Implementation

1. Initialize the Controller

Create a file named `main.go`. We will use `SharedInformerFactory` to keep track of the ConfigMap resources.

```
package main

import (
    "fmt"
    "time"
```

```
    "k8s.io/client-go/informers"
    "k8s.io/client-go/kubernetes"
    "k8s.io/client-go/tools/cache"
    "k8s.io/client-go/tools/clientcmd"
    "k8s.io/client-go/util/workqueue"
)

func main() {
    // 1. Build kubeconfig and clientset
    kubeconfig :=
clientcmd.NewNonInteractiveDeferredLoadingClientConfig(clientcmd.NewDefaultClientConfi
gLoadingRules(), &clientcmd.ConfigOverrides{}).ClientConfig()
    clientset := kubernetes.NewForConfigOrDie(kubeconfig)

    // 2. Initialize Informer Factory and WorkQueue
    factory := informers.NewSharedInformerFactory(clientset, time.Minute*10)
    informer := factory.Core().V1().ConfigMaps().Informer()
    queue :=
workqueue.NewNamedRateLimitingQueue(workqueue.DefaultControllerRateLimiter(),
"ConfigMapQueue")

    // 3. Register Event Handlers
    informer.AddEventHandler(cache.ResourceEventHandlerFuncs{
        AddFunc: func(obj interface{}) {
            key, _ := cache.MetaNamespaceKeyFunc(obj)
            queue.Add(key)
        },
        UpdateFunc: func(old, new interface{}) {
            key, _ := cache.MetaNamespaceKeyFunc(new)
            queue.Add(key)
        },
    })

    // ... continue to process loop
}
```

2. Implement the Reconcile Loop

Now, define the loop that consumes items from the queue.

```
func runWorker(queue workqueue.RateLimitingInterface, lister cache.GenericLister) {
    for {
        key, quit := queue.Get()
        if quit { return }

        namespace, name, _ := cache.SplitMetaNamespaceKey(key.(string))
        fmt.Printf("Reconciling ConfigMap: %s/%s\n", namespace, name)

        // Logic: Perform your desired state enforcement here
```

```
        queue.Done(key)
    }
}
```

3. Execution

To run this: 1. Start the `Informer` by calling `informer.Run(stopCh)`. 2. Wait for the local cache to synchronize (`cache.WaitForCacheSync`). 3. Run the `runWorker` function in a goroutine.

Troubleshooting Tips

- **Cache Sync:** Always ensure you call `cache.WaitForCacheSync` before starting the worker. If the cache isn't ready, your listers will return empty results.
- **Namespace Scoping:** If your controller is behaving unexpectedly, verify if you are using `NewSharedInformerFactory` (cluster-wide) or `NewFilteredSharedInformerFactory` (namespace-scoped).
- **Key Parsing:** Use `cache.MetaNamespaceKeyFunc` consistently to ensure that the keys in your workqueue can be correctly parsed back into namespace and name.

Why do this?

By writing this manually, you observe how `client-go` handles the complex orchestration of API watches. In the next stage, you will see how `controller-runtime` wraps this exact logic into a clean `Reconcile` method, drastically reducing the boilerplate while maintaining the same underlying mechanics.

2: Implement optimistic locking and leader election via Lease

2.2 Implementing Optimistic Locking and Leader Election via Lease

When scaling controllers, running multiple replicas of the same controller is a common requirement for high availability. However, this creates a "split-brain" scenario where multiple controllers attempt to reconcile the same resources simultaneously, leading to race conditions and inconsistent states.

In Kubernetes, we solve this by implementing **Leader Election** using the `Lease` API.

Understanding Distributed Locks in Kubernetes

A **Lease** is a Kubernetes resource that provides a distributed lock mechanism. It allows a set of controller replicas to compete for the "leadership" role.

- **Atomic Updates:** Using the `client-go/tools/leaderelection` package, controllers perform atomic updates to the Lease object.

- **Heartbeat Mechanism:** The leader periodically updates the `renewTime` of the Lease. If the leader crashes, the `renewTime` stops updating, and the Lease expires, allowing a follower to acquire leadership.
- **Optimistic Locking:** Kubernetes uses the `ResourceVersion` field in the Lease object to ensure that only one controller successfully updates the lease at any given time (Optimistic Concurrency Control).

Hands-on Lab: Implementing Leader Election

In this lab, we will extend the `client-go` controller created in the previous module to support high-availability through leader election.

Prerequisites

- A working `client-go` controller loop.
- `k8s.io/client-go/tools/leaderelection` and `k8s.io/client-go/tools/leaderelection/resourcelock` packages.

Step 1: Define the Lock Resource

First, we define the `Lease` object that will serve as the lock. This is typically done using the `LeaseLock` type within the `resourcelock` package.

```
import (
    "k8s.io/client-go/tools/leaderelection"
    "k8s.io/client-go/tools/leaderelection/resourcelock"
)

// Define the lease lock
lock := &resourcelock.LeaseLock{
    LeaseMeta: metav1.ObjectMeta{
        Name:      "my-controller-lock",
        Namespace: "kube-system",
    },
    Client: clientSet.CoordinationV1(),
    LockConfig: resourcelock.ResourceLockConfig{
        Identity: podName, // Unique identifier for this instance
    },
}
```

Step 2: Configure the Leader Election Loop

Initialize the `leaderelection` configuration. This is where you define the `Callbacks` that trigger when your controller becomes the leader or loses its status.

```
leaderelection.RunOrDie(ctx, leaderelection.LeaderElectionConfig{
```

```
    Lock:            lock,
    ReleaseOnCancel: true,
    LeaseDuration:   15 * time.Second,
    RenewDeadline:   10 * time.Second,
    RetryPeriod:     2 * time.Second,
    Callbacks: leaderelection.LeaderCallbacks{
        OnStartedLeading: func(ctx context.Context) {
            // Start your controller's Run() loop here
            controller.Run(ctx)
        },
        OnStoppedLeading: func() {
            log.Printf("Leader lost, shutting down.")
            os.Exit(0)
        },
        OnNewLeader: func(identity string) {
            log.Printf("New leader elected: %s", identity)
        },
    },
})
```

Step 3: Verifying the Setup

1. **Deploy Multiple Replicas:** Scale your controller deployment to 3 replicas.
2. **Monitor the Lease:** Observe the `Lease` object in the `kube-system` namespace: `kubectl get lease my-controller-lock -o yaml`
3. **Simulate Failure:** Delete the pod currently holding the lease (identified in the `holderIdentity` field). Observe the logs of the remaining pods to see the re-election process occur.

Best Practices

- **Idempotency:** Ensure your `Reconcile` function is truly idempotent. Even with leader election, there may be brief windows during transition where an old leader is still finishing a task.
- **TTL Tuning:** The `LeaseDuration` should be long enough to handle transient network blips but short enough to allow for rapid failover. A common default is 15 seconds.
- **Sharding:** If you have a massive number of resources, rather than just leader election, consider **Controller Sharding**, where you split the workload (e.g., based on namespace labels) so each controller instance is responsible for a subset of the resources.

3: Develop a custom Validating Webhook for container registry enforcement

Exercise 3: Developing a Custom Validating Webhook

In this exercise, you will extend your Kubernetes knowledge by implementing a **Validating Admission Webhook**. This component acts as a gatekeeper, intercepting requests to the `kube-apiserver` to ensure that any container images specified in your custom resources originate only

from pre-approved registries.

Technical Overview

Kubernetes Admission Controllers are plugins that govern and enforce policies on objects during create, update, or delete operations. A **Validating Webhook** is an HTTP callback that the API server invokes to determine whether a request should be allowed or rejected based on custom logic.

Unlike **Policy Engines** (such as OPA Gatekeeper or Kyverno) which provide a declarative DSL for policy management, a custom webhook allows you to implement complex, imperative business logic directly in Go, providing complete control over the validation lifecycle.

Prerequisites

- A running Kubernetes cluster (e.g., Minikube, Kind, or OpenShift).
- A working custom resource already reconciled by your controller.
- `cert-manager` installed in your cluster (for managing webhook TLS certificates).

Hands-on Lab: Registry Enforcement Webhook

In this lab, you will write a Go-based binary that validates the container image string against an allow-list provided via command-line flags.

Step 1: Define the Webhook Logic

Your binary needs to implement an HTTP handler that accepts an `admission.k8s.io/v1, Kind=AdmissionReview` request and returns a response indicating success or failure.

```
func validateRegistry(image string, approvedRegistries []string) bool {
    for _, registry := range approvedRegistries {
        if strings.HasPrefix(image, registry) {
            return true
        }
    }
    return false
}
```

Step 2: Configure Command-Line Flags

The webhook binary must accept the registry list at startup. Use the standard `flag` package to parse these inputs.

```
var approvedRegistries []string

func init() {
    flag.Func("approved-registries", "Comma-separated list of allowed registries",
func(s string) error {
```

```
        approvedRegistries = strings.Split(s, ",")
        return nil
    })
}
```

Step 3: Define the ValidatingWebhookConfiguration

You must register your webhook with the API server so it knows which resources to monitor and how to reach your service.

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: registry-validator
webhooks:
  - name: validate.registry.example.com
    rules:
      - apiGroups: ["your-api-group"]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources: ["your-custom-resources"]
    clientConfig:
      service:
        name: registry-validator-service
        namespace: default
        path: "/validate"
    admissionReviewVersions: ["v1"]
    sideEffects: None
```

Step 4: Deployment and Verification

1. **Build and Push**: Compile your Go binary, containerize it (including the flag setup), and push it to your registry.
2. **Deploy**: Create the `Deployment` and `Service` for the webhook. Ensure your `ValidatingWebhookConfiguration` points to this service.
3. **Test**: Attempt to create a custom resource using an image from a non-approved registry:

```
# This should be rejected by the API server
kubectl apply -f- <<EOF
apiVersion: your-api-group/v1
kind: YourCustomResource
metadata:
  name: test-resource
spec:
  image: "malicious-registry.com/my-app:latest"
EOF
```

```
# Expected output:
# Error from server: admission webhook "validate.registry.example.com" denied the
request
```

Troubleshooting Guidance

- **TLS Issues**: Webhooks require HTTPS. Ensure your `ValidatingWebhookConfiguration` includes the `caBundle` field, which should contain the CA certificate used to sign your webhook's serving certificate. `cert-manager` can automate this injection.
- **Network Connectivity**: If the `kube-apiserver` cannot reach your webhook, check your Service selector and ensure the `ValidatingWebhookConfiguration` namespace matches the target service.
- **Infinite Loops**: Avoid having your webhook trigger itself or perform operations that cause secondary resource changes that could trigger the webhook again (set `sideEffects: None`).

Recommended Reading

The following resources are strongly recommended as companions to this course. Each book or paper aligns with specific weekly modules and will deepen your understanding of the topics covered.

Programming Kubernetes

Michael Hausenblas and Stefan Schimanski (O'Reilly, 2019)

This book is the definitive guide to client-go internals, custom resources, and the API machinery that powers Kubernetes extensibility. It serves as an excellent companion to Week 1.1 and Week 1.2, where you will explore the Kubernetes resource model and controller foundations. Readers will gain a thorough understanding of how the API server processes requests and how to build robust clients and controllers.

Kubernetes Operators

Jason Dobies and Joshua Wood (O'Reilly, 2020)

This book covers the operator pattern end-to-end, from the Operator SDK to lifecycle management of complex applications on Kubernetes. It pairs well with Week 2.1 and Week 3.1, where you will write controllers with controller-runtime and extend the API server. It provides practical guidance on packaging, deploying, and upgrading operators in production.

CNCF Operator White Paper

CNCF TAG App Delivery — Operator White Paper

This industry reference defines the operator capability model and provides a comprehensive comparison of operator frameworks. It is the ideal companion to Week 4.2, where you will survey the operator landscape and alternative control plane architectures. The white paper offers a vendor-neutral perspective on when and how to build operators. # Recommended Reading

Recommended Reading

To deepen your understanding of Kubernetes development and distributed systems, the following resources are recommended:

- **Core Kubernetes Development:**
  - **Programming Kubernetes** by Michael Hausenblas and Stefan Schimanski: Covers the fundamentals of extending Kubernetes.
  - **Kubernetes Operators** by Jason Dobies and Joshua Wood: Essential for understanding automation and controller design.
  - **CNCF Operator Whitepaper**: Provides standardized guidance on the operator pattern.

- **Security and System Architecture:**
  - **Kubernetes Security and Observability** by Brendan Creane and Amit Gupta: Best practices for securing and monitoring clusters.
  - **Designing Distributed Systems** by Brendan Burns: Explores reusable patterns for building scalable, reliable applications on Kubernetes.

Recommended Reading

Programming Kubernetes (Hausenblas, Schimanski)

Recommended Reading: Programming Kubernetes

In the journey toward becoming a Kubernetes developer, understanding the "how" and "why" of the control plane is essential. Michael Hausenblas and Stefan Schimanski's **Programming Kubernetes: Developing Cloud-Native Applications** serves as the definitive foundational text for mastering the Kubernetes API and extending the platform.

Why this book is essential

While many resources focus on using `kubectl` to manage workloads, this text shifts the perspective to that of a **platform engineer**. It deconstructs the Kubernetes control plane, providing the technical depth required to transition from a consumer of Kubernetes resources to an architect of Kubernetes extensions.

Key insights provided by the authors include:

- **API-Centric Architecture:** A deep dive into how the `kube-apiserver` acts as the single source of truth and the gateway for all declarative configuration.
- **The Controller Pattern:** A clear explanation of the asynchronous reconciliation loop, which is the mechanism that maintains the desired state of the cluster.
- **Deep Dive into `client-go`:** Essential guidance on using the official Go client library to interact with the Kubernetes API, manage informers, and handle event propagation.
- **Custom Resource Definitions (CRDs):** Practical strategies for extending the Kubernetes API to manage domain-specific resources.

Key Concepts Covered

For developers following this learning path, the book aligns perfectly with our core objectives:

Topic	Relevance to Learning Path
The Control Plane	Provides the theoretical backing for our study of `etcd`, API validation, and the reconciliation pattern.
Custom Controllers	Mirrors our focus on building operators that utilize `client-go`, workqueues, and cache mechanisms.
Distributed Systems	Offers guidance on implementing the eventual consistency model and handling concurrency—crucial for high-availability operators.

How to use this resource in this course

We recommend using **Programming Kubernetes** as your primary reference manual during the hands-on labs. Specifically, as you begin building your first controller from scratch, refer to the

chapters covering:

1. **Workqueues and Informers:** Use these sections to understand the flow of events from the API server to your custom controller logic.
2. **API Machinery:** Use this as a reference when defining your Custom Resource Definition (CRD) schemas and managing `Spec` vs. `Status` fields.
3. **Optimistic Concurrency:** When we tackle the lesson on "Going distributed: locks, leases, and ownership," the book's coverage of resource versioning and conflict handling will be invaluable for implementing robust leader election.

Further Study

This book is not a "read once" resource; it is an architectural handbook. As you progress from basic controllers to advanced admission webhooks and custom API servers, revisit the technical implementation details documented in the text to ensure your custom infrastructure components adhere to the standard Kubernetes "look and feel."

For those currently navigating the transition from `kubectl` automation to binary-based controller development, focus heavily on the sections describing the `client-go` event-driven architecture. This is the cornerstone of the hands-on labs you will complete in this track.

Kubernetes Operators (Dobies, Wood)

Kubernetes Operators: Automating the Orchestration Platform

Understanding the operational paradigm of Kubernetes requires moving beyond simple deployments toward the concept of **Operators**. As defined by Jason Dobies and Joshua Wood in **Kubernetes Operators: Automating the Container Orchestration Platform**, an Operator is essentially a software extension to Kubernetes that makes use of custom resources to manage applications and their components.

The Operator Pattern

At its core, the Operator pattern extends the Kubernetes control plane's capabilities to manage complex, stateful applications—such as databases, monitoring stacks, or message brokers—that would otherwise require manual intervention.

The pattern relies on the **Reconciliation Loop**, where the controller: 1. **Observes:** Watches the API server for changes to the Custom Resource (CR). 2. **Analyzes:** Compares the "Desired State" (the Spec defined in the CR) with the "Observed State" (the current status of the cluster). 3. **Acts:** Executes the necessary CRUD operations (Create, Update, Patch, Delete) to align the observed state with the desired state.

Key Takeaways from Dobies & Wood

The text provides a architectural framework for moving from a standard controller to a fully functional Operator:

- **Custom Resources (CRs):** Operators leverage Custom Resource Definitions (CRDs) to expose an API that represents your domain-specific configuration.
- **The Reconcile Logic:** Unlike a standard loop, an Operator must be idempotent. If a process is interrupted or a component is partially deployed, the next iteration of the loop should safely resume or correct the state without causing side effects.
- **Spec vs. Status:** A critical distinction in the Operator lifecycle. The `Spec` represents the configuration requested by the user, while the `Status` represents the current reality of the application. Effective Operators report rich, actionable data in the `Status` field so users can monitor progress.

Comparison: Kubebuilder vs. Operator SDK

When beginning your development journey, choosing the right toolset is vital. While **Kubernetes Operators** focuses on the conceptual underpinnings, practical implementation often leans on the **Operator SDK** or **Kubebuilder**.

| Feature | Kubebuilder | Operator SDK | | :--- | :--- | :--- | | **Philosophy** | "Go-native," focuses on simplicity and standard `controller-runtime` patterns. | Built on top of Kubebuilder; provides additional "batteries-included" features. | | **Capability Levels** | Primarily Go-based. | Supports Go, Helm, and Ansible-based operators. | | **Tooling** | Minimalist; standard scaffold. | Includes features like `operator-scorecard` and enhanced lifecycle management. |

Recommended Practical Workflow

To successfully implement the concepts discussed by Dobies and Wood, align your development process with these industry standards:

1. **Define your API:** Before writing logic, follow the Kubernetes API Conventions.
2. **Lint your API:** Use `kube-api-linter` integrated with `golangci-lint` to catch common mistakes in your CRD definitions (e.g., missing validation or incorrect field types).
3. **Choose your Abstraction:** If you are building high-performance, complex systems, prioritize `controller-runtime` (via Kubebuilder) to maintain fine-grained control over the reconcile loop.
4. **Manage State:** Always prefer `PATCH` or `UPDATE` semantics that account for optimistic concurrency. Use standard Kubernetes `ResourceVersion` checks to prevent race conditions during updates.

Further Learning

For deep dives into the topics covered, refer to these resources: * **Kubernetes Operators** by Dobies and Wood (O'Reilly). * CNCF Operator Whitepaper: Provides a comprehensive view of how Operators fit into the broader cloud-native ecosystem. * The Kubebuilder Book: Essential documentation for mastering the reconciliation pattern in Go.

CNCF Operator Whitepaper

CNCF Operator Whitepaper

The CNCF Operator Whitepaper serves as the foundational architectural guide for developers

looking to extend Kubernetes through automation. As you transition from managing individual resources to building autonomous systems, understanding the principles defined in this document is essential for creating resilient, production-grade Operators.

The Essence of an Operator

An Operator is a software extension to Kubernetes that makes use of custom resources to manage applications and their components. Following the CNCF definitions, an Operator implements the **Control Loop**—a fundamental pattern in distributed systems that ensures the current state of a system matches the desired state defined by the user.

Key Components of an Operator

According to the CNCF whitepaper, an effective Operator relies on three primary pillars:

1. **Custom Resource Definitions (CRDs):** These allow you to define your own API objects, extending the Kubernetes API to understand the domain-specific logic of your application (e.g., a `Database` or `Queue` resource).
2. **Controller Logic:** The brain of the operation. It continuously watches the cluster state, compares it against the declared intent, and performs actions (CRUD operations) to reconcile discrepancies.
3. **Human Operational Knowledge:** An Operator is only as good as the operational experience it encodes. This includes lifecycle management tasks like installation, upgrades, backups, and recovery.

The Operator Maturity Model

A core contribution of the CNCF Operator whitepaper is the **Operator Maturity Model**. When designing your controller, it is helpful to categorize your goals into these levels:

- **Phase I: Basic Install:** The Operator automates the deployment of the application (e.g., creating Deployments, Services, and ConfigMaps).
- **Phase II: Seamless Upgrades:** The Operator handles application version updates and patch management.
- **Phase III: Full Lifecycle:** The Operator manages storage, backups, and scaling (e.g., handling persistent volume snapshots).
- **Phase IV: Deep Insights:** The Operator provides observability metrics, alerting, and log analysis.
- **Phase V: Auto-pilot:** The Operator performs automated tuning, horizontal scaling, and self-healing without human intervention.

Architectural Best Practices

When building your controllers using `client-go` or `controller-runtime`, refer to these whitepaper-recommended practices:

- **Idempotency:** Every action performed by the reconciliation loop must be idempotent. If an error occurs, the controller will requeue the work item; your code must be able to handle

"seeing" the same state multiple times without causing side effects.

- **Decoupled Logic:** Separate your API definition (the "Spec") from the operational implementation. This allows you to evolve your API surface independently of your controller's internal reconciliation logic.
- **Observability:** Operators act as "privileged" actors in the cluster. Always expose metrics (via Prometheus) so that users can monitor the health and progress of your control loop.

Further Reading

To deepen your understanding of these patterns, the CNCF documentation provides critical insights into how Operators interact with the Kubernetes API machinery:

- **CNCF Tag App Delivery:** Visit the [official CNCF Operator Whitepaper](https://tag-app-delivery.cncf.io/whitepapers/operator/) for the complete architectural specification.
- **Designing Distributed Systems:** Complement your learning by reading **Designing Distributed Systems** by Brendan Burns, which covers the sidecar and controller patterns that underpin the Operator framework.

When implementing your first `client-go` controller, keep in mind that the **WorkQueue** is your buffer. Informers watch for changes, Listers read from the cache, and the WorkQueue ensures that your reconciliation logic processes these events in a FIFO (First-In-First-Out) manner. Never attempt to bypass the Informer cache to read directly from the `kube-apiserver`, as this introduces latency and unnecessary load on the control plane.

Suggested Reading

The following resources provide additional depth on topics covered in this course. While not required, they are valuable references that complement specific weekly modules.

Kubernetes Security and Observability

Brendan Creane and Amit Gupta (O'Reilly, 2021)

This book is a practical guide to implementing robust security postures and effective monitoring strategies in Kubernetes clusters. It covers network policies, RBAC design, runtime security, and observability patterns in detail. It serves as an excellent companion to Week 3.2, where you will study networking, authentication, authorization, and cluster hardening.

Designing Distributed Systems

Brendan Burns (O'Reilly, 2024)

This book explores common distributed systems patterns — sidecar, ambassador, adapter — and shows how they are implemented with Kubernetes. It provides a strong conceptual foundation for understanding the distributed challenges covered in Week 2.2, including idempotency, leader election, and controller sharding. # Suggested Reading

Suggested Reading

The recommended literature for the Kubernetes Developer Learning Path focuses on securing, observing, and designing scalable distributed applications:

- **Kubernetes Security and Observability** (Creane & Gupta): Provides essential strategies for hardening cluster security and implementing effective observability practices.
- **Designing Distributed Systems** (Burns): Explores common, reusable software patterns designed to build scalable and reliable systems using Kubernetes primitives.

Suggested Reading

Kubernetes Security and Observability (Creane, Gupta)

Kubernetes Security and Observability

As Kubernetes clusters move from experimental projects to production-grade infrastructure, the complexity of securing and monitoring the environment grows exponentially. This module explores the core concepts of Kubernetes security and observability, drawing from the architectural insights provided by Brendan Creane and Amit Gupta in **Kubernetes Security and Observability**.

Security Foundations in Kubernetes

Securing a Kubernetes environment requires a defense-in-depth approach. Unlike traditional virtualized environments, Kubernetes requires security controls to be applied at multiple layers: the container image, the node, the network, and the API server.

The Security Architecture

The `kube-apiserver` acts as the central gatekeeper. As noted in the resource model, every request undergoes a strict lifecycle: * **Authentication:** Verifying who is making the request (X.509, OIDC, Tokens). * **Authorization:** Determining if the identity has the permissions for the requested action (RBAC, Node Authorization, ABAC). * **Admission Control:** Validating or mutating the request before it is persisted in `etcd`.

Policy Enforcement

A critical aspect of securing the cluster is moving beyond native RBAC toward automated governance. This involves: * **Validating Webhooks:** Intercepting requests to ensure they comply with organizational standards. * **Policy Engines:** Utilizing tools like OPA Gatekeeper or Kyverno to decouple security policies from application code, allowing for centralized, declarative governance.

Observability: Understanding Cluster State

Observability in Kubernetes is more than just logging; it is the ability to understand the internal state of the system by examining its external outputs (metrics, logs, and traces).

The Three Pillars

1. **Metrics:** Quantifiable data (e.g., CPU/Memory usage, request latency, error rates). These are typically gathered via the Metrics Server or Prometheus and provide the "what" is happening in the system.
2. **Logs:** Immutable records of discrete events (e.g., application startup, connection failures). Logs

provide the "why" something happened.

3. **Traces:** Distributed tracing allows engineers to follow a request as it traverses microservices, which is essential for troubleshooting performance bottlenecks in complex, distributed systems.

Hands-on Activity: Analyzing Security Contexts

In this exercise, we will apply security hardening principles by restricting a Pod's capabilities through the `SecurityContext`.

Create a restricted pod manifest (`restricted-pod.yaml`):

```
apiVersion: v1
kind: Pod
metadata:
  name: security-hardened-pod
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    fsGroup: 2000
  containers:
  - name: nginx
    image: nginx:latest
    securityContext:
      allowPrivilegeEscalation: false
      capabilities:
        drop:
          - ALL
```

Steps to apply and verify:

1. **Apply the configuration:**

   ```
   kubectl apply -f restricted-pod.yaml
   ```

2. **Verify security constraints:** Attempt to enter the container and perform a privileged action (like changing system time) to confirm the `capabilities` block effectively dropped `ALL` privileges.

   ```
   kubectl exec -it security-hardened-pod -- date -s "2025-01-01"
   # Expected output: Operation not permitted
   ```

Troubleshooting and Best Practices

- **Least Privilege:** Always grant only the permissions necessary for a controller or user to function. Use `RoleBindings` scoped to specific namespaces rather than `ClusterRoleBindings` whenever possible.
- **Visibility:** Ensure your observability stack (Prometheus/Grafana) has access to API audit logs.

Audit logs are the single most important tool for forensic analysis in a compromised cluster.

- **Immutable Infrastructure:** Treat your nodes and containers as immutable. If a container is misbehaving, replace it rather than patching it in place.

Recommended Reading

- Creane, Brendan, and Amit Gupta. **Kubernetes Security and Observability**. O'Reilly Media, Inc., 2021.

Designing Distributed Systems (Burns)

Designing Distributed Systems: Foundations for Kubernetes Controllers

In the context of Kubernetes development, transitioning from a single-instance controller to a resilient, distributed operator requires a fundamental shift in how we perceive state. Brendan Burns' **Designing Distributed Systems** provides the theoretical bedrock for the patterns we implement in `client-go` and `controller-runtime`.

To build robust Kubernetes controllers, you must move away from "transactional" thinking—where you expect a command to succeed immediately—and move toward "eventual consistency."

Core Distributed Principles for Kubernetes

When scaling controllers or ensuring high availability, we rely on specific patterns to manage state across distributed nodes:

- **The Sidecar Pattern:** Extending the functionality of a primary container without modifying its source code. In controller design, this is mirrored in how we use `Admission Webhooks` to intercept and modify resource requests before they reach `etcd`.
- **The Ambassador Pattern:** Offloading communication logic (like retries or mTLS) to a sidecar, similar to how Service Mesh architectures (like Istio or Linkerd) handle traffic management outside the application logic.
- **The Work Queue Pattern:** Essential for `client-go` informers. By decoupling the observation of state (events) from the processing of state (the Reconcile loop), we ensure that a spike in API traffic does not crash our controller.
- **The Replicated Worker Pattern:** The foundation of controller sharding. When you run multiple instances of a controller, you must ensure they don't fight over the same resources.

Implementing Distributed State Machines

The transition from a basic controller to a distributed system is defined by how you handle conflicts and state ownership.

1. Idempotency and Atomicity

In a distributed system, you cannot assume a request will only arrive once. Your `Reconcile` function must be **idempotent**: * **Definition:** An operation is idempotent if performing it multiple times yields the same result as performing it once. * **Application:** Always verify current cluster state

before applying changes. If the resource is already in the desired state, the `Reconcile` loop should exit gracefully without making unnecessary API calls.

2. Leader Election (Leases)

Running multiple replicas of a controller for high availability (HA) introduces a "Split Brain" risk, where two controllers attempt to modify the same resource simultaneously. * **The Mechanism:** Kubernetes uses the `Lease` resource (within the `coordination.k8s.io` API group) to implement distributed locking. * **The Workflow:** 1. Controllers compete to acquire a `Lease` object. 2. The controller that holds the lock is the "Leader." 3. Other instances wait or remain in a "standby" mode. 4. If the leader fails to renew the lease, a new leader is elected.

3. Controller Sharding

When a single controller becomes a bottleneck (e.g., managing thousands of Ingress resources), you implement **sharding**. By partitioning the work—where each replica is responsible for only a subset of resources (e.g., resources with a specific label or hash)—you increase throughput while maintaining individual controller simplicity.

Hands-on Concept: The Distributed Reconcile Logic

When writing your own controller (as outlined in our 2.2 objectives), apply these design principles:

```
// Simplified pseudo-code for a Reconcile loop demonstrating distributed principles
func (r *Reconcile) Reconcile(req ctrl.Request) (ctrl.Result, error) {
    // 1. Fetch the latest state (Do not rely on cached memory)
    instance := &v1.MyResource{}
    err := r.Get(ctx, req.NamespacedName, instance)

    // 2. Idempotent logic: Check if work is actually needed
    if instance.Status.Processed == true {
        return ctrl.Result{}, nil
    }

    // 3. Perform the work
    // Ensure this call is atomic or repeatable
    err = r.performAction(instance)

    // 4. Update Status (Optimistic Locking)
    // Always use Patch/Update to ensure we don't overwrite changes made by other
controllers
    return ctrl.Result{}, err
}
```

Key Takeaways for the Developer

- **Design for Failure:** Assume the network is unreliable and the API server may reject your request. Always use exponential backoff in your workqueues.
- **Respect the State:** Kubernetes is the source of truth. Your controller is merely a tool to drive

the cluster toward a desired configuration.

- **Leverage Existing Patterns:** Don't reinvent locks; use `client-go` leaderelection packages that utilize the `Lease` API to keep your implementation clean and idiomatic.

For further exploration, **Designing Distributed Systems** by Brendan Burns is essential reading to understand the transition from standalone scripts to production-grade, distributed Kubernetes operators.

Appendix

Resources

Download Course PDF

Glossary

API Server (kube-apiserver)

The central management component of the Kubernetes control plane. It exposes the Kubernetes API, validates and configures data for API objects, and serves as the front end for the cluster's shared state.

CRD (Custom Resource Definition)

A mechanism for extending the Kubernetes API by defining new resource types without building a custom API server. CRDs allow users to create and manage custom objects using standard Kubernetes tooling.

Controller

A control loop that watches the state of a cluster through the API server and makes changes to move the current state toward the desired state. Controllers are the core automation mechanism in Kubernetes.

Controller-runtime

A set of Go libraries for building Kubernetes controllers and operators. It provides high-level abstractions over client-go, including the Reconcile loop pattern, and is used by kubebuilder.

client-go

The official Go client library for interacting with the Kubernetes API. It provides typed clients, dynamic clients, discovery, informers, listers, and workqueues.

DeltaFIFO

A queue implementation in client-go that stores deltas (changes) for Kubernetes objects. It feeds events to the Informer's event handlers in the order they were received.

etcd

A distributed, consistent key-value store used as the backing store for all Kubernetes cluster data. It provides strong consistency guarantees through the Raft consensus algorithm.

Finalizer

A key on a Kubernetes resource that prevents the resource from being deleted until the controller responsible for the finalizer has completed its cleanup logic. Finalizers enable graceful teardown of dependent resources.

GVK (GroupVersionKind)

A tuple that uniquely identifies the type of a Kubernetes API object by its API group, version, and

kind. For example, `apps/v1 Deployment` has group `apps`, version `v1`, and kind `Deployment`.

GVR (GroupVersionResource)

A tuple that uniquely identifies a Kubernetes API resource endpoint by its API group, version, and resource name. GVR is used for RESTful URL construction, while GVK is used for object type identification.

Informer

A client-go component that watches for changes to Kubernetes resources and maintains an in-memory cache. Informers reduce load on the API server by serving reads from cache and delivering events through handlers.

KAL (kube-api-linter)

A linter for Kubernetes API types that enforces API conventions and best practices. It checks Go struct definitions for CRDs against the Kubernetes API guidelines.

Lease

A lightweight Kubernetes resource used to implement distributed coordination primitives such as leader election and heartbeating. Leases allow controllers to claim and renew ownership of a coordination lock.

Leader Election

A mechanism by which multiple replicas of a controller elect a single active instance (the leader) to perform work, while the others stand by. It is implemented using Leases or ConfigMaps in Kubernetes.

Lister

A client-go component that reads Kubernetes objects from the Informer's local cache rather than making API server calls. Listers provide fast, low-latency reads for controller logic.

OCC (Optimistic Concurrency Control)

A strategy used by the Kubernetes API server to detect conflicting writes. Each resource carries a `resourceVersion` field; updates that specify a stale version are rejected, forcing the client to re-read and retry.

Operator

A pattern for packaging, deploying, and managing a Kubernetes application using custom resources and custom controllers. Operators encode domain-specific operational knowledge into software that runs on the cluster.

Reconcile Loop

The core pattern in controller-runtime where a controller receives a request for a specific object and compares the current state to the desired state, taking action to converge them. Each invocation should be idempotent.

Reflector

A client-go component that watches the Kubernetes API server for a specific resource type and pushes changes into a DeltaFIFO queue. It is the first stage of the Informer pipeline.

Scheme

A registry in the Kubernetes API machinery that maps Go types to GVKs and vice versa. The Scheme is used for serialization, deserialization, and defaulting of API objects.

Server-Side Apply (SSA)

An API server feature that tracks field ownership per manager, enabling safe concurrent updates by multiple controllers. SSA uses Apply operations instead of traditional Update or Patch, reducing merge conflicts.

SharedInformerFactory

A factory in client-go that creates and manages shared Informers for multiple resource types. Sharing Informers across controllers reduces the number of API server watch connections.

Spec

The section of a Kubernetes resource that declares the desired state. Users and controllers write to the Spec to express intent; controllers then reconcile the cluster to match.

Status

The section of a Kubernetes resource that reports the observed state as determined by controllers. Status is typically updated by the controller that manages the resource, not by end users.

Workqueue

A rate-limited, de-duplicating queue in client-go used by controllers to process events. Items are added by Informer event handlers and consumed by worker goroutines in the controller's reconcile loop.